

```
In [ ]: from IPython.display import display, HTML

display(HTML("""
<div style="
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    height: 90vh;
    text-align: center;
">
    <h1 style="margin-bottom: 40px;">
        Final Project – Predicting Youtube Virality
    </h1>
    <p style="font-size: 18px; margin-bottom: 80px;">
        Content Creator Attributes
    </p>
    <p style="font-size: 16px; margin-bottom: 40px;">
        Max Chalekson, Eddie Deng, Selin Altiparmak
    </p>
    <p style="font-size: 16px; margin-bottom: 10px;">
        Week 8 – MSDS 422
    </p>
    <p style="font-size: 16px; margin-bottom: 80px;">
        Practical Machine Learning
    </p>
    <p style="font-size: 16px;">
        February 2026
    </p>
</div>
"""))
```

# Final Project - Predicting Youtube Virality

## Content Creator Attributes

Max Chalekson, Eddie Deng, Selin Altiparmak

Week 8 – MSDS 422

Practical Machine Learning

February 2026

## MSDS 422 Final Project — Integrated Final Report Notebook

### Predicting YouTube Video Virality Using Content, Creator, and Engagement Attributes

This notebook is the unified final deliverable for executive and technical

audiences. It includes:

- Full final report narrative (English)
- Complete methodology, assumptions, and model evaluation details
- Chicago author-date style references
- Aggregated project code from all core notebooks:
  1. `YT-pipeline.ipynb`
  2. `EDA.ipynb`
  3. `EDA_enriched_API.ipynb`
  4. `modeling_4_models.ipynb`

Note: Some code blocks (especially API enrichment and image processing) may require local environment variables, valid API credentials, and sufficient runtime resources.

## Final Report (Executive + Technical)

# MSDS 422 Final Project — Final Report

## Predicting YouTube Video Virality Using Content, Creator, and Engagement Attributes

**Team:** Selin Altiparmak, Max Chalekson, Eddie Deng

**Course:** MSDS 422 – Machine Learning

**Dataset:** YouTube Videos Data for ML and Trend Analysis (Kaggle)

---

### 1. Executive Summary

The objective of this project was to create an end-to-end machine learning process to be able to forecast the "virality" of videos on YouTube utilizing a multiple-view feature set: content metadata, technical/video features, creators/channels attributes, and optionally derived from thumbnails of visual proxy attributes. The workflow includes data enhancement using the YouTube Data API, exploratory data analysis, feature engineering, and training/evaluation of four different machine learning models (Multi Layer Perceptron, Elastic Net, Random Forest, and K-Nearest Neighbors). The **Random Forest Model**, after being optimized for performance, showed itself to be the best performing model of those considered, accounting for about **62% of the variation** in log

transformed view counts (RMS error=1.507;  $R^2=0.621$ ). We also found that **social capital at the level of the creator**, specifically subscriber count, channel view history, and average number of views per video, are the most significant factors determining the number of views, while brightness and colorfulness of the thumbnail provide additional but quantifiable signals. Additionally, we deliberately chose not to utilize likes and comments as predictors when forecasting views so as to prevent **target leakage** and to allow for **use before or immediately after a post has been published**. The report is written for an executive and will include a comparison of the models, consideration of the potential business impacts, lessons learned, and suggestions regarding **deployment strategy**, next steps, and other third-party data sources that can potentially enhance the results.

---

## 2. Research Problem and Research Objectives

### 2.1 Research Problem

YouTube has become one of the most powerful digital media platforms with respect to video visibility and user engagement. The level of engagement (e.g., view counts) of users on YouTube determines not only the amount of money an author receives but also the effectiveness of advertisements placed on YouTube and the dissemination of information within online communities. Because YouTube is the largest repository of user-generated content on the web, there is an increasing need to understand what types of characteristics (i.e., authors, videos) lead to popular or "viral" content.

Predicting which videos will receive high levels of engagement is still a difficult problem; however, engagement can be affected by a number of factors including, but not limited to, author reputation, content, visual design of the video, and other technical aspects of the video itself. Identifying those factors most likely to affect engagement can help both authors of content (for optimizing content creation strategies), and companies wishing to advertise (for optimizing advertising budgets) create effective marketing campaigns. Therefore, the purpose of this study is to investigate if video virality can be predicted using metadata and attributes related to the author of a video that are available either at or shortly after the time of publishing.

### 2.2 Research Objectives

The main goal of this research is to determine which attributes of YouTube videos and/or their authors are most closely related to a video's level of engagement

(view counts). More specifically, this study will investigate which attributes of the author (creator-level attributes), characteristics of the content (content features), attributes of the visual design (visual characteristics), and technical attributes of the video (technical video properties) have the most significant association with viral outcomes.

Another research objective of this study is to compare and contrast the impact of these different categories of attributes on engagement outcomes, based on how they contribute to models of various types. By comparing how author reputation, attributes of thumbnails, text-based metadata, and technical video attributes contribute to models of various types, we can determine how the different categories of predictors influence engagement outcomes.

In addition, this study is interested in assessing which predictors are relevant before **large numbers of people engage with the video**. In order to accomplish this goal, like and comment counts were **intentionally excluded from the predictive model** so that all predictions could be made using information that would reasonably be available prior to the point at which a large number of people engaged with the video.

The final research objective of this study was to create a **fully reproducible and automated analytical pipeline** that allows us to collect, process, transform, and model data about YouTube videos. This pipeline is designed to allow for systematic exploration of various machine learning models, and it will enable us to provide actionable recommendations to support data-driven decision making for content creators and stakeholders in the digital media industry.

## 2.3 Unit of Analysis and Key Assumptions

For this study, the unit of analysis is the **unique YouTube video**, and each observation in our dataset represents a specific `video_id`. Therefore, all predictor variables and engagement metrics are uniquely tied to a single video instance.

Our main outcome measure is the **total number of views received by each video**, which is transformed into `log1p(views)` so as to account for the very long right tail (i.e., the skewed nature) common to many online engagement measures. The log transformation of the dependent variable addresses issues of heteroscedasticity and reduces the effect of extreme outliers, allowing the statistical models used in this study to better estimate the relationships among the independent variables.

To prevent methodological problems, this study will avoid **target leakage**. We are excluding variables like likes and comments from the set of predictor variables when building our models of view counts because these represent audience responses to the video that are generated after the video is published. Therefore, by excluding these variables, the predictive framework developed in this study will be applicable to **both pre-publication and early post-publication scenarios**, when no audience response has occurred.

---

### 3. Literature Review

The results from the exploratory data analysis provided in this study show that many structural patterns in YouTube user engagement are consistent with previous studies on how users spread digital content (i.e., virality). This was due to the fact that the dataset was built using an automated API enrichment pipeline that added information about creators, viewer interactions with videos, and other relevant data for each video observation. Through the enrichment process each video could be described by multiple dimensions of predictors which included, but were limited to; creator reputation, video metadata, a record of all past viewer interaction with the video, and visual aspects of thumbnails (e.g., brightness and colorfulness).

Many theoretical arguments related to what drives online video virality have been explored in the literature, and these theoretical arguments are supported by the findings of the exploratory data analysis performed in this study.

#### 3.1 Social Capital and Channel-Level Influence

Strongest relationship among all those we observed is the association between creator size and video engagement. We used a hexbin density visualization to compare log-transformed subscriber count and log-transformed view count, and the resulting plot showed an upward trend, indicating that videos posted by larger channels tend to be viewed more often. Density of observations in the hexbin density plot is greatest between  $\log_{10}(\text{subscriber\_count}) = 2-6$  and  $\log_{10}(\text{video\_views}) = 3-7$ ; therefore, there is a range of creators with moderate engagement levels. However, we found no videos with view counts greater than  $\log_{10}(\text{video\_view\_count}) > 15$  except among channels with  $\log_{10}(\text{subscriber\_count}) > 10$ ; thus, there is a strong structural advantage for creators of large channels.

Our findings are consistent with those reported by Hussain et al. (2024), who

found that "Subscriber Count consistently exhibits a significant positive relationship with engagement metrics, including views, likes, and comments." Hussain et al. (2024) argued that creators with large subscriber bases benefit from their audiences' accumulated networks that amplify early visibility and enhance the likelihood of being promoted by the algorithm. Our correlation analysis confirmed the existence of this relationship: The correlation matrix displays a positive correlation between log-transformed subscriber count and log-transformed video views (approximately 0.47); similarly, log-transformed subscriber count is correlated with other engagement indicators, including likes and comments (approximately 0.3).

In addition to the relationship between log-transformed subscriber count and log-transformed video views, the correlation matrix also displays strong internal correlations among creator-level variables. For example, the correlation matrix displays a correlation between log-transformed channel view count and log-transformed subscriber count of approximately 0.9; this suggests that channels with a large view count also have a large subscriber base. Thus, creators of established channels tend to cluster in terms of both audience size and historical view volume. This structural clustering of creator metrics provides empirical support for the theoretical construct of creator social capital, which proposes that creators build social capital through their accumulated visibility and audience loyalty.

## 3.2 Category-Level Engagement Patterns

We also found that category has an effect on engagement patterns. Category distribution analysis demonstrated that the People & Blogs category (n=3,946 videos) has the largest number of videos in the dataset, followed by the Music category (n=2,966 videos) and the Entertainment category (n=2,252 videos). Together, these three categories make up approximately 40% of the dataset.

Although engagement patterns are generally consistent across categories, there is considerable variation. Views distribution boxplots demonstrate that categories like Music, Gaming and Entertainment have both higher median view counts and wider variability in view counts compared to informational categories like Education. Specifically, music videos exhibit many high end outliers with view counts well above the median for all categories; furthermore, some music videos had  $\log_{10}(\text{view\_count})$  values greater than 15, indicating very large raw view counts.

These findings are consistent with those reported by Baravkar et al. (2023) who noted that "Entertainment-oriented content tends to propagate more easily via

recommendation algorithms as a result of its emotional appeal and shareability." Our results support this claim: categories like Music and Entertainment have both larger standard deviation and more extreme values compared to more informational categories like Education.

A second interesting pattern emerged in the median creator size by category plot. Categories such as News & Politics and Education have median subscriber counts in excess of log-scale values of 100, implying that the creators of content in these domains produce content in larger channels. In contrast, categories such as People & Blogs and Comedy have median subscriber counts closer to log-scale values of 10–20, suggesting that there are many smaller creators in these categories.

Notwithstanding the fact that People & Blogs is the most common category in the dataset, and is therefore composed of a large number of smaller creators, the volume of content produced by creators in this domain is greater than in categories such as Education or News & Politics. Therefore, although there is considerable heterogeneity in platform participation, both large professional channels and smaller independent creators exist within the same ecosystem.

### 3.3 Thumbnail Salience Features

Koh and Cui (2022) have shown that "Thumbnail brightness and colorfulness can significantly improve click-through-rates as they enhance visual salience." They argue that thumbnails act as key decision points when users browse video recommendations.

Thumbnail brightness and colorfulness are two interpretable proxy measures for visual salience that we included in our dataset. The correlation matrix shows relatively small correlations between these variables and view counts (close to 0.02–0.03). Although the magnitude of the correlations between these variables and view counts is small, the results do not contradict prior findings. Rather, the results suggest that visual attributes may affect click-through-rates, but may not directly affect total view accumulation, which is affected by a host of other network effects.

In a subsequent paper, Peng and Bainbridge (2026) emphasized the importance of image memorability, arguing that "Visually memorable thumbnails are associated with higher comment engagement and interaction." They suggested that visual features may interact with other content attributes in non-linear ways. Since the current dataset contains simple visual proxies rather than deep visual embeddings, the variables used in this study may only partially capture the



perceptual effects discussed in prior studies.

### 3.4 Video Technical Attributes

We also analyzed technical attributes, including video resolution and encoding quality, during the exploratory phase of this study. Quality Tier Distribution demonstrates that 480p videos constitute the largest group of videos in the dataset (approximately 6,900 videos), followed by 1080p+ videos (approximately 4,300 videos). Lower quality tiers such as 360p occur less frequently.

Surprisingly, Mean Views by Quality Tier analysis demonstrates that 480p videos have the largest average views (approximately 87,000), followed by 1080p+ videos with approximately 80,000 average views, whereas 720p videos have much lower mean engagement (approximately 7,000 views). Therefore, the relationship between video quality and engagement success is not determined solely by video production quality.

These findings are consistent with those reported by Konukoğlu and Özman (2025) who reported that "Technical attributes such as video duration and resolution demonstrate limited predictive value once creator-level factors are considered." Our correlation matrix also demonstrates that variables such as duration, frame rate, bitrate, and resolution are each correlated with log-transformed view counts by less than 0.1, indicating little to no linear relationship.

### 3.5 Engagement Metrics and Structural Predictor Variables

Correlation Matrix also demonstrates that the correlation among engagement metrics themselves are strong. For example, the correlation between log-transformed likes and log-transformed comments is greater than 0.8; log-transformed likes and log-transformed comments are each also strongly correlated with log-transformed views (approximately 0.7–0.84). These strong correlations demonstrate the co-occurrence of engagement behaviors: Videos that attract many viewers also tend to attract many likes and comments.

Adam et al. (2023) and Jing et al. (2024) demonstrated that adding multiple engagement metrics to a machine learning model can significantly improve predictive accuracy. They demonstrated that engagement metrics can serve as indicators of audience reaction to online content and can help explain the diffusion mechanisms underlying online content.

Similar to the findings of Adam et al. (2023) and Jing et al. (2024), Gaur et al.

(2024) demonstrated that adding metadata features — including titles, descriptions, and other textual contexts — can predict video popularity. Machine learning models applied to YouTube datasets have demonstrated that combining engagement metrics with metadata improves the accuracy of modeling video popularity and user reaction (Endait et al., 2025). We included features such as title length and metadata-derived features in the dataset to capture additional contextual information that might influence viewer behavior.

## 3.6 Winner-Take-Most Dynamics in Online Attention

The exploratory analysis also revealed the highly skewed distribution of online attention in the dataset. Views distribution plots display numerous extreme outliers, suggesting that a few videos attract very large view counts, while most others receive modest view counts. This heavy-tailed distribution is consistent with the winner-take-most dynamics that are common in digital platforms.

Because early engagement advantages can be amplified by the algorithmic promotion of popular content, creating positive feedback loops where popular content attracts even more attention over time. The presence of extreme high-view outliers in categories such as Music and Entertainment illustrate this phenomenon clearly.

Overall the exploratory findings confirm that Youtube video popularity is primarily driven by strcuture factors specifically creator reputation and audience networks and the visual design and content attributes are secondary signals that can affect viewer attention and engagement behavior.

---

In [ ]:

## Reproducibility and Execution Order

Recommended execution order for full pipeline reproduction:

1. Run **YouTube API enrichment pipeline** section.
2. Run **EDA on base dataset** section.
3. Run **EDA on enriched dataset** section.
4. Run **Modeling and evaluation** section.

If `youtube_data_enriched.csv` already exists, you can skip the API enrichment section and directly run EDA/modeling sections.

# Data Preparation / Feature Engineering (code)

This section aggregates code from `YT-pipeline.ipynb`. Markdown context from the source notebook is summarized first, followed by all code cells.

## Source Notebook Context (Condensed)

# YouTube Pipeline Implementation

---

## Purpose

This notebook constructs a large-scale, analysis-ready dataset for studying YouTube virality by enriching a raw list of video URLs/IDs with creator-level, content-level, and engagement metadata from the YouTube Data API. The resulting dataset integrates textual metadata (titles, descriptions, tags), temporal features (video age, duration), creator social capital (subscriber counts, channel history), and engagement signals (views, likes, comments), enabling systematic investigation of which attributes are most strongly associated with viral outcomes.

## Feature scope and design choices

In addition to standard metadata, the pipeline optionally extracts lightweight thumbnail image features (e.g., brightness, colorfulness, resolution) as coarse proxies for visual salience. These features are intentionally limited in complexity and are used only as exploratory controls rather than deep semantic representations. Raw thumbnail images are cached temporarily during processing and discarded after feature extraction to maintain a compact, reproducible dataset.

## Methodological motivation

Consistent with prior work, this notebook prioritizes features that are observable at or near publication time (creator characteristics, metadata, visual proxies) while also retaining post-publication engagement metrics for comparative and explanatory analyses. This design reflects the central research objective: distinguishing predictors that plausibly contribute to virality from signals that

primarily emerge after diffusion has already occurred.

## Output dataset

The final output is a single enriched CSV file containing one row per YouTube video, identified by `video_id` (from the **youtube\_data.csv**), and integrating technical, semantic, social, and engagement-related features. The dataset includes raw video and encoding attributes such as duration, resolution, bitrate, frame rate, and codec, alongside scraped metadata including category, URL, title, description, hashtags, and observed view, like, and comment counts. These fields are augmented with structured YouTube Data API features capturing publication timing (`yt_published_at`), standardized duration (`yt_duration_sec`), content category (`yt_category_id`), language metadata, live-broadcast and audience-designation flags, and validated engagement metrics (`yt_view_count`, `yt_like_count`, `yt_comment_count`). Creator-level social capital is represented through channel identifiers and attributes, including channel age, country, subscriber count, total channel views, and upload volume. Finally, the dataset optionally includes lightweight, interpretable thumbnail-derived visual proxies—such as resolution, mean brightness, and colorfulness—computed from thumbnail URLs without retaining raw image data. Together, these columns produce an analysis-ready table that supports exploratory analysis, statistical modeling, and machine learning approaches for examining the relative influence of content design, creator reputation, and engagement dynamics on YouTube virality.

## Code Chunk 1 — Environment, Credentials, and Pipeline Initialization

```
In [ ]: from dotenv import load_dotenv
import os

ENV_PATH = "/Users/maxchalekson/Northwestern University/Winter-2026/MS

print("File exists?", os.path.exists(ENV_PATH))
loaded = load_dotenv(ENV_PATH, override=True)
print("dotenv loaded?", loaded)
print("Kernel sees key?", bool(os.getenv("YOUTUBE_API_KEY")))
```

```
File exists? False
dotenv loaded? False
Kernel sees key? True
```

## YT-pipeline.ipynb — Code Cell 2

```

In [ ]: from __future__ import annotations

import os
import re
import math
import time
import json
import hashlib
from dataclasses import dataclass
from typing import Optional, Dict, Any, List, Tuple

import pandas as pd
import requests
from tqdm.auto import tqdm

PROJECT_DIR = "/Users/maxchalekson/Northwestern University/Winter-2026
ENV_PATH = os.path.join(PROJECT_DIR, ".env")

def ensure_youtube_api_key(env_path: str = ENV_PATH) -> str:
    key = os.getenv("YOUTUBE_API_KEY")

    if not key:
        try:
            from dotenv import load_dotenv
            load_dotenv(env_path, override=True)
        except Exception:
            pass

    key = os.getenv("YOUTUBE_API_KEY")

    if not key:
        raise ValueError(
            f"Missing YouTube API key. Create {env_path} with:\n"
            "YOUTUBE_API_KEY=AizaSy..."
        )

    return key.strip().strip('"').strip("'")

_YT_ID_RE = re.compile(r"(?:v=|\/shorts\/|youtu\.be\/|\/embed\/)([A-Za

def extract_video_id(url: str) -> Optional[str]:
    if not isinstance(url, str) or not url.strip():
        return None
    m = _YT_ID_RE.search(url)
    return m.group(1) if m else None

def chunked(lst: List[str], n: int) -> List[List[str]]:

```

```

    return [lst[i : i + n] for i in range(0, len(lst), n)]

def safe_int(x: Any) -> Optional[int]:
    try:
        return int(x) if x is not None else None
    except Exception:
        return None

@dataclass
class YouTubeAPI:
    api_key: str
    base_url: str = "https://www.googleapis.com/youtube/v3"
    session: Optional[requests.Session] = None
    sleep_s: float = 0.1

    def _sess(self):
        if self.session is None:
            self.session = requests.Session()
        return self.session

    def _get(self, path: str, params: Dict[str, Any]) -> Dict[str, Any]:
        params = dict(params)
        params["key"] = self.api_key

        url = f"{self.base_url}/{path}"

        r = self._sess().get(url, params=params, timeout=30)

        if r.status_code != 200:
            raise RuntimeError(f"YT API error {r.status_code}: {r.text}")

        time.sleep(self.sleep_s)

        return r.json()

    def videos_list(self, video_ids: List[str], parts: str):
        out = []

        for batch in tqdm(chunked(video_ids, 50), desc="Fetching video
            data = self._get(
                "videos",
                {"part": parts, "id": ",".join(batch), "maxResults": 5
            )

            out.extend(data.get("items", []))

        return out

    def channels_list(self, channel_ids: List[str], parts: str):
        out = []

```

```

        for batch in tqdm(chunked(channel_ids, 50), desc="Fetching cha
            data = self._get(
                "channels",
                {"part": parts, "id": ", ".join(batch), "maxResults": 5
            )

            out.extend(data.get("items", []))

        return out

def iso8601_duration_to_seconds(dur: Optional[str]) -> Optional[int]:

    if not dur:
        return None

    h = m = s = 0

    mobj = re.match(r"^PT(?: (\d+)H)?(?: (\d+)M)?(?: (\d+)S)?$", dur)

    if not mobj:
        return None

    if mobj.group(1):
        h = int(mobj.group(1))
    if mobj.group(2):
        m = int(mobj.group(2))
    if mobj.group(3):
        s = int(mobj.group(3))

    return h * 3600 + m * 60 + s

def pick_best_thumbnail(thumbnails):

    order = ["maxres", "standard", "high", "medium", "default"]

    for k in order:
        if k in thumbnails and "url" in thumbnails[k]:
            return thumbnails[k]["url"], k

    return None, None

def enrich_youtube_csv(
    csv_path: str,
    api_key: Optional[str] = None,
    video_id_col="video_id",
    url_col="url",
    out_csv_path=None,
    add_thumbnail_features=False,

```

```
thumbnail_cache_dir="thumb_cache",
):

api_key = api_key or ensure_youtube_api_key()

print("Using API key prefix:", api_key[:8], "...")

df = pd.read_csv(csv_path)

if video_id_col not in df.columns:
    df[video_id_col] = pd.NA

if url_col in df.columns:

    missing = df[video_id_col].isna() | (
        df[video_id_col].astype(str).str.strip() == ""
    )

    df.loc[missing, video_id_col] = df.loc[missing, url_col].apply

df[video_id_col] = df[video_id_col].astype(str).str.strip()

video_ids = df[video_id_col].dropna().unique().tolist()

if not video_ids:
    raise ValueError("No video IDs found.")

yt = YouTubeAPI(api_key)

video_items = yt.videos_list(
    video_ids,
    parts="snippet,contentDetails,statistics,status",
)

rows = {}

for item in video_items:

    vid = item["id"]

    snippet = item.get("snippet", {})
    stats = item.get("statistics", {})
    content = item.get("contentDetails", {})
    status = item.get("status", {})

    thumb_url, thumb_quality = pick_best_thumbnail(
        snippet.get("thumbnails", {})
    )

    rows[vid] = {
        "yt_title": snippet.get("title"),
        "yt_published_at": snippet.get("publishedAt"),
```



```

        "yt_duration_sec": iso8601_duration_to_seconds(
            content.get("duration")
        ),
        "yt_view_count": safe_int(stats.get("viewCount")),
        "yt_like_count": safe_int(stats.get("likeCount")),
        "yt_comment_count": safe_int(stats.get("commentCount")),
        "yt_channel_id": snippet.get("channelId"),
        "yt_thumb_url": thumb_url,
        "yt_thumb_quality": thumb_quality,
    }

    video_df = pd.DataFrame.from_dict(rows, orient="index")
    video_df.index.name = video_id_col

    df = df.merge(video_df, left_on=video_id_col, right_index=True, how="left")

    if out_csv_path:
        df.to_csv(out_csv_path, index=False)

    return df

```

## YT-pipeline.ipynb — Code Cell 3

# Exploratory Data Analysis (Base Dataset Code)

This section aggregates code from `EDA.ipynb`. Markdown context from the source notebook is summarized first, followed by all code cells.

## 4. Exploratory Data Analysis

### 4.1 Data Sources and Structure

A total of **17,589 observations with 17 variables** were collected in this research from the *youtube\_data.csv* file that includes a number of technical and metadata parameters for randomly selected YouTube videos. There is one row per video with each variable being one piece of information about the video. Technical parameters of interest included video duration, bitrate, height and width, frame rate, codec, etc. Metadata parameters of interest included content category, URL, title, description, hashtags, etc. Additionally, technical and metadata parameters related to viewer engagement, including views, likes and comments were collected.

To increase the ability to analyze the data using multiple approaches, a data enrichment process was applied using the **YouTube Data API v3**, to create a

second dataset entitled *youtube\_data\_enriched.csv*. All 17,589 video records in the *youtube\_data.csv* file were preserved during the enrichment process. However, instead of preserving the same 17 variables from the original dataset, 28 new variables were added to make a total of **45 variables**.

In addition to the original 17 variables, the enriched dataset captures a variety of creator-level and contextual variables, including creator reputation (e.g., `yt_view_count`, `yt_subscriber_count`, `yt_channel_view_count`), which measure the level of engagement and size of the audience associated with a channel; and variables representing supplementary visual and temporal features relevant to analyzing engagement with a video. These include thumbnail-level visual parameters (e.g., `thumb_mean_brightness`, `thumb_colorfulness`); publication metadata (e.g., video publication date, channel creation date, language indicators, country attributes); and other variables relevant to understanding how viewers engage with content on YouTube.

Collectively, the additional variables enable each video record to be examined from a multi-dimensional perspective by utilizing a wide range of predictor variables measuring creator reputation, content metadata, visual presentation, and engagement context.

## 4.2 Data Quality and Profiling

Data quality assessments were performed on both the original dataset and the dataset after the API enrichment process to validate the accuracy of the subsequent analyses. Missing values in the base dataset were most frequently located within the textual metadata fields (i.e., description and hashtags). Most of the new engineered features after the API enrichment process had less than 1% missing values, except for `yt_channel_country` (approximately 88%), which due to the very high level of missingness, it was treated as auxiliary and removed from some of the model specifications.

Additionally, duplicate observations were assessed to confirm that every video has a unique `video_id` and there were no complete row duplicates after the enrichment process, thereby validating that each observation corresponds to a unique video instance.

Exploratory analysis of the distribution of primary engagement metrics indicated that the metrics for views, likes, and comments exhibited right-skew distributions with a large number of videos exhibiting minimal engagement and a smaller number of videos receiving an overwhelming amount of engagement. Log transformations of the engagement metrics helped to stabilize variance and

transform the distributions to be more symmetrical, therefore improving their suitability as input for regression-based machine learning models.

## 4.3 Data Analysis, Visualizations and Data Mining Techniques

To analyze the structure of the dataset and identify significant trends, a combination of exploratory data analysis and visualization techniques were employed. Initially, univariate analysis was performed to assess the distributions of both numerical and categorical variables (e.g., duration, bitrate, resolution, category, and codec type) in the dataset. Extended descriptive statistics (e.g., skewness, kurtosis, and interquartile range) were calculated to determine the degree of spread and normality of the variables. The results of the extended descriptive statistics influenced the decision-making process of whether or not to scale/transform the variables.

Following the assessment of the individual variables, bivariate analysis was employed to investigate the relationship(s) between the variables. A correlation matrix was constructed using the numerical variables to assess the presence of multicollinearity and other structural relationships between the predictor variables. Additionally, clustered correlation visualizations were generated to visualize clusters of variables exhibiting similar correlation patterns. Scatterplots were also created to investigate the relationship(s) between specific pairs of variables (e.g., duration vs. bitrate and title length vs. view count).

Further analysis at the category-level provided additional insight into the differences in engagement associated with various types of content. Primary engagement metrics (i.e., views, likes, and comments) were analyzed by category of content, along with several derived engagement rates (e.g., like rate and comment rate). Several visualization techniques (i.e., box plots and violin plots) were used to provide a graphical representation of the diversity in engagement rates across categories of content.

In addition to the analysis of primary and derived engagement metrics, several additional derived metrics were created to capture additional contextual and behavioral information related to creators and videos. Examples include channel-level metrics (e.g., average views per channel video), temporal metrics (e.g., video age and channel age), and textual proxies (e.g., title length, description length, and hashtag count). These additional derived features were intended to capture aspects of creators' efforts, content presentation, and temporal context that could potentially influence viewers' engagement with videos.

## 4.4 Assumptions and Feature Interactions

There were several modeling assumptions made in developing the analytical framework for this study. First, the total video view count was utilized as a suitable proxy for virality; this represented the overall reach/popularity of a video. Second, the engagement variables exhibited skewed distributions; consequently, the modeling was completed using the logarithm of the engagement variables to enhance statistical stability and to minimize the impact of extreme outliers. Third, the features used in this analysis were assumed to be representative of the platform segment represented by the Kaggle dataset; however, it was acknowledged that the sample does not completely represent all segments of the YouTube ecosystem. Fourth, the modeling framework explicitly did not utilize leakage variables as predictors. For example, likes and comments were not used as predictors for view counts, since they are outcome variables that are determined after a video is published.

Finally, potential feature interactions were also considered when selecting the best modeling approach. As previously mentioned, tree-based models, particularly Random Forest models can capture nonlinear relationships and complex interactions between features without the need for feature engineering. Linear models, such as Elastic Net and instance-based models, such as K-Nearest Neighbors were also included as baselines to compare against. Given the exploratory analysis and previous research, it was expected that there would be interactions between the creator-level features and the visual presentation features. For example, videos from high-subscriber channels may benefit from visually attractive thumbnails more so than videos from low-subscriber channels, representing an example of how the reputation of a creator and the visual presentation of a video may interact to influence viewers' engagement with the video.

## Source Notebook Context (Condensed)

# YouTube Video Data - Exploratory Data Analysis (EDA)

This notebook presents a comprehensive exploratory data analysis of a YouTube video dataset. The analysis proceeds through data quality assessment, univariate and bivariate examination of key variables, derivation of engagement metrics, and investigation of relationships between technical, textual, and engagement-related features. The findings inform methodological choices for subsequent predictive

modelling and provide an empirical basis for understanding content-level determinants of audience engagement on the platform.

## Environment Setup and Data Loading

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', 50)
pd.set_option('display.max_colwidth', 60)

try:
    plt.style.use('seaborn-v0_8-whitegrid')
except (OSError, ValueError):
    try:
        plt.style.use('seaborn-whitegrid')
    except (OSError, ValueError):
        pass
sns.set_palette('husl')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11

df = pd.read_csv('youtube_data.csv')
df_raw = df.copy()

print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f}")
df.head(10)
```

Dataset shape: 17,589 rows × 17 columns  
Memory usage: 13.83 MB

Out[ ]:

|   | video_id    | duration | bitrate | bitrate(video) | height | width | frame<br>rate | fran<br>rate(est |
|---|-------------|----------|---------|----------------|--------|-------|---------------|------------------|
| 0 | --F7dc-_FSI | 180      | 5777    | 5640           | 1920   | 1080  | 25.00         | 25.0             |
| 1 | --cCAD-8Y_U | 930      | 1195    | 1001           | 1280   | 720   | 30.00         | 30.0             |
| 2 | --g2gG8pQ0w | 233      | 3028    | 2833           | 1280   | 720   | 23.98         | 23.9             |
| 3 | -0DR7-voRCU | 562      | 431     | 300            | 320    | 240   | 29.97         | 0.0              |
| 4 | -0Fkp-2EzX0 | 300      | 3087    | 2929           | 1280   | 720   | 23.98         | 23.9             |
| 5 | -0J-a-kKR1M | 135      | 467     | 371            | 320    | 240   | 30.00         | 0.0              |
| 6 | -0bc_-HP6dE | 43       | 634     | 501            | 480    | 360   | 29.97         | 29.9             |
| 7 | -0hjSaYCRnA | 4        | 738     | 605            | 540    | 360   | 29.97         | 0.0              |
| 8 | -0kAY-vAVBc | 228      | 5880    | 5686           | 1920   | 1080  | 25.00         | 25.0             |
| 9 | -0INh-4ZuTE | 268      | 492     | 364            | 640    | 360   | 25.00         | 0.0              |

## EDA.ipynb — Code Cell 2

```
In [ ]: def get_sample(col):
        try:
            if df[col].notna().any():
                return str(df[col].dropna().iloc[0][:40])
        except (IndexError, KeyError):
            pass
        return 'N/A'

schema = pd.DataFrame({
    'Column': df.columns,
    'Dtype': df.dtypes.values,
    'Non-Null': df.count().values,
    'Unique': [df[col].nunique() for col in df.columns],
    'Sample': [get_sample(col) for col in df.columns]
})
schema
```

Out[ ]:

|    | Column           | Dtype   | Non-Null | Unique | Sample                                   |
|----|------------------|---------|----------|--------|------------------------------------------|
| 0  | video_id         | object  | 17589    | 17589  | --F7dc-_FSI                              |
| 1  | duration         | int64   | 17589    | 1320   | 180                                      |
| 2  | bitrate          | int64   | 17589    | 3512   | 5777                                     |
| 3  | bitrate(video)   | int64   | 17589    | 3444   | 5640                                     |
| 4  | height           | int64   | 17589    | 266    | 1920                                     |
| 5  | width            | int64   | 17589    | 172    | 1080                                     |
| 6  | frame rate       | float64 | 17589    | 467    | 25.0                                     |
| 7  | frame rate(est.) | float64 | 17589    | 259    | 25.0                                     |
| 8  | codec            | object  | 17589    | 4      | h264                                     |
| 9  | category         | object  | 17589    | 16     | News & Politics                          |
| 10 | url              | object  | 17589    | 17589  | https://www.youtube.com/watch?v=--F7dc-_ |
| 11 | title            | object  | 17589    | 17515  | السودان ينتفض «أمام السفارة بالقاهرة»    |
| 12 | description      | object  | 16477    | 12773  | حرية سلام وعدالة، الثورة خيار الشعب.. به |
| 13 | hashtags         | object  | 16013    | 15108  | في، عسكري، حكم، حرية، بخصوص، المصري اليو |
| 14 | views            | int64   | 17589    | 4280   | 4078                                     |
| 15 | likes            | int64   | 17589    | 640    | 41                                       |
| 16 | comments         | int64   | 17589    | 281    | 3                                        |

## EDA.ipynb — Code Cell 3

```
In [ ]: missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.DataFrame({'Missing': missing, 'Pct (%)': missing_pct})
missing_df = missing_df[missing_df['Missing'] > 0].sort_values('Missing')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

if len(missing_df) > 0:
    bars = axes[0].barh(missing_df.index, missing_df['Missing'], color='red')
    axes[0].set_xlabel('Missing Count')
    axes[0].set_title('Missing Values by Column')
    axes[0].axvline(x=len(df)*0.05, color='red', linestyle='--', alpha=0.5)
```



```

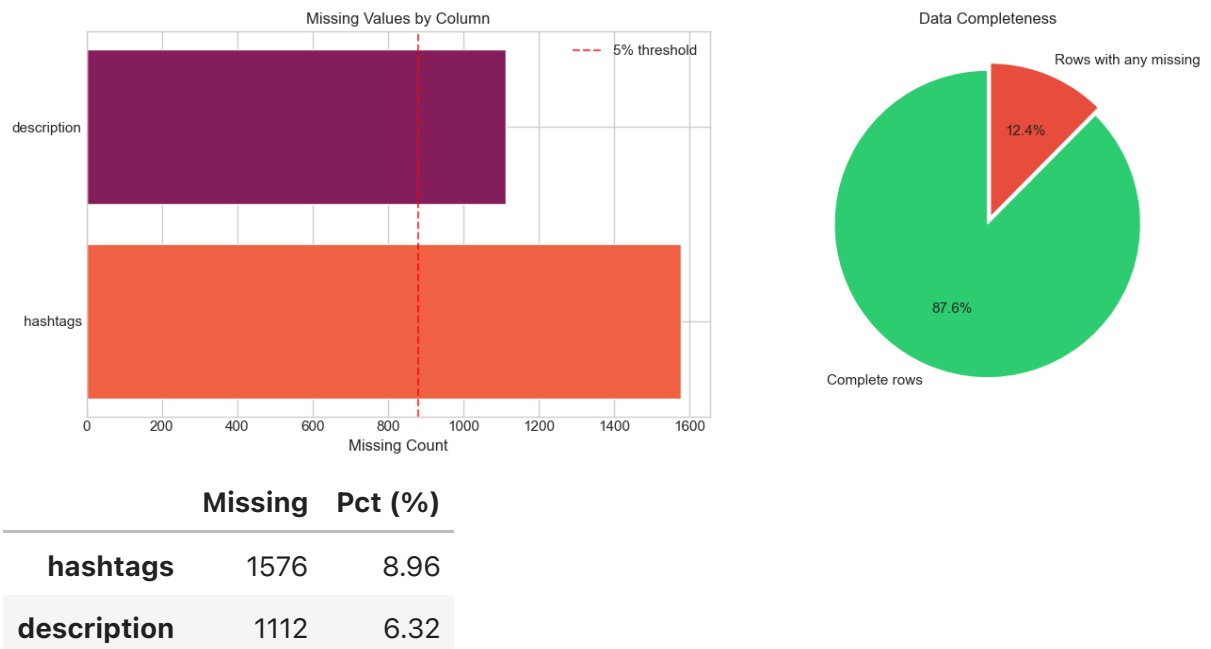
axes[0].legend()

complete = df.dropna().shape[0]
incomplete = len(df) - complete
axes[1].pie([complete, incomplete], labels=['Complete rows', 'Rows with any missing'],
            colors=['#2ecc71', '#e74c3c'], explode=(0, 0.05), startangle=90)
axes[1].set_title('Data Completeness')
else:
    axes[0].text(0.5, 0.5, 'No missing values', ha='center', va='center')
    axes[1].text(0.5, 0.5, '100% Complete', ha='center', va='center')

plt.tight_layout()
plt.show()

if len(missing_df) > 0:
    display(missing_df)

```



## Figure Interpretation

This plot shows how the featured variables vary in this section of the analysis. The main takeaway is the relative differences in level and spread across groups, which indicates meaningful signal for understanding and predicting virality.

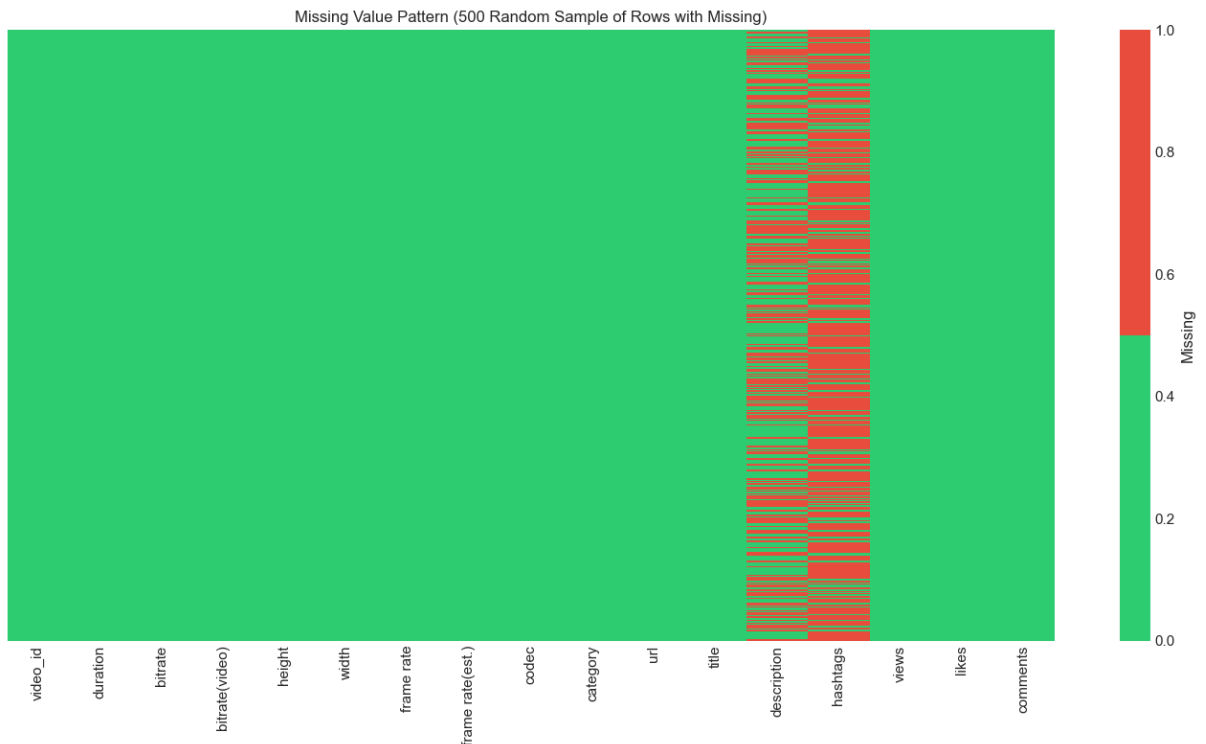
## EDA.ipynb — Code Cell 4

```

In [ ]: from matplotlib.colors import ListedColormap
miss_cmap = ListedColormap(['#2ecc71', '#e74c3c'])
rows_with_missing = df[df.isnull().any(axis=1)]
if len(rows_with_missing) > 0 and len(rows_with_missing) <= 500:

```

```
plt.figure(figsize=(14, min(8, len(rows_with_missing) * 0.02)))
sns.heatmap(rows_with_missing.isnull(), cbar_kws={'label': 'Missing'})
plt.title('Missing Value Pattern (Rows with Any Missing)')
plt.tight_layout()
plt.show()
elif len(rows_with_missing) > 500:
    sample = rows_with_missing.sample(500, random_state=42)
    plt.figure(figsize=(14, 8))
    sns.heatmap(sample.isnull(), cbar_kws={'label': 'Missing'}, cmap=m
    plt.title('Missing Value Pattern (500 Random Sample of Rows with M
    plt.tight_layout()
    plt.show()
```



## Figure Interpretation

This plot shows how the featured variables vary in this section of the analysis.

The main takeaway is the relative differences in level and spread across groups, which indicates meaningful signal for understanding and predicting virality.

## EDA.ipynb — Code Cell 5

```
In [ ]: dup_video_id = df['video_id'].duplicated().sum()
dup_rows = df.duplicated().sum()

print(f"Duplicate video_id: {dup_video_id}")
print(f"Fully duplicate rows: {dup_rows}")

if dup_video_id > 0:
```

```
dup_ids = df[df['video_id'].duplicated(keep=False)][['video_id']].va
print(f"\nSample duplicate video_ids:\n{dup_ids}")
```

Duplicate video\_id: 0  
Fully duplicate rows: 0

## EDA.ipynb — Code Cell 6

```
In [ ]: numeric_cols = ['duration', 'bitrate', 'bitrate(video)', 'height', 'wi
numeric_cols = [c for c in numeric_cols if c in df.columns]

stats = df[numeric_cols].describe().T
stats['skewness'] = df[numeric_cols].skew()
stats['kurtosis'] = df[numeric_cols].kurtosis()
stats['IQR'] = stats['75%'] - stats['25%']
stats['range'] = stats['max'] - stats['min']
stats.round(2)
```

```
Out [ ]:
```

|                         | count   | mean     | std        | min    | 25%   | 50%    | 75%     |   |
|-------------------------|---------|----------|------------|--------|-------|--------|---------|---|
| <b>duration</b>         | 17589.0 | 241.55   | 493.03     | 1.00   | 51.0  | 135.00 | 268.00  | 2 |
| <b>bitrate</b>          | 17589.0 | 1271.35  | 1375.36    | 0.00   | 437.0 | 743.00 | 1293.00 | 2 |
| <b>bitrate(video)</b>   | 17589.0 | 1150.42  | 1351.80    | 0.00   | 326.0 | 632.00 | 1184.00 | 2 |
| <b>height</b>           | 17589.0 | 766.78   | 467.29     | 108.00 | 426.0 | 640.00 | 960.00  | 2 |
| <b>width</b>            | 17589.0 | 504.59   | 262.73     | 88.00  | 320.0 | 480.00 | 720.00  | 1 |
| <b>frame rate</b>       | 17589.0 | 26.47    | 6.04       | 3.75   | 25.0  | 29.97  | 29.97   | 5 |
| <b>frame rate(est.)</b> | 17589.0 | 9.47     | 13.25      | 0.00   | 0.0   | 0.00   | 25.00   | 3 |
| <b>views</b>            | 17589.0 | 68941.58 | 3537491.15 | 0.00   | 38.0  | 152.00 | 800.00  | 4 |
| <b>likes</b>            | 17589.0 | 208.86   | 8477.50    | 0.00   | 0.0   | 1.00   | 4.00    | 8 |
| <b>comments</b>         | 17589.0 | 12.90    | 225.84     | 0.00   | 0.0   | 0.00   | 1.00    | 1 |

## EDA.ipynb — Code Cell 7

```
In [ ]: fig, axes = plt.subplots(2, 1, figsize=(14, 10))

cat_counts = df['category'].value_counts()
colors = plt.cm.Spectral(np.linspace(0.2, 0.9, len(cat_counts)))
bars = axes[0].barh(cat_counts.index, cat_counts.values, color=colors)
axes[0].set_xlabel('Video Count')
axes[0].set_title('Video Count by Category', fontsize=14, fontweight='
for i, (idx, val) in enumerate(zip(cat_counts.index, cat_counts.values
```

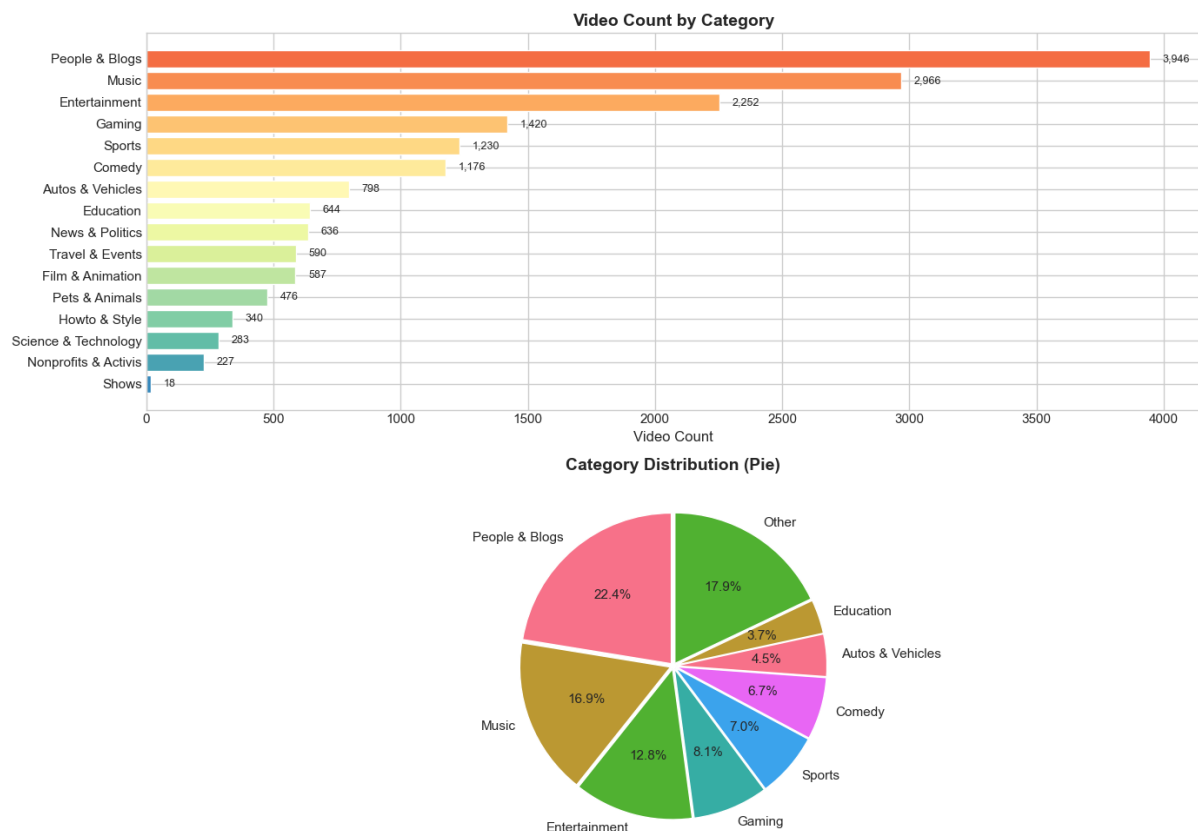
```

axes[0].text(val + 50, i, f'{val:,}', va='center', fontsize=9)
axes[0].invert_yaxis()

top_n = 8
top_cats = cat_counts.head(top_n)
other_count = cat_counts.iloc[top_n:].sum() if len(cat_counts) > top_n
pie_data = list(top_cats.values) + ([other_count] if other_count > 0 else 0)
pie_labels = list(top_cats.index) + (['Other'] if other_count > 0 else [])
axes[1].pie(pie_data, labels=pie_labels, autopct='%1.1f%%', startangle=90)
axes[1].set_title('Category Distribution (Pie)', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This plot shows how the featured variables vary in this section of the analysis. The main takeaway is the relative differences in level and spread across groups, which indicates meaningful signal for understanding and predicting virality.

## EDA.ipynb — Code Cell 8

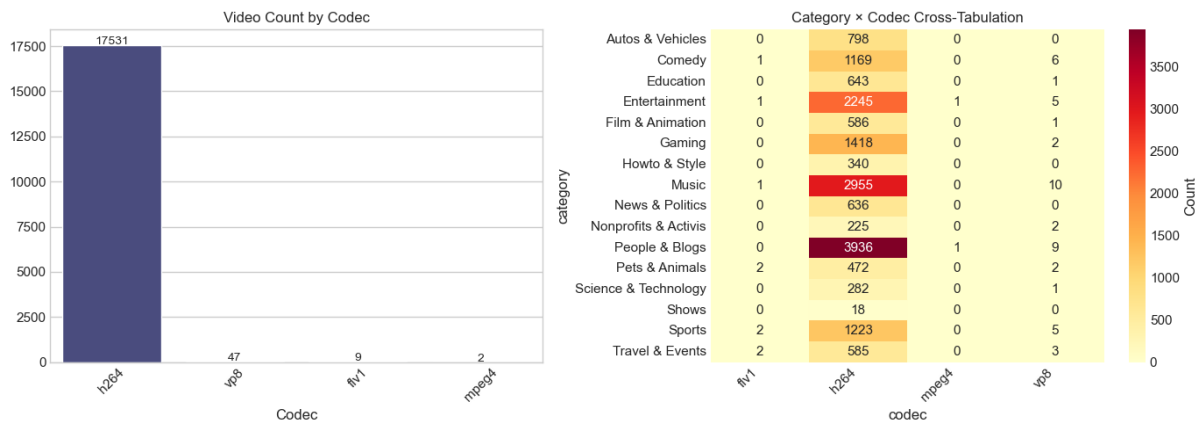
```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```

codec_counts = df['codec'].value_counts()
sns.barplot(x=codec_counts.index, y=codec_counts.values, ax=axes[0], p
axes[0].set_title('Video Count by Codec')
axes[0].set_xlabel('Codec')
axes[0].tick_params(axis='x', rotation=45)
for i, v in enumerate(codec_counts.values):
    axes[0].text(i, v + 50, str(v), ha='center', fontsize=10)

cross = pd.crosstab(df['category'], df['codec'])
sns.heatmap(cross, annot=True, fmt='d', cmap='YlOrRd', ax=axes[1], cba
axes[1].set_title('Category × Codec Cross-Tabulation')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```



## Figure Interpretation

This plot shows how the featured variables vary in this section of the analysis.

The main takeaway is the relative differences in level and spread across groups, which indicates meaningful signal for understanding and predicting virality.

## EDA.ipynb — Code Cell 9

```

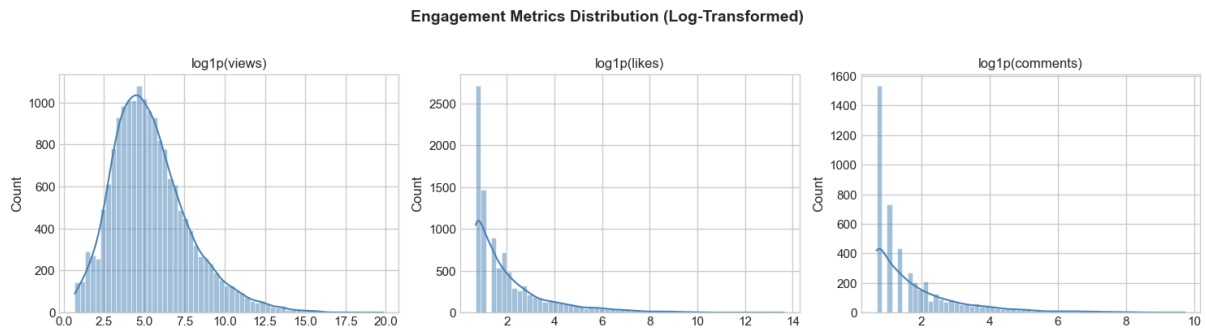
In [ ]: eng_cols = ['views', 'likes', 'comments']
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, col in zip(axes, eng_cols):
    data = np.log1p(df[col].replace(0, np.nan).dropna())
    sns.histplot(data, kde=True, ax=ax, bins=60, color='steelblue', ed
    ax.set_title(f'log1p({col})')
    ax.set_xlabel('')

plt.suptitle('Engagement Metrics Distribution (Log-Transformed)', font
plt.tight_layout()

```

```
plt.show()
```



## Figure Interpretation

This figure highlights the distribution and variability of the plotted features. Differences across groups and the presence of outliers suggest non-uniform behavior that can inform feature importance and model performance.

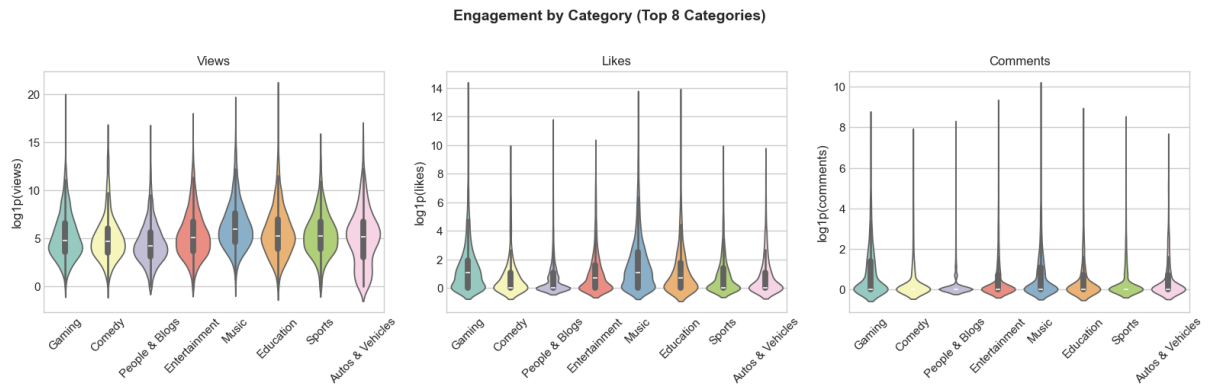
## EDA.ipynb — Code Cell 10

```
In [ ]: top_categories = df['category'].value_counts().head(8).index.tolist()
df_top = df[df['category'].isin(top_categories)].copy()
df_top['log_views'] = np.log1p(df_top['views'])

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for ax, col, log_col in zip(axes, ['views', 'likes', 'comments'], ['log_views', 'log_likes', 'log_comments']):
    if log_col and col == 'views':
        sns.violinplot(data=df_top, x='category', y='log_views', ax=ax)
        ax.set_ylabel('log1p(views)')
    else:
        y_vals = np.log1p(df_top[col])
        plot_df = df_top.assign(log_val=y_vals)
        sns.violinplot(data=plot_df, x='category', y='log_val', ax=ax,
                        ax.set_ylabel(f'log1p({col})'))
        ax.set_title(col.title())
        ax.tick_params(axis='x', rotation=45)
        ax.set_xlabel('')

plt.suptitle('Engagement by Category (Top 8 Categories)', fontsize=14,
plt.tight_layout()
plt.show()
```



## Figure Interpretation

This figure highlights the distribution and variability of the plotted features. Differences across groups and the presence of outliers suggest non-uniform behavior that can inform feature importance and model performance.

## EDA.ipynb — Code Cell 11

```
In [ ]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))

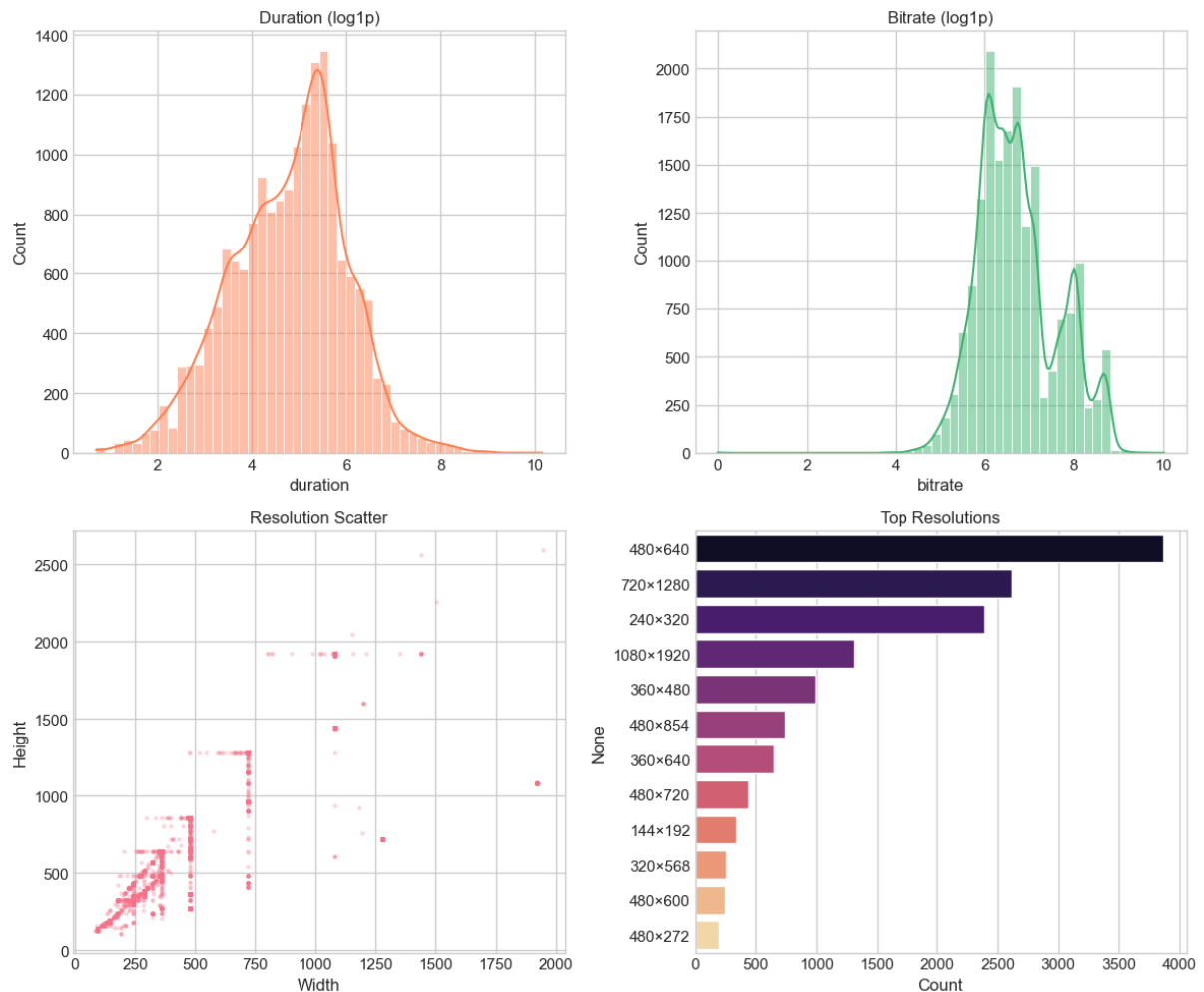
sns.histplot(np.log1p(df['duration']), kde=True, ax=axes[0,0], bins=50)
axes[0,0].set_title('Duration (log1p)')

sns.histplot(np.log1p(df['bitrate']), kde=True, ax=axes[0,1], bins=50,
axes[0,1].set_title('Bitrate (log1p)')

axes[1,0].scatter(df['width'], df['height'], alpha=0.2, s=5)
axes[1,0].set_xlabel('Width')
axes[1,0].set_ylabel('Height')
axes[1,0].set_title('Resolution Scatter')

res_label = df['width'].astype(str) + 'x' + df['height'].astype(str)
res_counts = res_label.value_counts().head(12)
sns.barplot(y=res_counts.index, x=res_counts.values, ax=axes[1,1], pal
axes[1,1].set_title('Top Resolutions')
axes[1,1].set_xlabel('Count')

plt.tight_layout()
plt.show()
```



## Figure Interpretation

This figure highlights the distribution and variability of the plotted features. Differences across groups and the presence of outliers suggest non-uniform behavior that can inform feature importance and model performance.

## EDA.ipynb — Code Cell 12

```
In [ ]: df_eng = df.copy()
df_eng['like_rate'] = df_eng['likes'] / (df_eng['views'] + 1)
df_eng['comment_rate'] = df_eng['comments'] / (df_eng['views'] + 1)
df_eng['engagement_score'] = np.log1p(df_eng['likes'] + df_eng['comment_rate'])
df_eng['title_len'] = df_eng['title'].fillna('').str.len()
df_eng['hashtag_count'] = df_eng['hashtags'].fillna('').apply(lambda x: len(x.split('#')))

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

sns.histplot(df_eng['like_rate'].clip(upper=df_eng['like_rate'].quantile(0.99)),
             axes=[0,0].set_title('Like Rate (capped at 99th percentile)'))

sns.histplot(df_eng['comment_rate'].clip(upper=df_eng['comment_rate'].quantile(0.99)),
             axes=[0,1].set_title('Comment Rate (capped at 99th percentile)'))
```



```

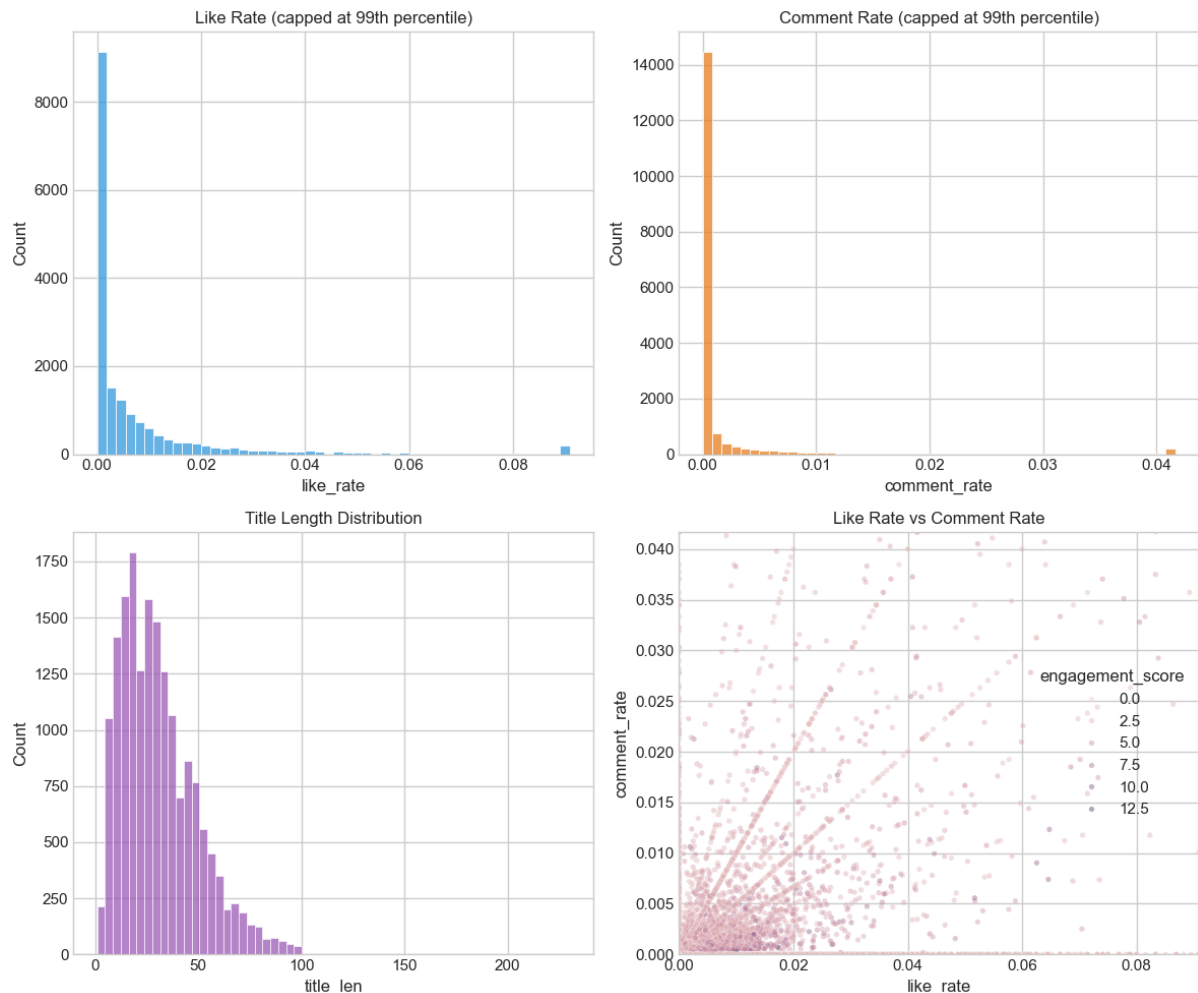
axes[0,1].set_title('Comment Rate (capped at 99th percentile)')

sns.histplot(df_eng['title_len'], ax=axes[1,0], bins=60, color='#9b59b6')
axes[1,0].set_title('Title Length Distribution')

sns.scatterplot(data=df_eng, x='like_rate', y='comment_rate', hue='eng')
axes[1,1].set_xlim(0, df_eng['like_rate'].quantile(0.99))
axes[1,1].set_ylim(0, df_eng['comment_rate'].quantile(0.99))
axes[1,1].set_title('Like Rate vs Comment Rate')

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This figure highlights the distribution and variability of the plotted features. Differences across groups and the presence of outliers suggest non-uniform behavior that can inform feature importance and model performance.

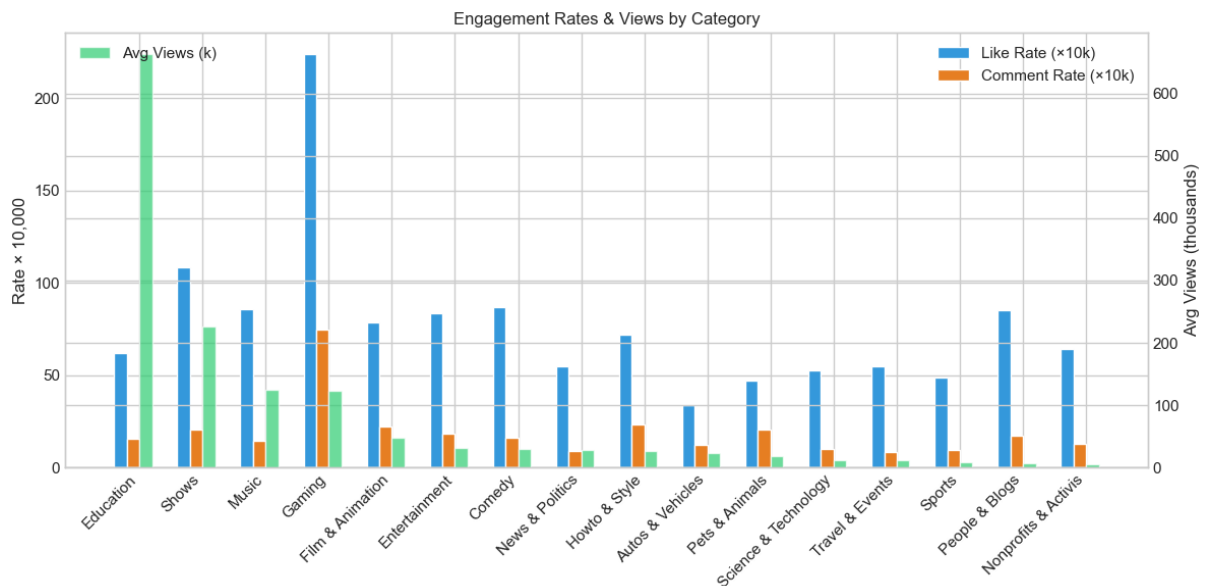
## EDA.ipynb — Code Cell 13

```

In [ ]: rate_by_cat = df_eng.groupby('category').agg({
        'like_rate': 'mean',
        'comment_rate': 'mean',
        'views': 'mean',
        'engagement_score': 'mean'
    }).round(6)
rate_by_cat.columns = ['Avg Like Rate', 'Avg Comment Rate', 'Avg Views']
rate_by_cat = rate_by_cat.sort_values('Avg Views', ascending=False)

fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(rate_by_cat))
w = 0.2
ax.bar(x - 1.5*w, rate_by_cat['Avg Like Rate']*10000, width=w, label='Avg Like Rate (x10k)')
ax.bar(x - 0.5*w, rate_by_cat['Avg Comment Rate']*10000, width=w, label='Avg Comment Rate (x10k)')
ax2 = ax.twinx()
ax2.bar(x + 0.5*w, rate_by_cat['Avg Views']/1000, width=w, label='Avg Views (thousands)')
ax.set_xticks(x)
ax.set_xticklabels(rate_by_cat.index, rotation=45, ha='right')
ax.set_ylabel('Rate x 10,000')
ax2.set_ylabel('Avg Views (thousands)')
ax.legend(loc='upper right')
ax2.legend(loc='upper left')
ax.set_title('Engagement Rates & Views by Category')
plt.tight_layout()
plt.show()

```



## Figure Interpretation

This figure highlights the distribution and variability of the plotted features. Differences across groups and the presence of outliers suggest non-uniform behavior that can inform feature importance and model performance.

## EDA.ipynb — Code Cell 14

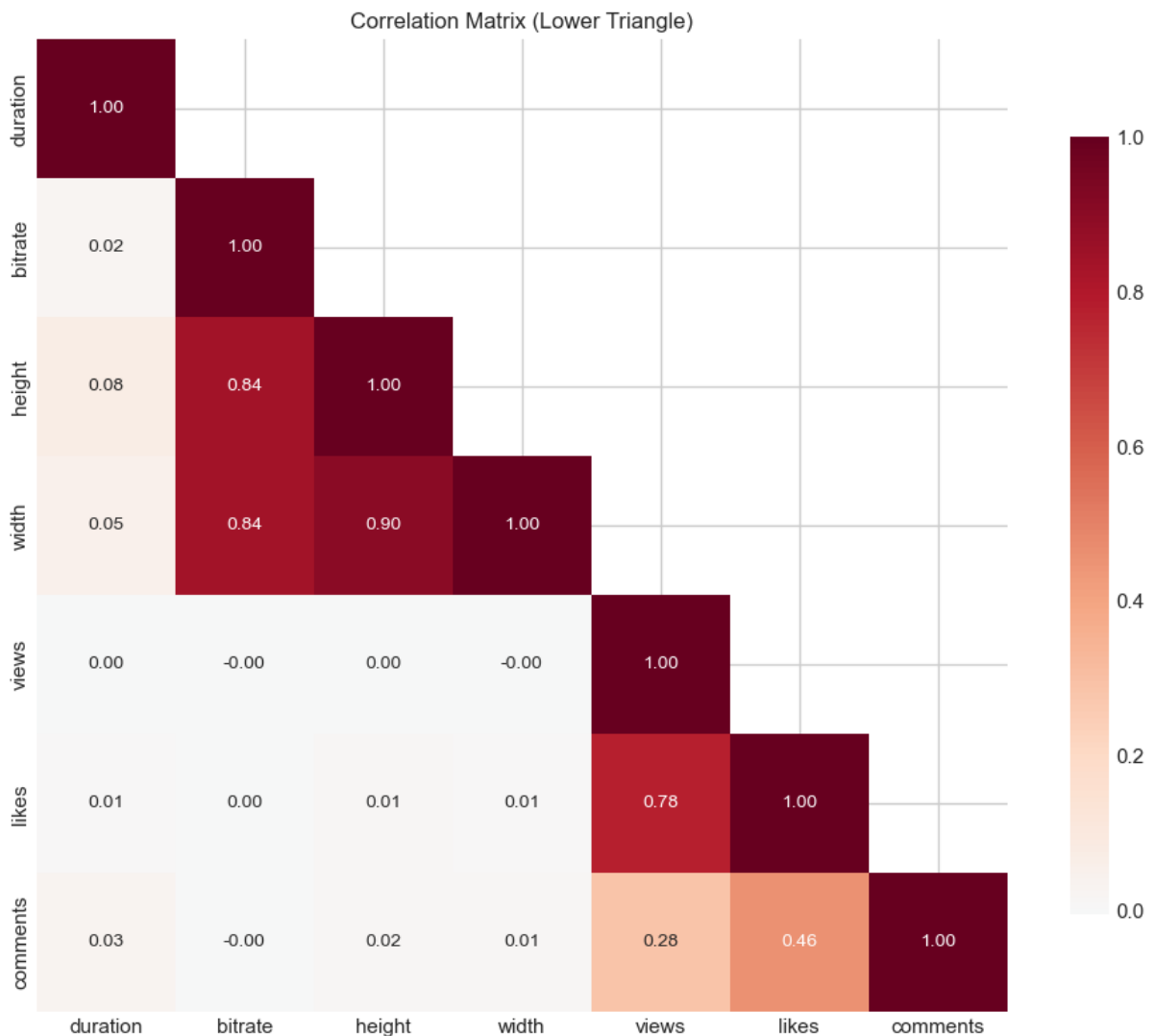
```

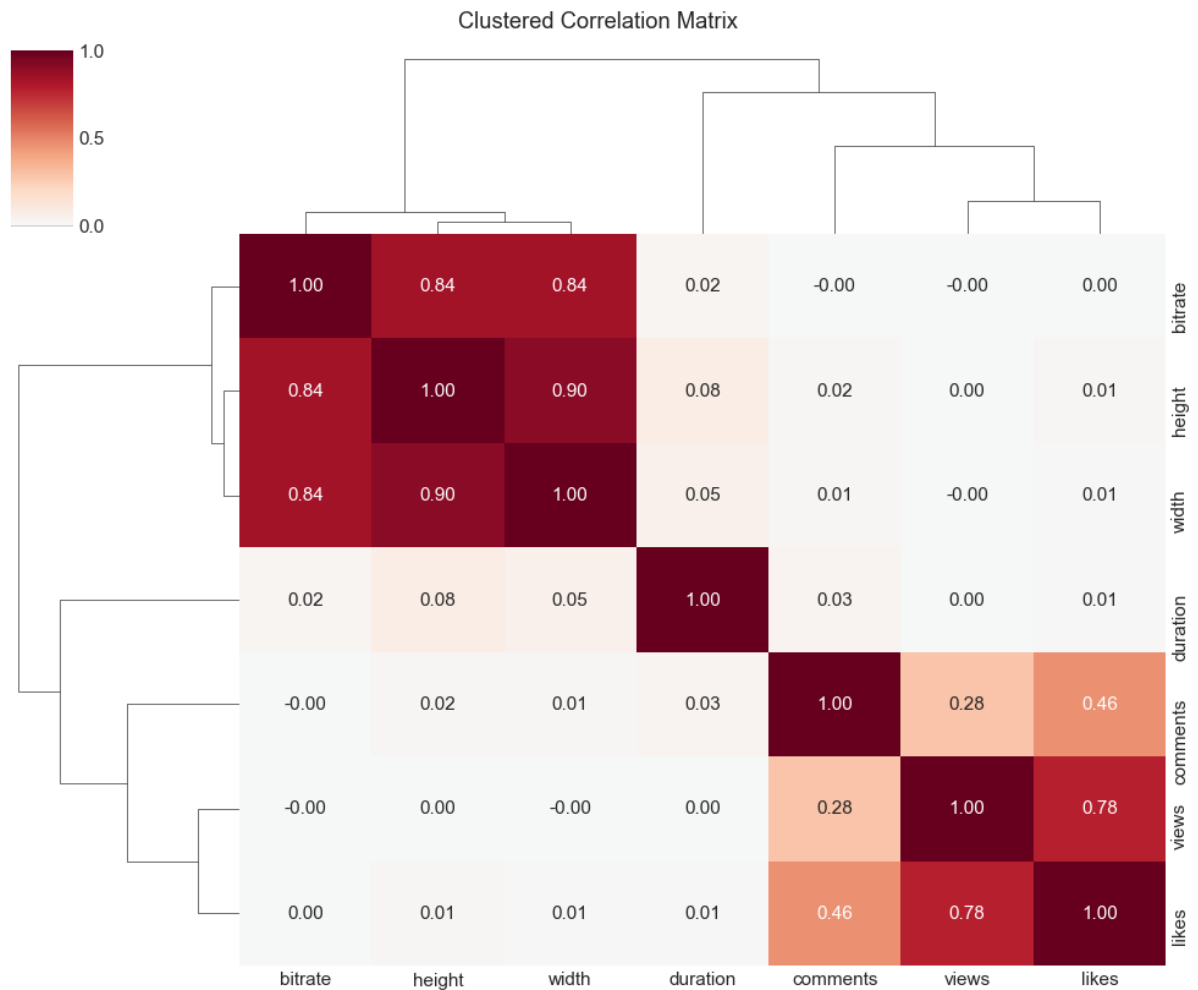
In [ ]: corr_cols = ['duration', 'bitrate', 'height', 'width', 'views', 'likes']
corr_cols = [c for c in corr_cols if c in df.columns]
corr_matrix = df[corr_cols].corr()

fig, ax = plt.subplots(figsize=(10, 8))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)
sns.heatmap(corr_matrix, mask=mask, annot=True, cmap='RdBu_r', center=0,
            ax=ax, cbar_kws={'shrink': 0.8}, annot_kws={'size': 10})
ax.set_title('Correlation Matrix (Lower Triangle)')
plt.tight_layout()
plt.show()

g = sns.clustermap(corr_matrix, annot=True, cmap='RdBu_r', center=0, f
g.fig.suptitle('Clustered Correlation Matrix', y=1.02)
plt.show()

```





## Figure Interpretation

The chart summarizes comparative behavior across the plotted groups. The result suggests that group-level differences are substantial, which supports using these variables for segmentation and predictive modeling.

## EDA.ipynb — Code Cell 15

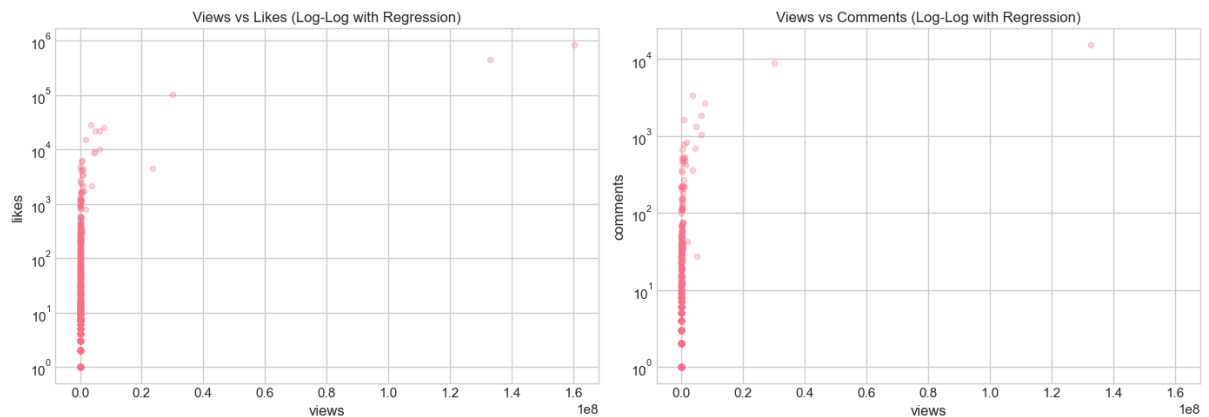
```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sample = df.sample(min(3000, len(df)), random_state=42)

sns.regplot(data=sample, x='views', y='likes', ax=axes[0], scatter_kws=
            line_kws={'color': 'red', 'lw': 2}, logx=True)
axes[0].set_yscale('log')
axes[0].set_title('Views vs Likes (Log-Log with Regression)')

sns.regplot(data=sample, x='views', y='comments', ax=axes[1], scatter_
            line_kws={'color': 'red', 'lw': 2}, logx=True)
axes[1].set_yscale('log')
axes[1].set_title('Views vs Comments (Log-Log with Regression)')
```

```
plt.tight_layout()
plt.show()
```



## Figure Interpretation

The chart summarizes comparative behavior across the plotted groups. The result suggests that group-level differences are substantial, which supports using these variables for segmentation and predictive modeling.

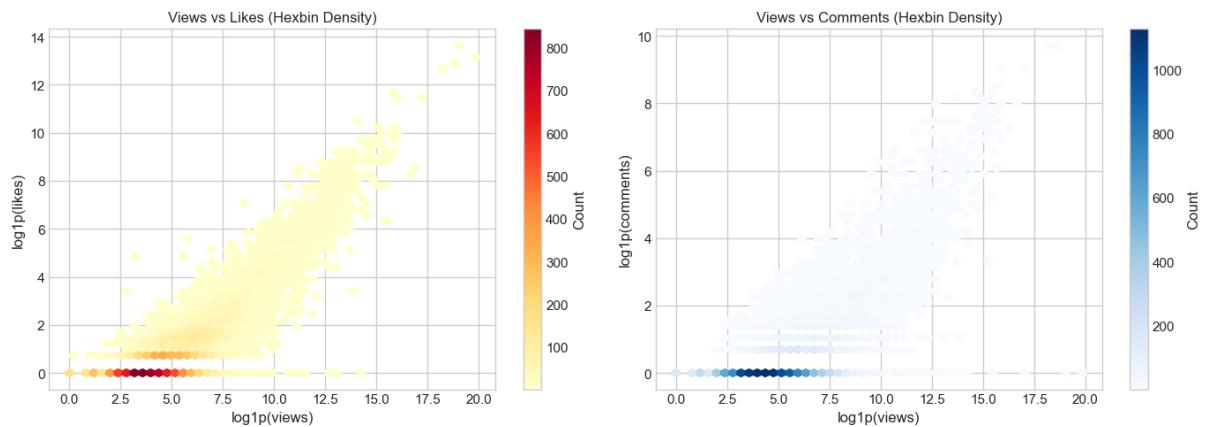
## EDA.ipynb — Code Cell 16

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

h1 = axes[0].hexbin(np.log1p(df['views']), np.log1p(df['likes']), grid
plt.colorbar(h1, ax=axes[0], label='Count')
axes[0].set_xlabel('log1p(views)')
axes[0].set_ylabel('log1p(likes)')
axes[0].set_title('Views vs Likes (Hexbin Density)')

h2 = axes[1].hexbin(np.log1p(df['views']), np.log1p(df['comments']), g
plt.colorbar(h2, ax=axes[1], label='Count')
axes[1].set_xlabel('log1p(views)')
axes[1].set_ylabel('log1p(comments)')
axes[1].set_title('Views vs Comments (Hexbin Density)')

plt.tight_layout()
plt.show()
```



## Figure Interpretation

The chart summarizes comparative behavior across the plotted groups. The result suggests that group-level differences are substantial, which supports using these variables for segmentation and predictive modeling.

## EDA.ipynb — Code Cell 17

```
In [ ]: def count_outliers_iqr(series):
    Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
    IQR = Q3 - Q1
    low, high = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    return ((series < low) | (series > high)).sum()

outlier_counts = pd.DataFrame({
    'Column': ['views', 'likes', 'comments', 'duration'],
    'Outliers (IQR)': [count_outliers_iqr(df[c]) for c in ['views', 'likes', 'comments', 'duration']],
    'Pct (%)': [(count_outliers_iqr(df[c])/len(df)*100).round(2) for c in ['views', 'likes', 'comments', 'duration']]
})
outlier_counts
```

```
Out [ ]:
```

|   | Column   | Outliers (IQR) | Pct (%) |
|---|----------|----------------|---------|
| 0 | views    | 2888           | 16.42   |
| 1 | likes    | 2565           | 14.58   |
| 2 | comments | 2424           | 13.78   |
| 3 | duration | 1376           | 7.82    |

## EDA.ipynb — Code Cell 18

```
In [ ]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))

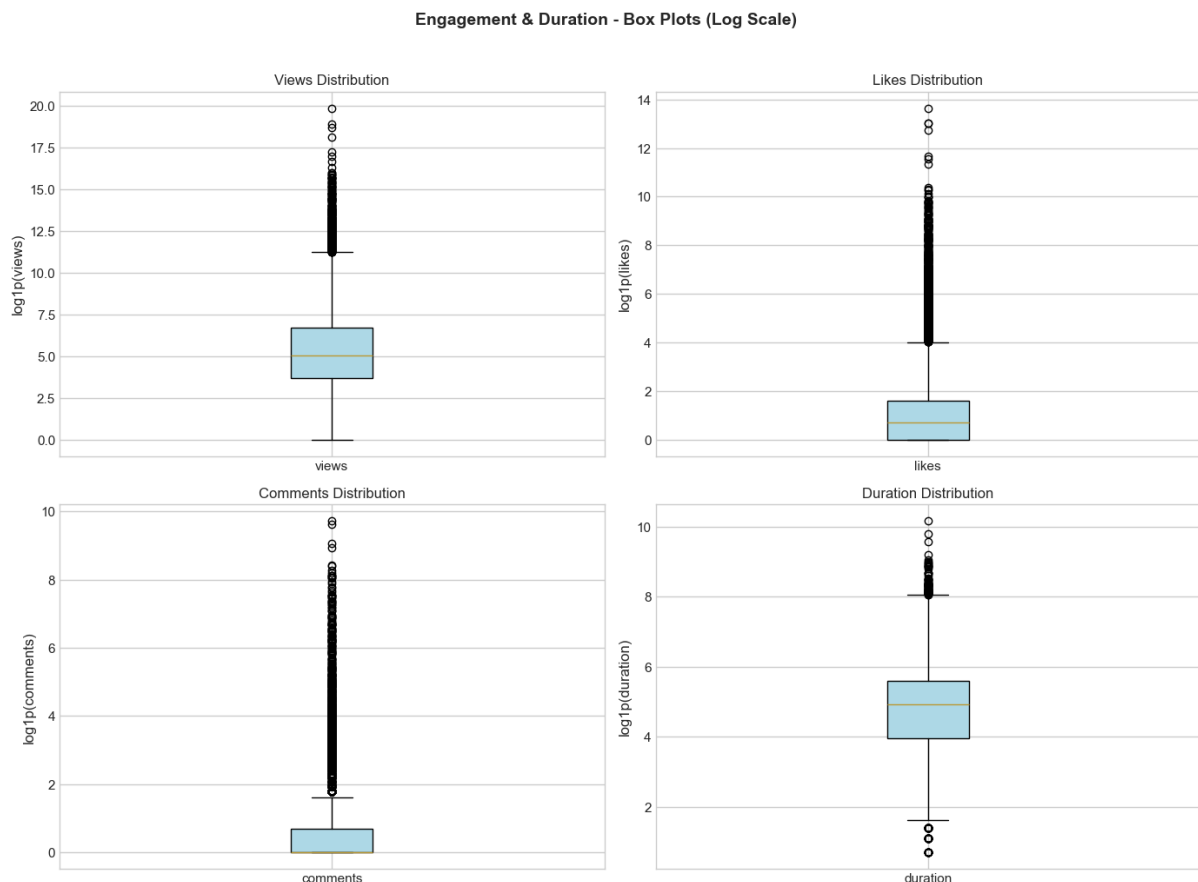
for ax, col in zip(axes.flat, ['views', 'likes', 'comments', 'duration']):
```

```

data = np.log1p(df[col])
bp = ax.boxplot([data], patch_artist=True)
bp['boxes'][0].set_facecolor('lightblue')
ax.set_ylabel(f'log1p({col})')
ax.set_title(f'{col.title()} Distribution')
ax.set_xticklabels([col])

plt.suptitle('Engagement & Duration - Box Plots (Log Scale)', fontsize
plt.tight_layout()
plt.show()

```



## Figure Interpretation

The chart summarizes comparative behavior across the plotted groups. The result suggests that group-level differences are substantial, which supports using these variables for segmentation and predictive modeling.

## EDA.ipynb — Code Cell 19

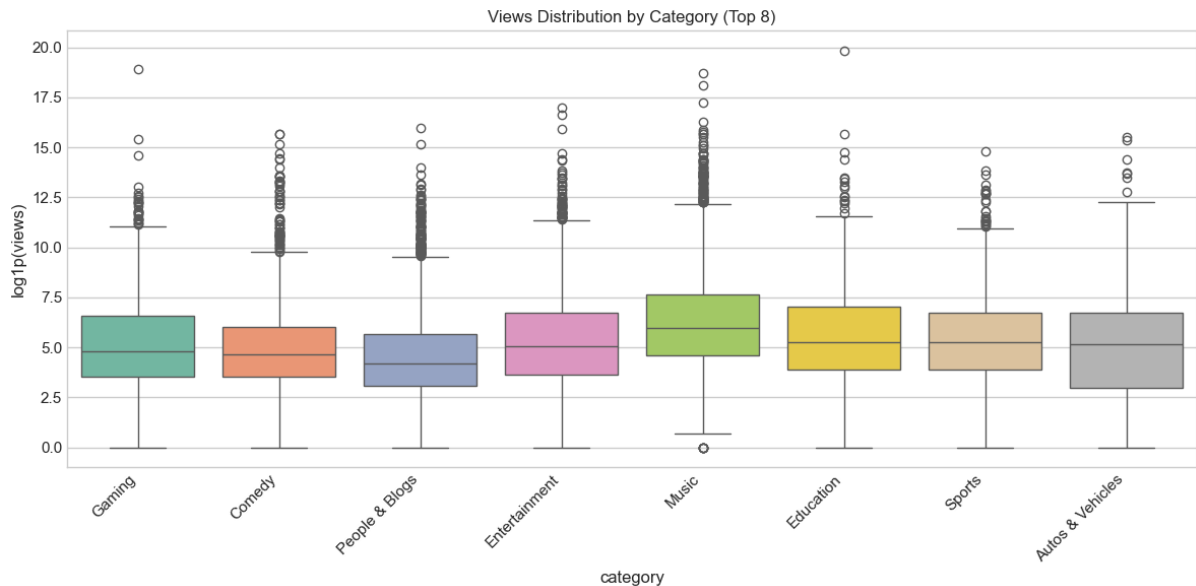
```

In [ ]: top_categories = df['category'].value_counts().head(8).index.tolist()
df_top = df[df['category'].isin(top_categories)].copy()
df_top['log_views'] = np.log1p(df_top['views'])

plt.figure(figsize=(12, 6))

```

```
sns.boxplot(data=df_top, x='category', y='log_views', palette='Set2')
plt.xticks(rotation=45, ha='right')
plt.ylabel('log1p(views)')
plt.title('Views Distribution by Category (Top 8)')
plt.tight_layout()
plt.show()
```



## Figure Interpretation

The chart summarizes comparative behavior across the plotted groups. The result suggests that group-level differences are substantial, which supports using these variables for segmentation and predictive modeling.

## EDA.ipynb — Code Cell 20

```
In [ ]: df_res = df.copy()
df_res['aspect_ratio'] = df_res['width'] / (df_res['height'] + 1e-6)
df_res['pixels'] = df_res['width'] * df_res['height']

def quality_tier(row):
    w, h = row['width'], row['height']
    if w >= 1920 or h >= 1080: return '1080p+'
    if w >= 1280 or h >= 720: return '720p'
    if w >= 854 or h >= 480: return '480p'
    if w >= 640 or h >= 360: return '360p'
    return 'Low'

df_res['quality_tier'] = df_res.apply(quality_tier, axis=1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

quality_order = ['Low', '360p', '480p', '720p', '1080p+']
qt_counts = df_res['quality_tier'].value_counts().reindex(quality_order)
```



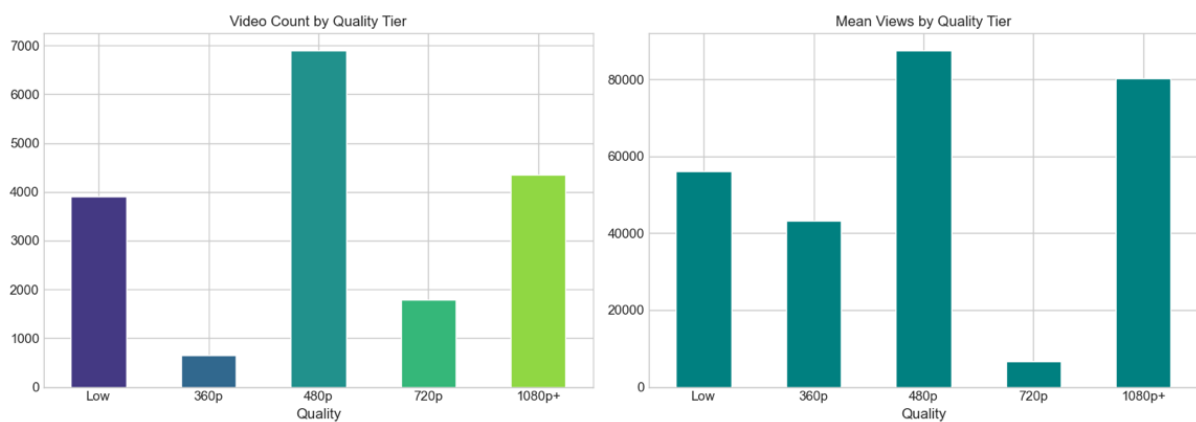
```

if len(qt_counts) > 0:
    qt_counts.plot(kind='bar', ax=axes[0], color=sns.color_palette('vi
axes[0].set_title('Video Count by Quality Tier')
axes[0].set_xlabel('Quality')
axes[0].tick_params(axis='x', rotation=0)

mean_views_qt = df_res.groupby('quality_tier')['views'].mean().reindex
if len(mean_views_qt) > 0:
    mean_views_qt.plot(kind='bar', ax=axes[1], color='teal')
axes[1].set_title('Mean Views by Quality Tier')
axes[1].set_xlabel('Quality')
axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This visualization emphasizes differences in the outcome and predictor distributions across categories. The key result is that category-specific patterns are visible, indicating that category context is relevant for interpretation and modeling.

## EDA.ipynb — Code Cell 21

```

In [ ]: df_txt = df.copy()
df_txt['title_len'] = df_txt['title'].fillna('').str.len()
df_txt['desc_len'] = df_txt['description'].fillna('').str.len()
df_txt['hashtag_count'] = df_txt['hashtags'].fillna('').apply(lambda x

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

sns.histplot(df_txt['title_len'], ax=axes[0,0], bins=60, kde=True, col
axes[0,0].axvline(df_txt['title_len'].median(), color='red', linestyle
axes[0,0].set_title('Title Length')
axes[0,0].legend()

```

```

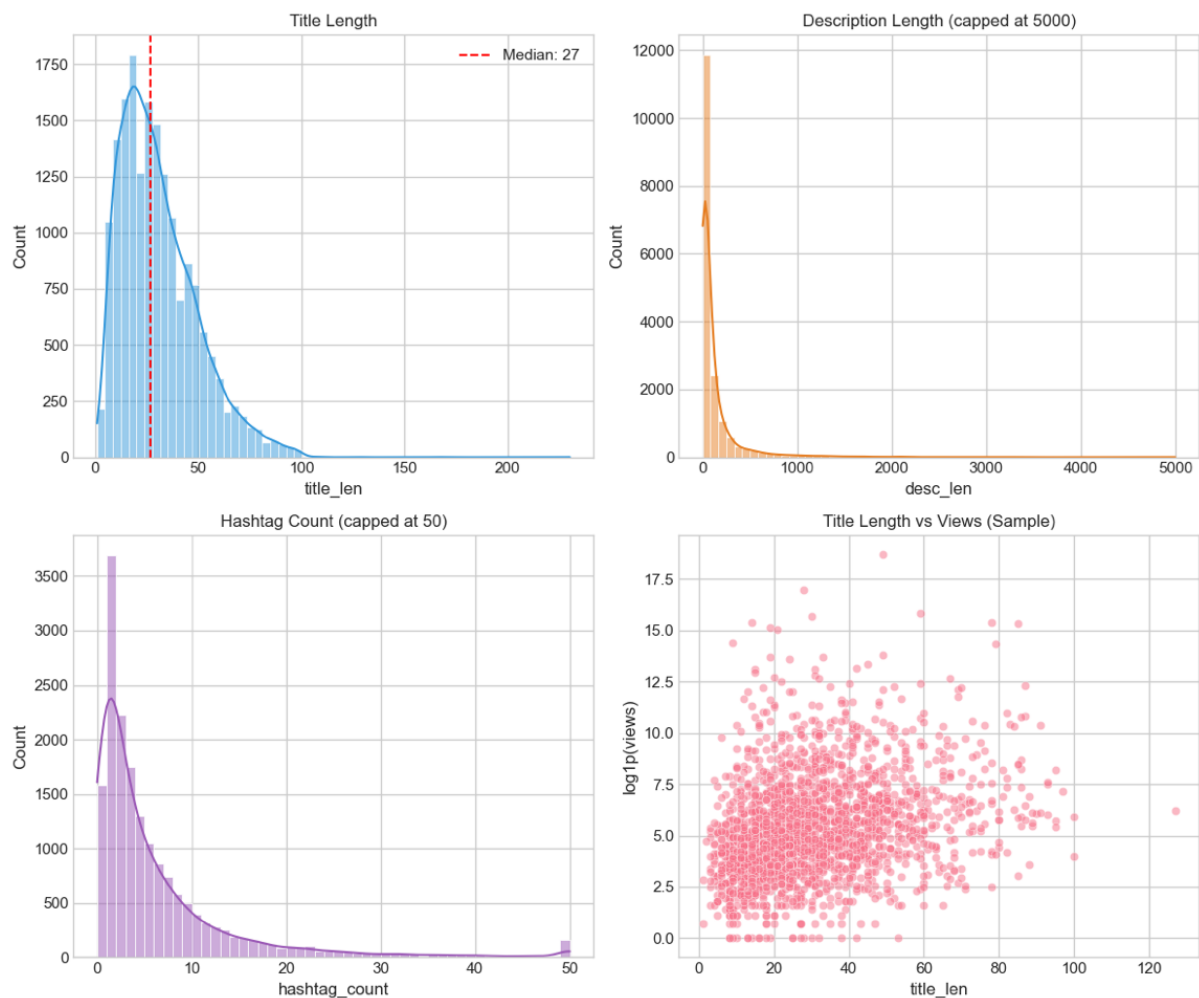
sns.histplot(df_txt['desc_len'].clip(upper=5000), ax=axes[0,1], bins=6
axes[0,1].set_title('Description Length (capped at 5000)')

sns.histplot(df_txt['hashtag_count'].clip(upper=50), ax=axes[1,0], bin
axes[1,0].set_title('Hashtag Count (capped at 50)')

sample_txt = df_txt.sample(min(2000, len(df_txt)), random_state=42)
sns.scatterplot(data=sample_txt, x='title_len', y=np.log1p(sample_txt[
axes[1,1].set_ylabel('log1p(views)')
axes[1,1].set_title('Title Length vs Views (Sample)')

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This visualization emphasizes differences in the outcome and predictor distributions across categories. The key result is that category-specific patterns are visible, indicating that category context is relevant for interpretation and modeling.

## EDA.ipynb — Code Cell 22

```
In [ ]: print('='*60)
print('EDA SUMMARY - YouTube Video Dataset')
print('='*60)
print(f'\n📊 Dataset: {len(df):,} videos, {len(df.columns)} columns')
print(f'\n📌 Categories: {df["category"].nunique()} unique')
print(f'    Top 3: {list(df["category"].value_counts().head(3).index)}')
print(f'\n📌 Views: min={df["views"].min():,}, max={df["views"].max():,}')
print(f'\n📌 Correlations:')
print(f'    views-likes: {df["views"].corr(df["likes"]):.3f}')
print(f'    views-comments: {df["views"].corr(df["comments"]):.3f}')
print(f'    likes-comments: {df["likes"].corr(df["comments"]):.3f}')
print(f'\n📌 Missing: {df.isnull().sum().sum()} total missing values')
print(f'\n📌 Duplicates: {df["video_id"].duplicated().sum()} duplicate video_ids')
print('='*60)
```

```
=====
EDA SUMMARY - YouTube Video Dataset
=====
```

📊 Dataset: 17,589 videos, 17 columns

📌 Categories: 16 unique  
Top 3: ['People & Blogs', 'Music', 'Entertainment']

📌 Views: min=0, max=410,384,910, median=152

📌 Correlations:  
views-likes: 0.778  
views-comments: 0.284  
likes-comments: 0.461

📌 Missing: 2688 total missing values across 2 columns

📌 Duplicates: 0 duplicate video\_ids

```
=====
```

## 5. Data Preparation / Feature Engineering

### 5.1 Feature Engineering and Handling of Features

In order to obtain a multiple-view representation of each video, we performed feature engineering based on a variety of different types of data, such as creator level variables, temporal variables, textual metadata, visual variables, and technical video variables. Creator-level variables were `yt_subscriber_count`, `yt_channel_view_count`, and `yt_channel_video_count`.

All three of these variables had very heavily-tailed distributions so we took

logarithms to make their variances more stable and to reduce the influence of extreme values. Additionally, we created a new derived variable called **channel\_avg\_views** to measure the average number of views per upload for a particular creator. It was created by dividing the total number of views for a particular creator by the number of videos they have uploaded ( $\text{channel\_views} / (\text{channel\_video\_count} + 1)$ ). We then took the  $\log_{10}$  of this value to keep it positive.

We also engineered some temporal variables to capture some of the lifecycle information about the videos and the creators. The time stamps for the creation of the videos and the creation of the channels were converted into date time formats to allow us to create new derived temporal variables. Using these time stamps, we created the variables **video\_age\_days** and **channel\_age\_days** to measure how long ago a video or channel was created. This can be important because older videos and channels may have been viewed more times than newer ones.

We created some text-based proxy variables by extracting available metadata fields. The lengths of the titles and descriptions were measured simply by counting the characters. This resulted in the variables **title\_len** and **desc\_len**, which represent simple measures of the amount of text in a title and description, and therefore represent a simple measure of the quality of the text. In addition, we created a **hashtag\_count** variable to measure how many hashtags were being used to tag the video in order to make it easier for people to find.

We captured visual presentation using interpretable thumbnail-level proxy features. If thumbnail images existed, we extracted two visual descriptors: **thumb\_mean\_brightness** and **thumb\_colorfulness**. These two variables provide simple measures of the visual attractiveness of the thumbnails and may impact whether a user will click on a video recommendation.

Lastly, we included some technical video attributes to represent production aspects of the videos. Some of these include video duration ( $\text{yt\_duration\_sec}$  when available), bitrate, resolution height and width, frame rate, codec type, and categorical metadata such as video category. As well as some platform-level variables such as  $\text{yt\_made\_for\_kids}$ ,  $\text{yt\_live\_broadcast\_content}$ ,  $\text{yt\_channel\_country}$ ,  $\text{yt\_default\_language}$ , and  $\text{yt\_default\_audio\_language}$ . These variables represent a complete picture of all of the structural and contextual properties of each video.

## 5.2 Preprocessing of Variables, Scaling, Assumptions and

## Tests

Our main goal for our study was to predict the logged view count of each video. Therefore, we defined a new variable **target\_log\_views** to be the log base e of one plus the view count of each video. If the enriched variable `yt_view_count` was available, we used it, otherwise we fell back to the original `views` variable. Before applying the log transformation, we clipped negative values to be equal to zero in order to avoid taking logs of negative numbers. The log transformation was necessary because view counts and other engagement metrics follow a highly-skewed distribution and modeling them on a log scale helps improve the model's stability and ease of interpretation.

Prior to training our predictive models, we treated missing numeric values for the numeric predictors using median imputation. Median imputation treats missing values as if they were equal to the median of the non-missing values for that variable. By doing this, we maintained the structure of the distributions of the numeric predictors. Additionally, since the Random Forest model is insensitive to the scaling of its inputs, we did not apply the **StandardScaler** to the numeric predictors before training the Random Forest model. However, for the Elastic Net regression model, K-Nearest Neighbors model, and multilayer perceptron model, we standardized the numeric predictors to have zero mean and unit variance, as these models are sensitive to the scale of their inputs.

For the categorical predictors, we handled missing values using mode imputation. Mode imputation is similar to median imputation except instead of treating missing values as equal to the median of the non-missing values, we treat missing values as equal to the most frequently occurring value of the non-missing values. Since the Random Forest model does not take advantage of the relationships between the different levels of a categorical predictor, we used **OneHotEncoding** with the `handle_unknown='ignore'` parameter to encode all of the categorical predictors. The `handle_unknown='ignore'` parameter prevents model failure due to missing categorical values that appear during prediction but have never appeared in the training set.

When evaluating the performance of the predictive models, we divided the dataset into an 80% training set and 20% testing set with a fixed random seed (`random_state=42`). We made sure to prevent target leakage during model training by excluding variables like likes and comments from the set of predictors for models that were predicting view counts. This was done because likes and comments reflect interactions that occurred after the view count had already accumulated.

## 5.3 Methodology and Tools Used in Data Preparation

The data preparation process was performed using widely-used data science tools in Python. The data manipulation and preprocessing were done using **pandas** and **numpy**, while the machine learning pipelines were implemented using **scikit-learn**. The key preprocessing tools used were **ColumnTransformer** for transforming the different types of features together, **Pipeline** for implementing reproducible preprocessing and modeling workflows, and **SimpleImputer**, **StandardScaler**, and **OneHotEncoder** for specific preprocessing tasks.

The data enrichment and thumbnail feature extraction process was implemented in the Jupyter Notebook `YT-pipeline.ipynb`. The notebook utilized the metadata retrieved via the YouTube Data API to generate creator-level and visual features. The exploratory data analysis was performed in the Jupyter Notebooks `EDA.ipynb` and `EDA_enriched_API.ipynb` and utilized descriptive statistics, correlation matrices, and visualization methods to determine the structure of the dataset. Finally, the final set of features and the configuration of the preprocessing steps for the models were implemented in the Jupyter Notebook `modeling_4_models.ipynb`, in which four different machine learning models were tested utilizing the same preprocessing pipeline.

---

## Exploratory Data Analysis (Enriched/API Code)

This section aggregates code from `EDA_enriched_API.ipynb`. Markdown context from the source notebook is summarized first, followed by all code cells.

### Source Notebook Context (Condensed)

#### YouTube Video Data - Exploratory Data Analysis - (Enhanced Feature Set)

This notebook presents an exploratory data analysis of an enhanced YouTube video dataset that extends the original crawl-level data with platform-native and creator-level features retrieved via the YouTube Data API. In addition to technical encoding attributes and surface metadata, the enriched dataset incorporates temporal information, standardized engagement metrics, channel-level social capital indicators, and platform context variables.

The analysis proceeds through a systematic assessment of data quality and

coverage, followed by univariate and bivariate exploration of video-level, channel-level, and engagement-related features. Particular attention is given to distinguishing content attributes from exposure-driven factors, enabling clearer interpretation of how creator popularity, publication timing, and platform context interact with observed engagement outcomes. Where applicable, derived metrics and normalized representations are examined to support downstream statistical modeling.

Overall, this notebook serves to characterize the expanded feature space, highlight structural differences relative to the original dataset, and inform subsequent modeling decisions by clarifying which signals are intrinsic to content and which primarily reflect post-publication diffusion dynamics.

---

## 1. Environment Setup & Data Loading

---

---

## 2. Data Quality & Profiling

---

---

### 2.1 Basic Schema & Data Types

---

---

### 2.2 Missing Values Analysis

---

---

### 2.3 Duplicates & Data Integrity

---

---

## Enriched Data Loading and Initial Profiling

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', 50)
pd.set_option('display.max_colwidth', 60)

try:
```

```
plt.style.use('seaborn-v0_8-whitegrid')
except (OSError, ValueError):
    try:
        plt.style.use('seaborn-whitegrid')
    except (OSError, ValueError):
        pass
sns.set_palette('husl')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11

df = pd.read_csv('/Users/maxchalekson/Northwestern University/Winter-2
df_raw = df.copy()

print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f}
df.head(10)
```

Dataset shape: 17,589 rows × 45 columns

Memory usage: 40.48 MB



Out[ ]:

|   | video_id     | duration | bitrate | bitrate(video) | height | width | frame<br>rate | fran<br>rate(est |
|---|--------------|----------|---------|----------------|--------|-------|---------------|------------------|
| 0 | --F7dc-_FSI  | 180      | 5777    | 5640           | 1920   | 1080  | 25.00         | 25.0             |
| 1 | --cCAD-8Y_U  | 930      | 1195    | 1001           | 1280   | 720   | 30.00         | 30.0             |
| 2 | --g2gG8pQ0w  | 233      | 3028    | 2833           | 1280   | 720   | 23.98         | 23.9             |
| 3 | --0DR7-voRCU | 562      | 431     | 300            | 320    | 240   | 29.97         | 0.0              |
| 4 | --0Fkp-2EzX0 | 300      | 3087    | 2929           | 1280   | 720   | 23.98         | 23.9             |
| 5 | --0J-a-kKR1M | 135      | 467     | 371            | 320    | 240   | 30.00         | 0.0              |
| 6 | --0bc_-HP6dE | 43       | 634     | 501            | 480    | 360   | 29.97         | 29.9             |
| 7 | --0hjSaYCRnA | 4        | 738     | 605            | 540    | 360   | 29.97         | 0.0              |
| 8 | --0kAY-vAVBc | 228      | 5880    | 5686           | 1920   | 1080  | 25.00         | 25.0             |
| 9 | --0INh-4ZuTE | 268      | 492     | 364            | 640    | 360   | 25.00         | 0.0              |

## EDA\_enriched\_API.ipynb — Code Cell 2

```
In [ ]: def get_sample(col):
    try:
        if df[col].notna().any():
            return str(df[col].dropna().iloc[0])[:40]
    except (IndexError, KeyError):
        pass
    return 'N/A'

schema = pd.DataFrame({
    'Column': df.columns,
    'Dtype': df.dtypes.values,
    'Non-Null': df.count().values,
    'Unique': [df[col].nunique() for col in df.columns],
    'Sample': [get_sample(col) for col in df.columns]
})
schema
```

Out[ ]:

|    | Column           | Dtype   | Non-Null | Unique |                                                             |
|----|------------------|---------|----------|--------|-------------------------------------------------------------|
| 0  | video_id         | object  | 17589    | 17589  |                                                             |
| 1  | duration         | int64   | 17589    | 1320   |                                                             |
| 2  | bitrate          | int64   | 17589    | 3512   |                                                             |
| 3  | bitrate(video)   | int64   | 17589    | 3444   |                                                             |
| 4  | height           | int64   | 17589    | 266    |                                                             |
| 5  | width            | int64   | 17589    | 172    |                                                             |
| 6  | frame rate       | float64 | 17589    | 467    |                                                             |
| 7  | frame rate(est.) | float64 | 17589    | 259    |                                                             |
| 8  | codec            | object  | 17589    | 4      |                                                             |
| 9  | category         | object  | 17589    | 16     |                                                             |
| 10 | url              | object  | 17589    | 17589  | <a href="https://www.youtube.co">https://www.youtube.co</a> |
| 11 | title            | object  | 17589    | 17515  | مفارة بالقاهرة»                                             |
| 12 | description      | object  | 16477    | 12773  | خيار الشعب.. به                                             |
| 13 | hashtags         | object  | 16013    | 15108  | بخصوص, المصري اليو                                          |
| 14 | views            | int64   | 17589    | 4280   |                                                             |
| 15 | likes            | int64   | 17589    | 640    |                                                             |
| 16 | comments         | int64   | 17589    | 281    |                                                             |
| 17 | yt_title         | object  | 17491    | 17418  | مفارة بالقاهرة»                                             |

EDA\_enriched\_API.ipynb — Code Cell 3

file:///Users/maxchalekson/Northwestern%20University/Winter-202...final-project/Final-Notebook-Script/Final\_Report\_Complete.html Page 51 of 122

```

missing_df = pd.DataFrame({'Missing': missing, 'Pct (%)': missing_pct})
missing_df = missing_df[missing_df['Missing'] > 0].sort_values('Missing')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

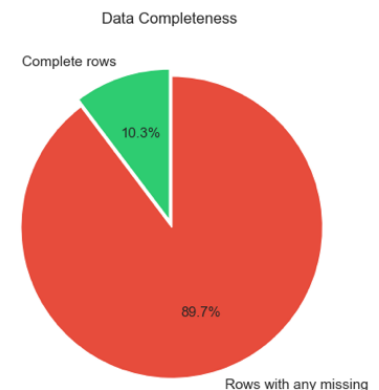
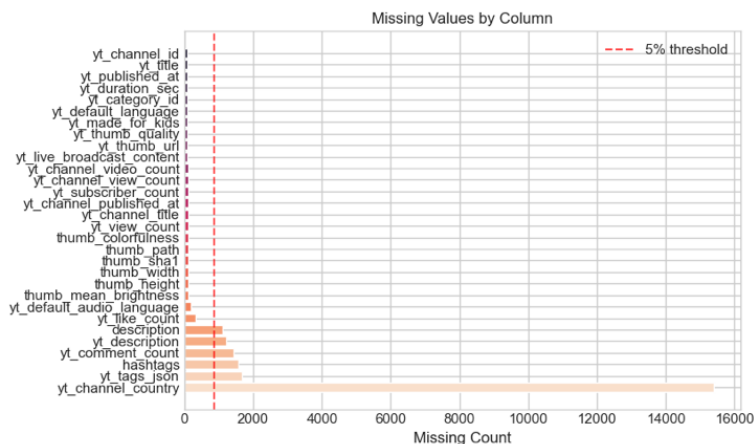
if len(missing_df) > 0:
    bars = axes[0].barh(missing_df.index, missing_df['Missing'], color='red')
    axes[0].set_xlabel('Missing Count')
    axes[0].set_title('Missing Values by Column')
    axes[0].axvline(x=len(df)*0.05, color='red', linestyle='--', alpha=0.5)
    axes[0].legend()

    complete = df.dropna().shape[0]
    incomplete = len(df) - complete
    axes[1].pie([complete, incomplete], labels=['Complete rows', 'Rows with any missing'],
                colors=['#2ecc71', '#e74c3c'], explode=(0, 0.05), startangle=90)
    axes[1].set_title('Data Completeness')
else:
    axes[0].text(0.5, 0.5, 'No missing values', ha='center', va='center')
    axes[1].text(0.5, 0.5, '100% Complete', ha='center', va='center')

plt.tight_layout()
plt.show()

if len(missing_df) > 0:
    display(missing_df)

```



|                    | Missing | Pct (%) |
|--------------------|---------|---------|
| yt_channel_country | 15397   | 87.54   |
| yt_tags_json       | 1670    | 9.49    |
| hashtags           | 1576    | 8.96    |
| yt_comment_count   | 1429    | 8.12    |
| yt_description     | 1205    | 6.85    |
| description        | 1112    | 6.32    |

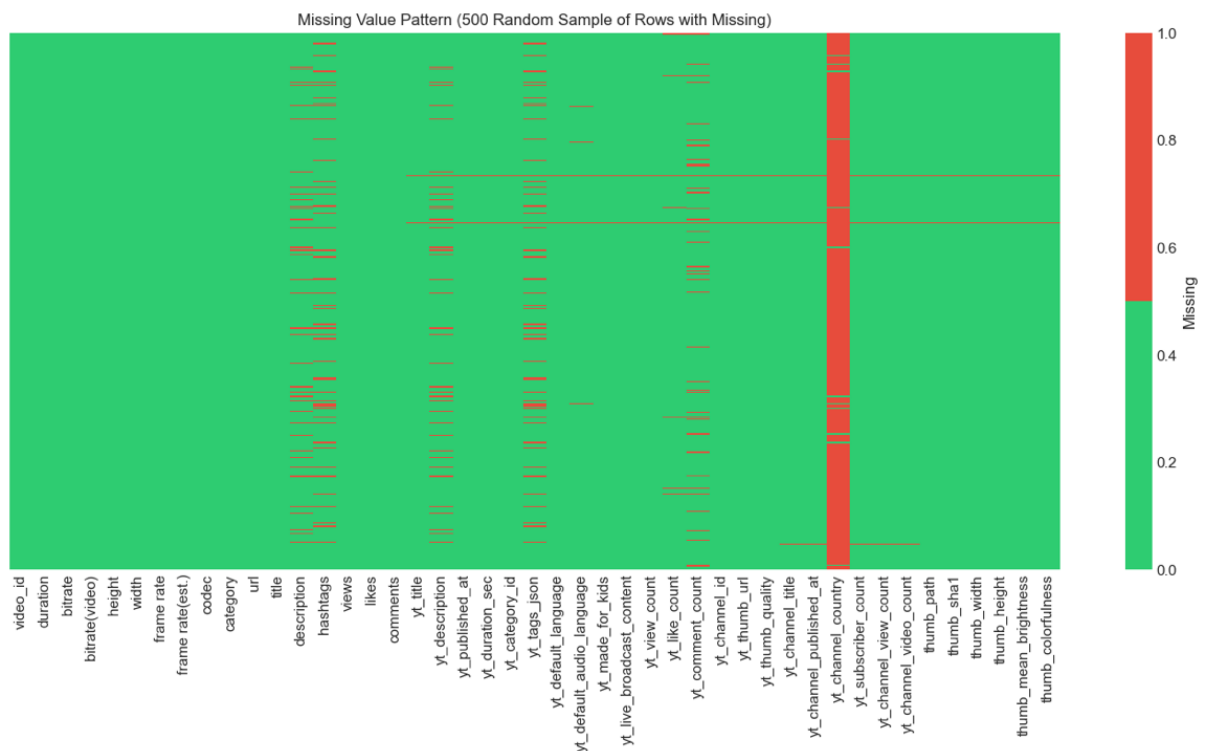
|                                  |     |      |
|----------------------------------|-----|------|
| <b>yt_like_count</b>             | 314 | 1.79 |
| <b>yt_default_audio_language</b> | 192 | 1.09 |
| <b>thumb_mean_brightness</b>     | 106 | 0.60 |
| <b>thumb_height</b>              | 106 | 0.60 |
| <b>thumb_width</b>               | 106 | 0.60 |
| <b>thumb_sha1</b>                | 106 | 0.60 |
| <b>thumb_path</b>                | 106 | 0.60 |
| <b>thumb_colorfulness</b>        | 106 | 0.60 |
| <b>yt_view_count</b>             | 106 | 0.60 |
| <b>yt_channel_title</b>          | 105 | 0.60 |
| <b>yt_channel_published_at</b>   | 105 | 0.60 |
| <b>yt_subscriber_count</b>       | 105 | 0.60 |
| <b>yt_channel_view_count</b>     | 105 | 0.60 |
| <b>yt_channel_video_count</b>    | 105 | 0.60 |
| <b>yt_live_broadcast_content</b> | 98  | 0.56 |
| <b>yt_thumb_url</b>              | 98  | 0.56 |
| <b>yt_thumb_quality</b>          | 98  | 0.56 |
| <b>yt_made_for_kids</b>          | 98  | 0.56 |
| <b>yt_default_language</b>       | 98  | 0.56 |
| <b>yt_category_id</b>            | 98  | 0.56 |
| <b>yt_duration_sec</b>           | 98  | 0.56 |
| <b>yt_published_at</b>           | 98  | 0.56 |
| <b>yt_title</b>                  | 98  | 0.56 |
| <b>yt_channel_id</b>             | 98  | 0.56 |

## Figure Interpretation

This visualization emphasizes differences in the outcome and predictor distributions across categories. The key result is that category-specific patterns are visible, indicating that category context is relevant for interpretation and modeling.

## EDA\_enriched\_API.ipynb — Code Cell 4

```
In [ ]: from matplotlib.colors import ListedColormap
miss_cmap = ListedColormap(['#2ecc71', '#e74c3c'])
rows_with_missing = df[df.isnull().any(axis=1)]
if len(rows_with_missing) > 0 and len(rows_with_missing) <= 500:
    plt.figure(figsize=(14, min(8, len(rows_with_missing) * 0.02)))
    sns.heatmap(rows_with_missing.isnull(), cbar_kws={'label': 'Missing'})
    plt.title('Missing Value Pattern (Rows with Any Missing)')
    plt.tight_layout()
    plt.show()
elif len(rows_with_missing) > 500:
    sample = rows_with_missing.sample(500, random_state=42)
    plt.figure(figsize=(14, 8))
    sns.heatmap(sample.isnull(), cbar_kws={'label': 'Missing'}, cmap=miss_cmap)
    plt.title('Missing Value Pattern (500 Random Sample of Rows with Missing)')
    plt.tight_layout()
    plt.show()
```



## Figure Interpretation

This visualization emphasizes differences in the outcome and predictor distributions across categories. The key result is that category-specific patterns are visible, indicating that category context is relevant for interpretation and modeling.

## EDA\_enriched\_API.ipynb — Code Cell 5

```
In [ ]: dup_video_id = df['video_id'].duplicated().sum()
dup_rows = df.duplicated().sum()

print(f"Duplicate video_id: {dup_video_id}")
print(f"Fully duplicate rows: {dup_rows}")

if dup_video_id > 0:
    dup_ids = df[df['video_id'].duplicated(keep=False)][['video_id']].va
    print(f"\nSample duplicate video_ids:\n{dup_ids}")
```

Duplicate video\_id: 0  
Fully duplicate rows: 0

## EDA\_enriched\_API.ipynb — Code Cell 6

```
In [ ]: numeric_cols = [

    'duration',
    'bitrate',
    'bitrate(video)',
    'height',
    'width',
    'frame_rate',
    'frame_rate(est.)',

    'views',
    'likes',
    'comments',
    'yt_view_count',
    'yt_like_count',
    'yt_comment_count',

    'like_rate',
    'comment_rate',
    'engagement_score',

    'yt_subscriber_count',
    'yt_channel_video_count',

    'video_age_days',
    'channel_age_days',

    'thumb_width',
    'thumb_height',
    'thumb_mean_brightness',
    'thumb_colorfulness'

]
```

```

numeric_cols = [c for c in numeric_cols if c in df.columns]

stats = df[numeric_cols].describe().T

stats['skewness'] = df[numeric_cols].skew(numeric_only=True)
stats['kurtosis'] = df[numeric_cols].kurtosis(numeric_only=True)
stats['IQR'] = stats['75%'] - stats['25%']
stats['range'] = stats['max'] - stats['min']

stats = stats.round(2)
stats

```

Out[ ]:

|                               | count   | mean     | std        | min    | 25%    | 50%    |
|-------------------------------|---------|----------|------------|--------|--------|--------|
| <b>duration</b>               | 17589.0 | 241.55   | 493.03     | 1.00   | 51.00  | 135.00 |
| <b>bitrate</b>                | 17589.0 | 1271.35  | 1375.36    | 0.00   | 437.00 | 743.00 |
| <b>bitrate(video)</b>         | 17589.0 | 1150.42  | 1351.80    | 0.00   | 326.00 | 632.00 |
| <b>height</b>                 | 17589.0 | 766.78   | 467.29     | 108.00 | 426.00 | 640.00 |
| <b>width</b>                  | 17589.0 | 504.59   | 262.73     | 88.00  | 320.00 | 480.00 |
| <b>frame rate</b>             | 17589.0 | 26.47    | 6.04       | 3.75   | 25.00  | 29.97  |
| <b>frame rate(est.)</b>       | 17589.0 | 9.47     | 13.25      | 0.00   | 0.00   | 0.00   |
| <b>views</b>                  | 17589.0 | 68941.58 | 3537491.15 | 0.00   | 38.00  | 152.00 |
| <b>likes</b>                  | 17589.0 | 208.86   | 8477.50    | 0.00   | 0.00   | 1.00   |
| <b>comments</b>               | 17589.0 | 12.90    | 225.84     | 0.00   | 0.00   | 0.00   |
| <b>yt_view_count</b>          | 17483.0 | 69589.71 | 3578902.31 | 0.00   | 38.00  | 154.00 |
| <b>yt_like_count</b>          | 17275.0 | 214.03   | 8619.18    | 0.00   | 0.00   | 1.00   |
| <b>yt_comment_count</b>       | 16160.0 | 14.06    | 236.79     | 0.00   | 0.00   | 0.00   |
| <b>yt_subscriber_count</b>    | 17484.0 | 75444.41 | 1725311.37 | 0.00   | 3.00   | 24.00  |
| <b>yt_channel_video_count</b> | 17484.0 | 2875.80  | 26943.55   | 0.00   | 8.00   | 33.00  |
| <b>thumb_width</b>            | 17483.0 | 735.53   | 331.70     | 66.00  | 480.00 | 640.00 |
| <b>thumb_height</b>           | 17483.0 | 489.45   | 148.41     | 90.00  | 360.00 | 480.00 |
| <b>thumb_mean_brightness</b>  | 17483.0 | 90.23    | 40.56      | 0.00   | 61.40  | 90.59  |
| <b>thumb_colorfulness</b>     | 17483.0 | 41.22    | 23.57      | 0.00   | 24.95  | 37.04  |

## EDA\_enriched\_API.ipynb — Code Cell 7



```

In [ ]: MIN_N = 50

fig, axes = plt.subplots(3, 1, figsize=(14, 16))

cat_counts = df['category'].value_counts()
colors = plt.cm.Spectral(np.linspace(0.2, 0.9, len(cat_counts)))

bars = axes[0].barh(cat_counts.index, cat_counts.values, color=colors)
axes[0].set_xlabel('Video Count')
axes[0].set_title('Video Count by Category', fontsize=14, fontweight='bold')

for i, val in enumerate(cat_counts.values):
    axes[0].text(val + max(cat_counts.values)*0.01, i, f'{val:,}', va='bottom')

axes[0].invert_yaxis()

top_n = 8
top_cats = cat_counts.head(top_n)
other_count = cat_counts.iloc[top_n:].sum() if len(cat_counts) > top_n else 0
pie_data = list(top_cats.values) + ([other_count] if other_count > 0 else [])
pie_labels = list(top_cats.index) + (['Other'] if other_count > 0 else [])

axes[1].pie(
    pie_data,
    labels=pie_labels,
    autopct='%1.1f%%',
    startangle=90,
    explode=[0.02]*len(pie_data)
)
axes[1].set_title('Category Distribution (Pie)', fontsize=14, fontweight='bold')

if 'yt_subscriber_count' in df.columns:
    subs_df = df[['category', 'yt_subscriber_count']].dropna()

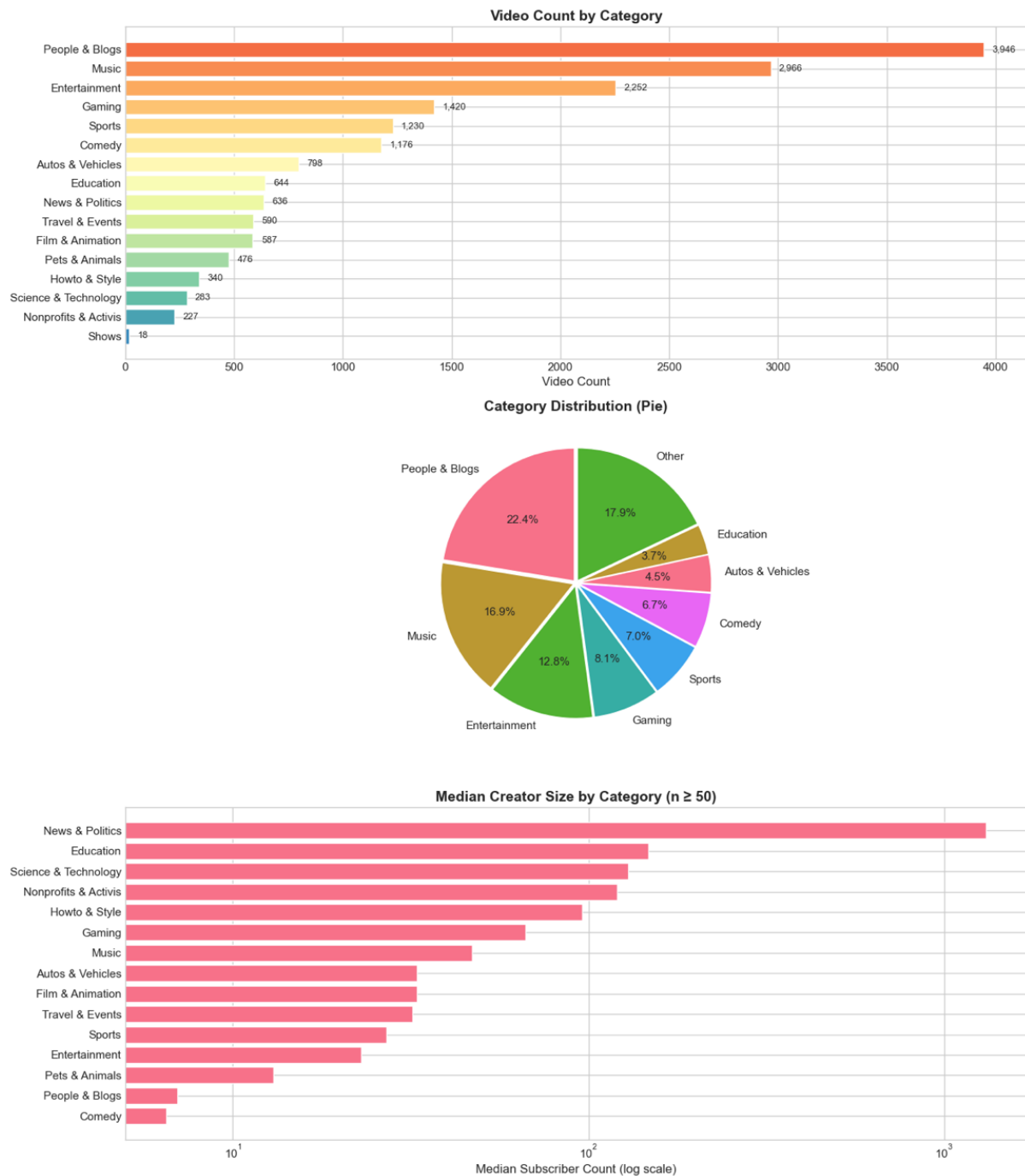
    valid_cats = subs_df['category'].value_counts()
    valid_cats = valid_cats[valid_cats >= MIN_N].index
    subs_df = subs_df[subs_df['category'].isin(valid_cats)]

    med_subs = subs_df.groupby('category')['yt_subscriber_count'].median()

    axes[2].barh(med_subs.index, med_subs.values)
    axes[2].invert_yaxis()
    axes[2].set_xscale('log')
    axes[2].set_xlabel('Median Subscriber Count (log scale)')
    axes[2].set_title(f'Median Creator Size by Category (n ≥ {MIN_N})')
else:
    axes[2].axis("off")
    axes[2].text(0.5, 0.5, "yt_subscriber_count not found in df", ha='center')

```

```
plt.tight_layout()
plt.show()
```



## Figure Interpretation

This visualization emphasizes differences in the outcome and predictor distributions across categories. The key result is that category-specific patterns are visible, indicating that category context is relevant for interpretation and modeling.

## EDA\_enriched\_API.ipynb — Code Cell 8

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))

cat_counts = df['category'].value_counts()
sns.barplot(
    y=cat_counts.index,
    x=cat_counts.values,
    ax=axes[0],
    palette='Spectral'
)

axes[0].set_title('Video Count by Category', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Video Count')
axes[0].set_ylabel('Category')

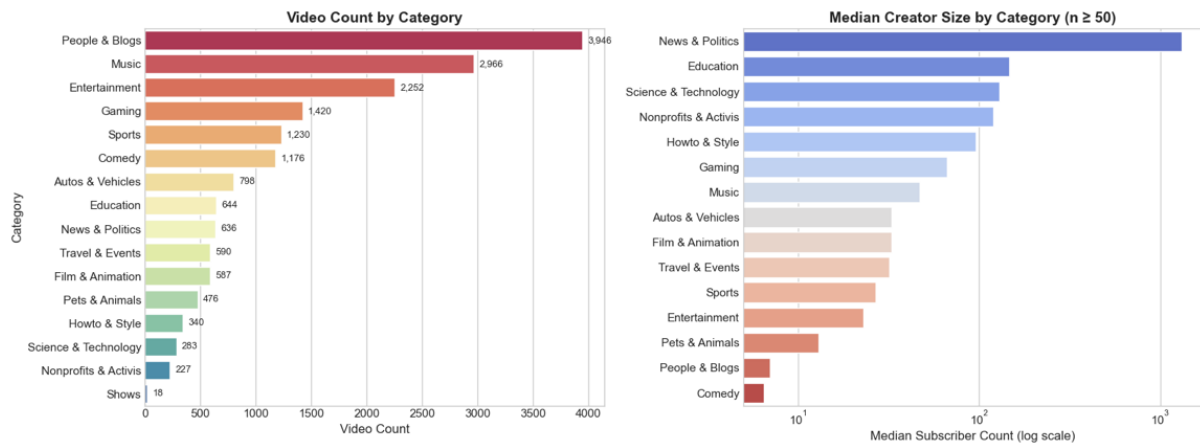
for i, v in enumerate(cat_counts.values):
    axes[0].text(v + 50, i, f'{v:,}', va='center', fontsize=9)

creator_stats = (
    df.dropna(subset=['yt_subscriber_count'])
    .groupby('category')
    .agg(
        median_subs=('yt_subscriber_count', 'median'),
        n_videos=('video_id', 'count')
    )
    .query('n_videos >= 50')
    .sort_values('median_subs', ascending=False)
)

sns.barplot(
    y=creator_stats.index,
    x=creator_stats['median_subs'],
    ax=axes[1],
    palette='coolwarm'
)

axes[1].set_xscale('log')
axes[1].set_title('Median Creator Size by Category (n ≥ 50)', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Median Subscriber Count (log scale)')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()
```



## Figure Interpretation

This figure compares predicted and observed behavior for the selected setup. The closer the pattern is to the reference trend, the stronger the model fit; deviations indicate where error remains.

## EDA\_enriched\_API.ipynb — Code Cell 9

```
In [ ]: tech_cols = [
    'duration', 'bitrate', 'bitrate(video)',
    'height', 'width',
    'frame rate', 'frame rate(est.)'
]

engagement_counts = [
    'views', 'likes', 'comments',
    'yt_view_count', 'yt_like_count', 'yt_comment_count'
]

creator_cols = [
    'yt_subscriber_count',
    'yt_channel_video_count'
]

thumbnail_cols = [
    'thumb_width', 'thumb_height',
    'thumb_mean_brightness',
    'thumb_colorfulness'
]

numeric_groups = {
    "Video Technical Features": tech_cols,
    "Engagement Counts (log1p)": engagement_counts,
    "Creator-Level Scale (log1p)": creator_cols,
    "Thumbnail Visual Features": thumbnail_cols
}
```

```
for title, cols in numeric_groups.items():
    cols = [c for c in cols if c in df.columns]
    if not cols:
        continue

    n_cols = 3
    n_rows = int(np.ceil(len(cols) / n_cols))
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 4 * n_rows))
    axes = axes.flatten()

    for ax, col in zip(axes, cols):
        data = df[col].replace(0, np.nan).dropna()

        if col in engagement_counts or col in creator_cols:
            data = np.log1p(data)
            label = f'log1p({col})'
        else:
            label = col

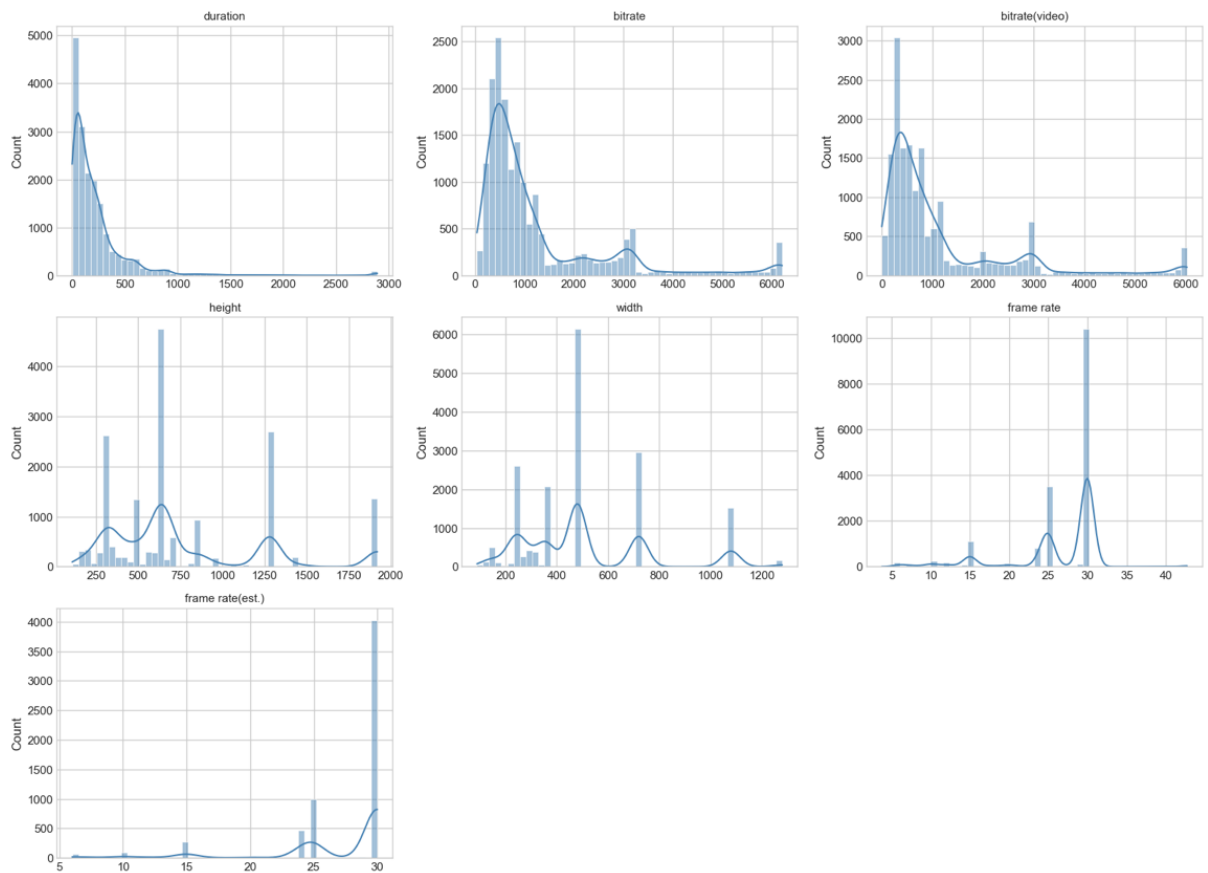
        upper = data.quantile(0.995)
        data = data.clip(upper=upper)

        sns.histplot(
            data,
            kde=True,
            bins=50,
            ax=ax,
            color='steelblue',
            edgecolor='white'
        )
        ax.set_title(label, fontsize=11)
        ax.set_xlabel('')
        ax.set_ylabel('Count')

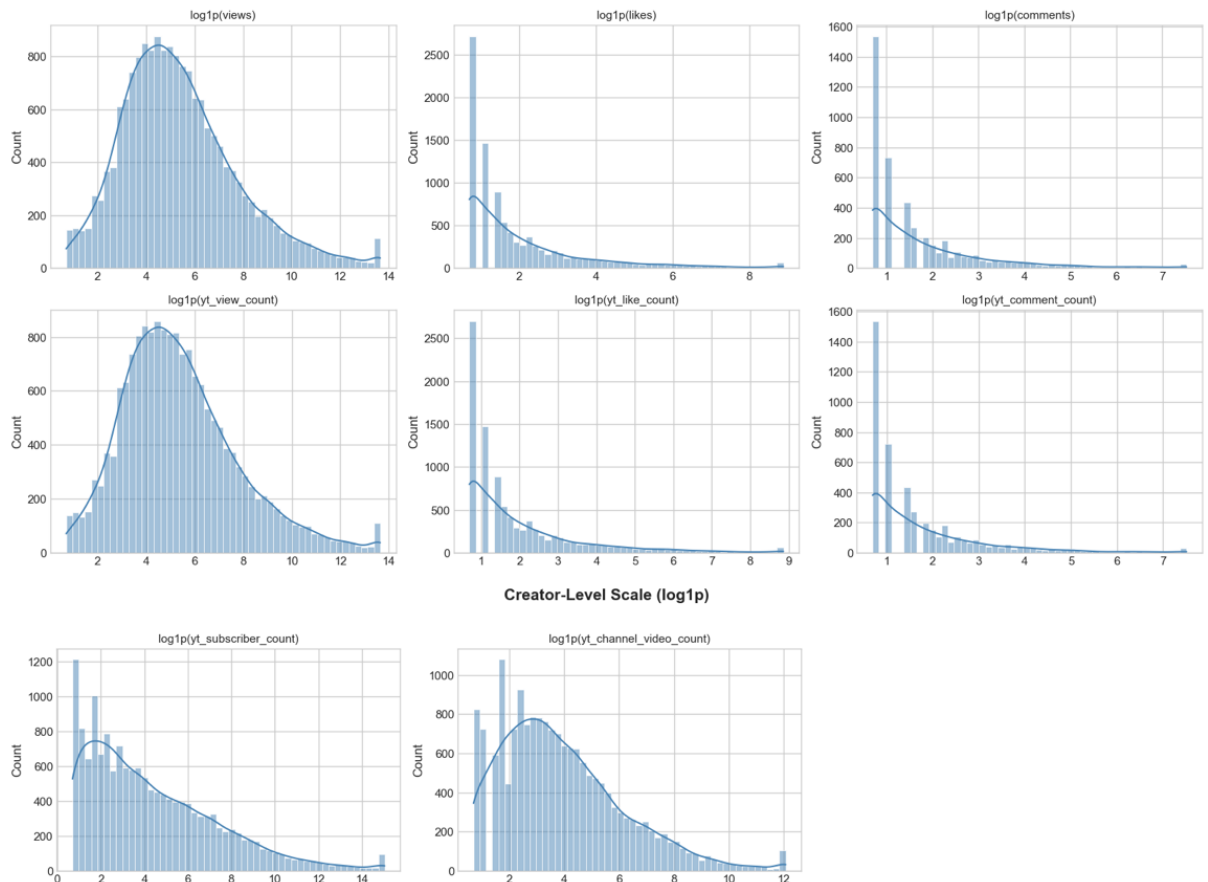
    for ax in axes[len(cols):]:
        ax.axis('off')

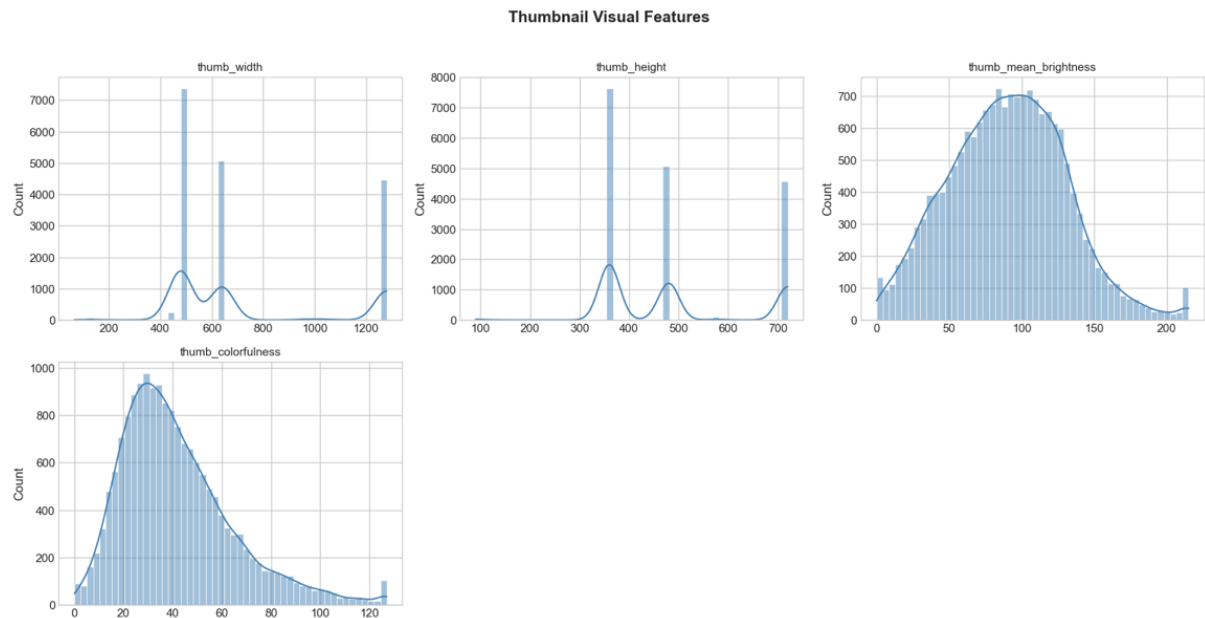
    plt.suptitle(title, fontsize=15, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()
```

## Video Technical Features



## Engagement Counts (log1p)





## Figure Interpretation

This figure compares predicted and observed behavior for the selected setup. The closer the pattern is to the reference trend, the stronger the model fit; deviations indicate where error remains.

## EDA\_enriched\_API.ipynb — Code Cell 10

```
In [ ]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

TOP_N = 8
top_categories = df['category'].value_counts().head(TOP_N).index.tolist()
df_top = df[df['category'].isin(top_categories)].copy()

df_top = df_top.dropna(subset=["category"])

palette = sns.color_palette("Set2", n_colors=len(top_categories))
category_palette = dict(zip(top_categories, palette))

for col in ["views", "likes", "comments",
            "yt_view_count", "yt_like_count", "yt_comment_count",
            "yt_subscriber_count", "yt_channel_video_count",
            "thumb_mean_brightness", "thumb_colorfulness"]:
    if col in df_top.columns:
```

```

df_top[f"log1p_{col}"] = np.log1p(pd.to_numeric(df_top[col], e

views_base = "yt_view_count" if "yt_view_count" in df_top.columns else
likes_base = "yt_like_count" if "yt_like_count" in df_top.columns else
comments_base = "yt_comment_count" if "yt_comment_count" in df_top.col

df_top[views_base] = pd.to_numeric(df_top[views_base], errors="coerce"
df_top[likes_base] = pd.to_numeric(df_top[likes_base], errors="coerce"
df_top[comments_base] = pd.to_numeric(df_top[comments_base], errors="c

df_top["like_rate"] = df_top[likes_base] / df_top[views_base].replace(
df_top["comment_rate"] = df_top[comments_base] / df_top[views_base].re

df_top["like_rate_cap"] = df_top["like_rate"].clip(upper=df_top["like_
df_top["comment_rate_cap"] = df_top["comment_rate"].clip(upper=df_top[

plot_groups = [
    {
        "title": "Engagement (log1p counts)",
        "cols": [
            ("log1p_"+views_base, f"log1p({views_base})"),
            ("log1p_"+likes_base, f"log1p({likes_base})"),
            ("log1p_"+comments_base, f"log1p({comments_base})"),
        ],
    },
    {
        "title": "Creator scale (log1p)",
        "cols": [
            ("log1p_yt_subscriber_count", "log1p(subscribers)"),
            ("log1p_yt_channel_video_count", "log1p(channel uploads)"),
        ],
    },
    {
        "title": "Thumbnail visual features",
        "cols": [
            ("thumb_mean_brightness", "mean brightness"),
            ("thumb_colorfulness", "colorfulness"),
        ],
    },
    {
        "title": "Engagement rates (capped @ 99th pct)",
        "cols": [
            ("like_rate_cap", "like_rate"),
            ("comment_rate_cap", "comment_rate"),
        ],
    },
]

```



```

filtered_groups = []
for g in plot_groups:
    available = [(c, lab) for (c, lab) in g["cols"] if c in df_top.col
    if available:
        g2 = dict(g)
        g2["cols"] = available
        filtered_groups.append(g2)

sns.set(style="whitegrid")

for g in filtered_groups:
    n = len(g["cols"])
    fig, axes = plt.subplots(1, n, figsize=(6*n, 5))
    if n == 1:
        axes = [axes]

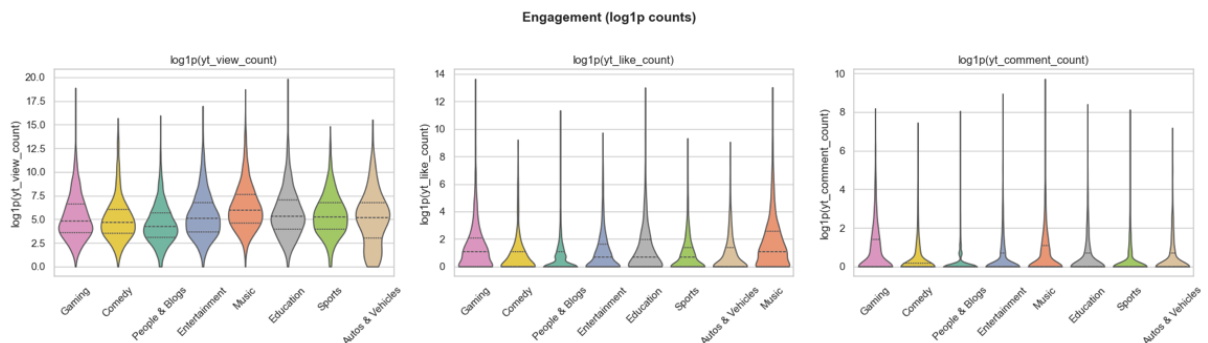
    for ax, (col, label) in zip(axes, g["cols"]):
        plot_df = df_top.dropna(subset=[col]).copy()

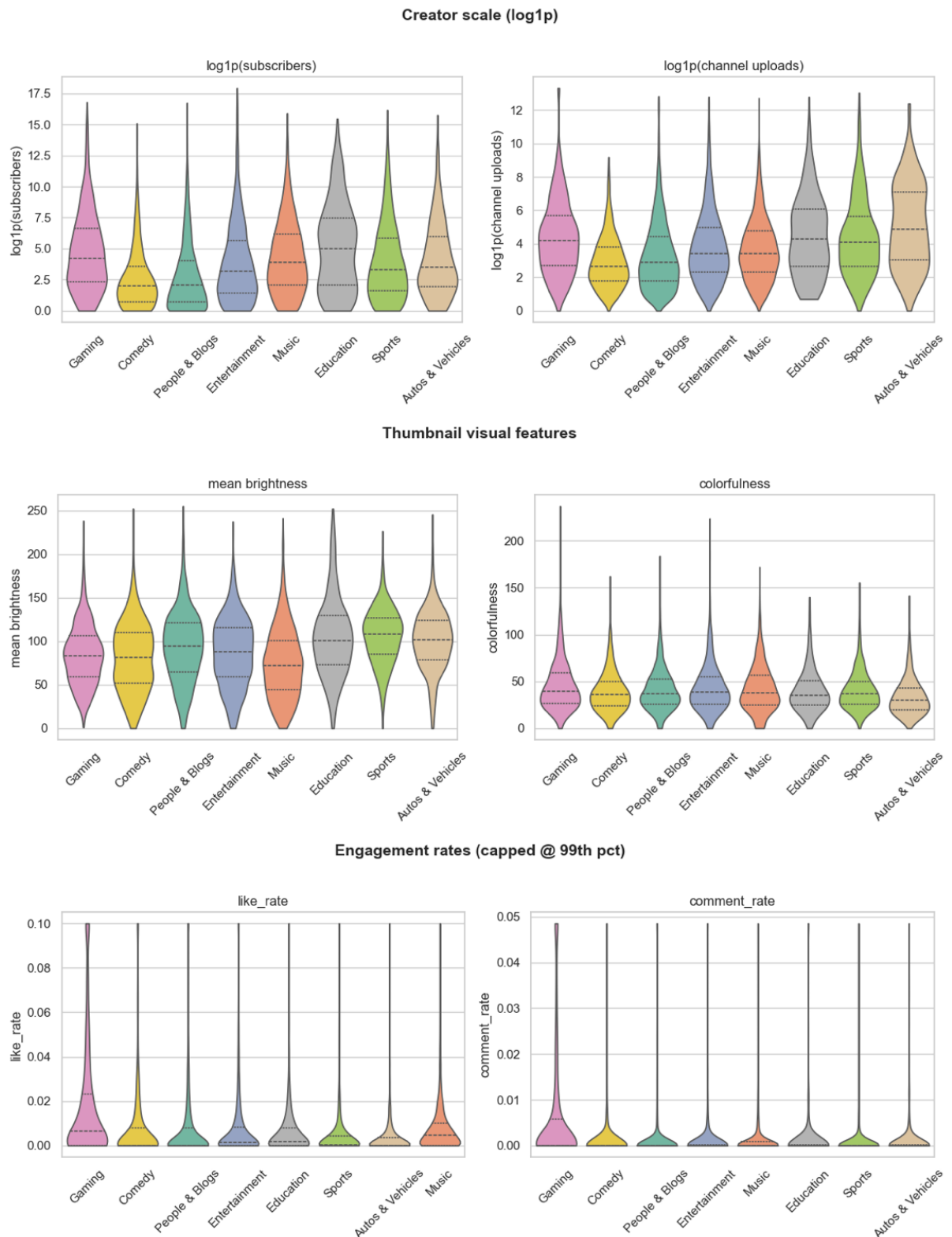
        sns.violinplot(
            data=plot_df,
            x="category",
            y=col,
            ax=ax,
            palette=category_palette,
            inner="quartile",
            cut=0
        )

        ax.set_title(label)
        ax.set_xlabel("")
        ax.set_ylabel(label)
        ax.tick_params(axis="x", rotation=45)

    plt.suptitle(g["title"], fontsize=14, fontweight="bold", y=1.02)
    plt.tight_layout()
    plt.show()

```





## Figure Interpretation

This figure compares predicted and observed behavior for the selected setup. The closer the pattern is to the reference trend, the stronger the model fit; deviations indicate where error remains.

## EDA\_enriched\_API.ipynb — Code Cell 11

```

In [ ]: sns.set(style="whitegrid")

def _num(s):
    return pd.to_numeric(s, errors="coerce")

dur_col = "duration" if "duration" in df.columns else None
bit_col = "bitrate" if "bitrate" in df.columns else None
bitv_col = "bitrate(video)" if "bitrate(video)" in df.columns else None
fr_col = "frame rate" if "frame rate" in df.columns else None
fr_est_col = "frame rate(est.)" if "frame rate(est.)" in df.columns else None

w_col = "width" if "width" in df.columns else None
h_col = "height" if "height" in df.columns else None

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

if dur_col:
    sns.histplot(np.log1p(_num(df[dur_col]).dropna()), kde=True, ax=axes[0, 0])
    axes[0, 0].set_title("Duration (log1p)")
else:
    axes[0, 0].axis("off")

if bit_col:
    sns.histplot(np.log1p(_num(df[bit_col]).dropna()), kde=True, ax=axes[0, 1])
    axes[0, 1].set_title("Bitrate (log1p)")
else:
    axes[0, 1].axis("off")

if bitv_col:
    sns.histplot(np.log1p(_num(df[bitv_col]).dropna()), kde=True, ax=axes[0, 2])
    axes[0, 2].set_title("Bitrate(video) (log1p)")
elif fr_col:
    sns.histplot(_num(df[fr_col]).dropna(), kde=True, ax=axes[0, 2], bins=10)
    axes[0, 2].set_title("Frame rate")
else:
    axes[0, 2].axis("off")

if w_col and h_col:
    ww = _num(df[w_col])
    hh = _num(df[h_col])
    m = ww.notna() & hh.notna()
    axes[1, 0].scatter(ww[m], hh[m], alpha=0.2, s=6)
    axes[1, 0].set_xlabel("Width")

```

```

axes[1, 0].set_ylabel("Height")
axes[1, 0].set_title("Resolution Scatter")
else:
    axes[1, 0].axis("off")

if w_col and h_col:
    res_label = _num(df[w_col]).astype("Int64").astype(str) + "x" + _n
    res_counts = res_label.replace("<NA>x<NA>", np.nan).dropna().value

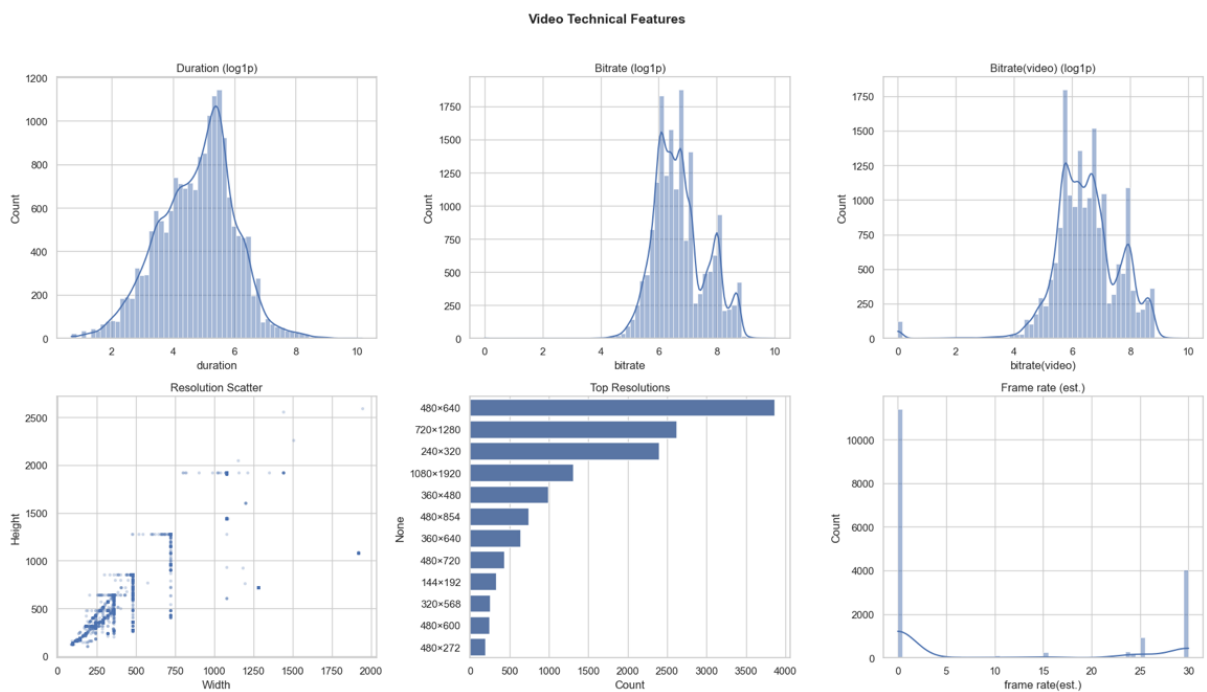
    sns.barplot(y=res_counts.index, x=res_counts.values, ax=axes[1, 1])
    axes[1, 1].set_title("Top Resolutions")
    axes[1, 1].set_xlabel("Count")
else:
    axes[1, 1].axis("off")

if fr_est_col:
    sns.histplot(_num(df[fr_est_col]).dropna(), kde=True, ax=axes[1, 2])
    axes[1, 2].set_title("Frame rate (est.)")
elif fr_col and not bitv_col:

    axes[1, 2].axis("off")
elif fr_col:
    sns.histplot(_num(df[fr_col]).dropna(), kde=True, ax=axes[1, 2], b
    axes[1, 2].set_title("Frame rate")
else:
    axes[1, 2].axis("off")

plt.suptitle("Video Technical Features", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

```



## Figure Interpretation

This figure compares predicted and observed behavior for the selected setup. The closer the pattern is to the reference trend, the stronger the model fit; deviations indicate where error remains.

## EDA\_enriched\_API.ipynb — Code Cell 12

```
In [ ]: sns.set_theme(style="whitegrid", context="notebook")

df_eng = df.copy()

def pick_col(primary, fallback):
    return primary if primary in df_eng.columns else fallback

views_col = pick_col("yt_view_count", "views")
likes_col = pick_col("yt_like_count", "likes")
comments_col = pick_col("yt_comment_count", "comments")

for c in [
    views_col, likes_col, comments_col,
    "yt_subscriber_count", "yt_channel_video_count",
    "yt_channel_view_count", "yt_duration_sec"
]:
    if c in df_eng.columns:
        df_eng[c] = pd.to_numeric(df_eng[c], errors="coerce")

df_eng["yt_published_at_dt"] = pd.to_datetime(
    df_eng.get("yt_published_at"), errors="coerce", utc=True
)
df_eng["yt_channel_published_at_dt"] = pd.to_datetime(
    df_eng.get("yt_channel_published_at"), errors="coerce", utc=True
)

ref_time = df_eng["yt_published_at_dt"].max()
if pd.isna(ref_time):
    ref_time = pd.Timestamp.utcnow()

df_eng["video_age_days"] = (ref_time - df_eng["yt_published_at_dt"]).dt.days
df_eng["channel_age_days"] = (ref_time - df_eng["yt_channel_published_at_dt"]).dt.days

df_eng["like_rate"] = df_eng[likes_col] / df_eng[views_col].replace(0, 1)
df_eng["comment_rate"] = df_eng[comments_col] / df_eng[views_col].replace(0, 1)

df_eng["like_rate_cap"] = df_eng["like_rate"].clip(
    upper=df_eng["like_rate"].quantile(0.99)
```

```

)
df_eng["comment_rate_cap"] = df_eng["comment_rate"].clip(
    upper=df_eng["comment_rate"].quantile(0.99)
)

df_eng["engagement_score"] = np.log1p(
    df_eng[likes_col].fillna(0) + 2 * df_eng[comments_col].fillna(0)
)

df_eng["log1p_subscribers"] = np.log1p(df_eng.get("yt_subscriber_count")
df_eng["log1p_channel_uploads"] = np.log1p(df_eng.get("yt_channel_vide
df_eng["log1p_video_len_sec"] = np.log1p(df_eng.get("yt_duration_sec")

title_src = "yt_title" if "yt_title" in df_eng.columns else "title"
df_eng["title_len"] = df_eng[title_src].fillna("").astype(str).str.len

if "yt_channel_country" in df_eng.columns:
    top_ctry = (
        df_eng["yt_channel_country"]
        .fillna("Unknown")
        .value_counts()
        .head(8)
        .index
    )
    df_eng["channel_country_group"] = df_eng["yt_channel_country"].fil
    df_eng.loc[~df_eng["channel_country_group"].isin(top_ctry), "chann
else:
    df_eng["channel_country_group"] = "Unknown"

fig, axes = plt.subplots(3, 3, figsize=(18, 14))

sns.histplot(
    df_eng["like_rate_cap"].dropna(),
    bins=60, kde=True, ax=axes[0, 0],
    color="#3498db"
)
axes[0, 0].set_title("Like Rate (capped @ 99th pct)")

sns.histplot(
    df_eng["comment_rate_cap"].dropna(),
    bins=60, kde=True, ax=axes[0, 1],
    color="#e67e22"
)
axes[0, 1].set_title("Comment Rate (capped @ 99th pct)")

sns.scatterplot(
    data=df_eng,
    x="like_rate_cap",

```

```
y="comment_rate_cap",
hue="engagement_score",
palette="viridis",
alpha=0.4,
s=14,
ax=axes[0, 2],
legend=False
)
axes[0, 2].set_title("Like vs Comment Rate (colored by engagement)")

sns.histplot(
    df_eng["log1p_subscribers"].dropna(),
    bins=60, kde=True, ax=axes[1, 0],
    color="#8e44ad"
)
axes[1, 0].set_title("Subscribers (log1p)")

sns.histplot(
    df_eng["channel_age_days"].dropna(),
    bins=60, kde=True, ax=axes[1, 1],
    color="#16a085"
)
axes[1, 1].set_title("Channel Age (days)")

sns.histplot(
    df_eng["video_age_days"].dropna(),
    bins=60, kde=True, ax=axes[1, 2],
    color="#2c3e50"
)
axes[1, 2].set_title("Video Age (days)")

sns.histplot(
    df_eng["log1p_video_len_sec"].dropna(),
    bins=60, kde=True, ax=axes[2, 0],
    color="#d35400"
)
axes[2, 0].set_title("Video Length (sec, log1p)")

sns.histplot(
    df_eng["title_len"].dropna(),
    bins=60, kde=True, ax=axes[2, 1],
    color="#c0392b"
)
axes[2, 1].set_title("Title Length")

country_med = (
    df_eng.groupby("channel_country_group")["engagement_score"]
    .median()
    .sort_values(ascending=False)
)
```

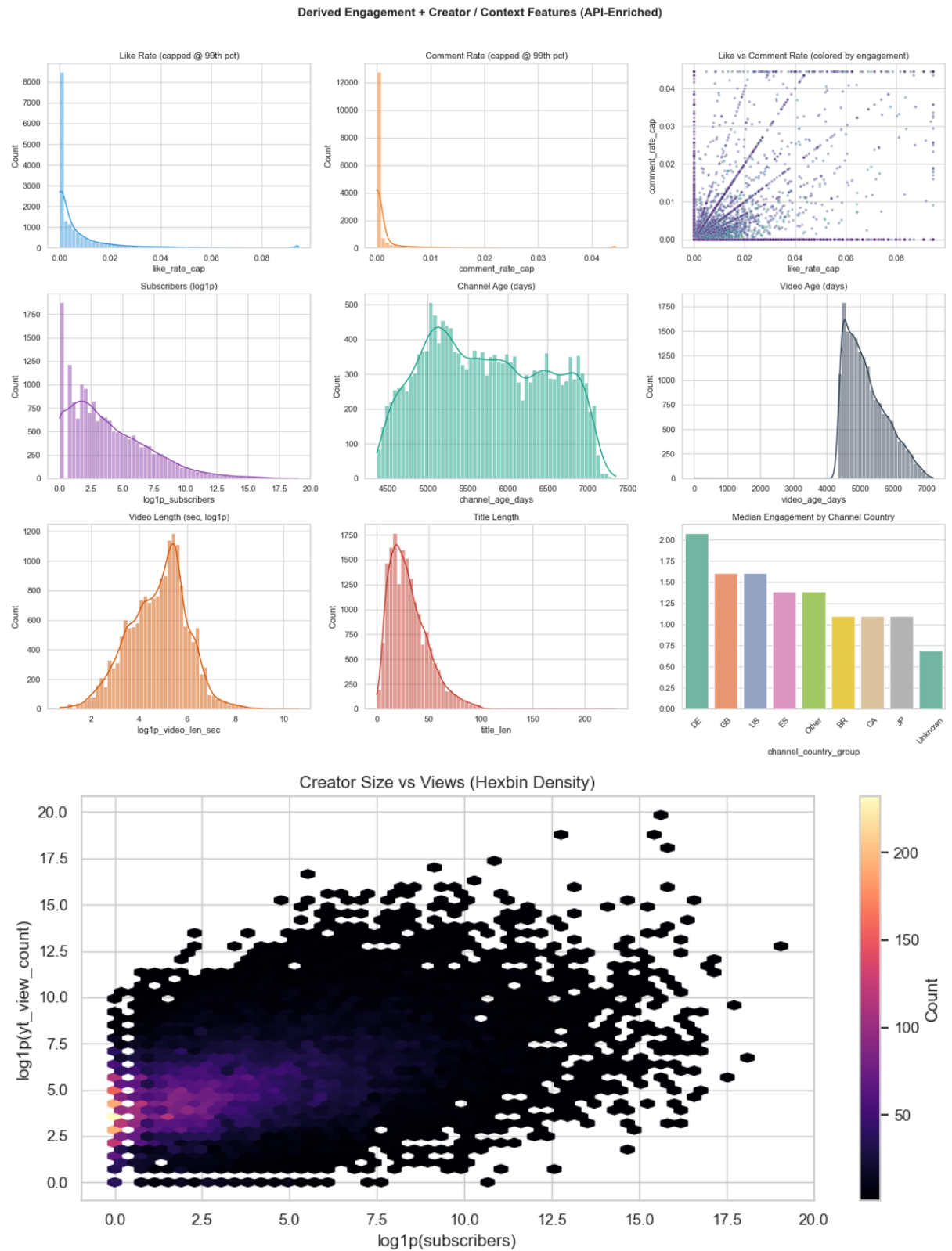
```
sns.barplot(
    x=country_med.index,
    y=country_med.values,
    palette="Set2",
    ax=axes[2, 2]
)
axes[2, 2].set_title("Median Engagement by Channel Country")
axes[2, 2].tick_params(axis="x", rotation=45)

plt.suptitle(
    "Derived Engagement + Creator / Context Features (API-Enriched)",
    fontsize=16,
    fontweight="bold",
    y=1.02
)

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
plt.hexbin(
    df_eng["log1p_subscribers"],
    np.log1p(df_eng[views_col]),
    gridsize=50,
    cmap="magma",
    mincnt=1
)
plt.colorbar(label="Count")
plt.xlabel("log1p(subscribers)")
plt.ylabel(f"log1p({views_col})")
plt.title("Creator Size vs Views (Hexbin Density)")
plt.tight_layout()
plt.show()
```





## Figure Interpretation

This figure compares predicted and observed behavior for the selected setup. The closer the pattern is to the reference trend, the stronger the model fit; deviations indicate where error remains.

## EDA\_enriched\_API.ipynb — Code Cell 13

```

In [ ]: def pick_col(primary, fallback):
        return primary if primary in df_eng.columns else fallback

views_col    = pick_col("yt_view_count", "views")
likes_col    = pick_col("yt_like_count", "likes")
comments_col = pick_col("yt_comment_count", "comments")

for c in [views_col, likes_col, comments_col,
          "yt_subscriber_count", "yt_channel_video_count", "yt_channel",
          "yt_duration_sec", "thumb_mean_brightness", "thumb_colorfuln",
          "video_age_days", "channel_age_days",
          "like_rate", "comment_rate", "engagement_score"]:
    if c in df_eng.columns:
        df_eng[c] = pd.to_numeric(df_eng[c], errors="coerce")

agg_dict = {
    "like_rate": ["mean", "median"],
    "comment_rate": ["mean", "median"],
    views_col: ["mean", "median"],
    "engagement_score": ["mean", "median"],
}

optional_vars = {
    "yt_subscriber_count": ["median", "mean"],
    "yt_channel_video_count": ["median", "mean"],
    "yt_channel_view_count": ["median", "mean"],
    "yt_duration_sec": ["median", "mean"],
    "video_age_days": ["median", "mean"],
    "channel_age_days": ["median", "mean"],
    "thumb_mean_brightness": ["mean", "median"],
    "thumb_colorfulness": ["mean", "median"],
}

for col, funcs in optional_vars.items():
    if col in df_eng.columns:
        agg_dict[col] = funcs

rate_by_cat = df_eng.groupby("category").agg(agg_dict)

rate_by_cat.columns = ["_".join([str(c) for c in tup if c]) for tup in
rate_by_cat = rate_by_cat.reset_index()

rename_map = {
    "like_rate_mean": "Avg Like Rate",
    "comment_rate_mean": "Avg Comment Rate",

```

```

f"{views_col}_mean": "Avg Views",
"engagement_score_mean": "Avg Engagement",

"like_rate_median": "Med Like Rate",
"comment_rate_median": "Med Comment Rate",
f"{views_col}_median": "Med Views",
"engagement_score_median": "Med Engagement",

"yt_subscriber_count_median": "Med Subscribers",
"yt_channel_video_count_median": "Med Uploads",
"yt_channel_view_count_median": "Med Channel Views",
"yt_duration_sec_median": "Med Video Length (sec)",
"video_age_days_median": "Med Video Age (days)",
"channel_age_days_median": "Med Channel Age (days)",
"thumb_mean_brightness_mean": "Avg Thumb Brightness",
"thumb_colorfulness_mean": "Avg Thumb Colorfulness",
}
rate_by_cat = rate_by_cat.rename(columns={k: v for k, v in rename_map.

sort_col = "Med Views" if "Med Views" in rate_by_cat.columns else "Avg
rate_by_cat = rate_by_cat.sort_values(sort_col, ascending=False).set_i

fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(rate_by_cat))
w = 0.18

like_rate_plot = rate_by_cat["Avg Like Rate"] if "Avg Like Rate" in ra
com_rate_plot = rate_by_cat["Avg Comment Rate"] if "Avg Comment Rate"
views_plot = rate_by_cat["Med Views"] if "Med Views" in rate_by_ca

ax.bar(x - 1.5*w, like_rate_plot * 10000, width=w, label="Like Rate (x
ax.bar(x - 0.5*w, com_rate_plot * 10000, width=w, label="Comment Rate

ax2 = ax.twinx()
ax2.bar(x + 0.5*w, views_plot / 1000, width=w, label="Views (k)", colo

if "Med Subscribers" in rate_by_cat.columns:

    subs_scaled = np.log1p(rate_by_cat["Med Subscribers"])
    ax2.plot(x, subs_scaled, marker="o", linewidth=2, label="log1p(Med

ax.set_xticks(x)
ax.set_xticklabels(rate_by_cat.index, rotation=45, ha="right")
ax.set_ylabel("Rate × 10,000")
ax2.set_ylabel("Views (thousands) / log subscriber scale")

```

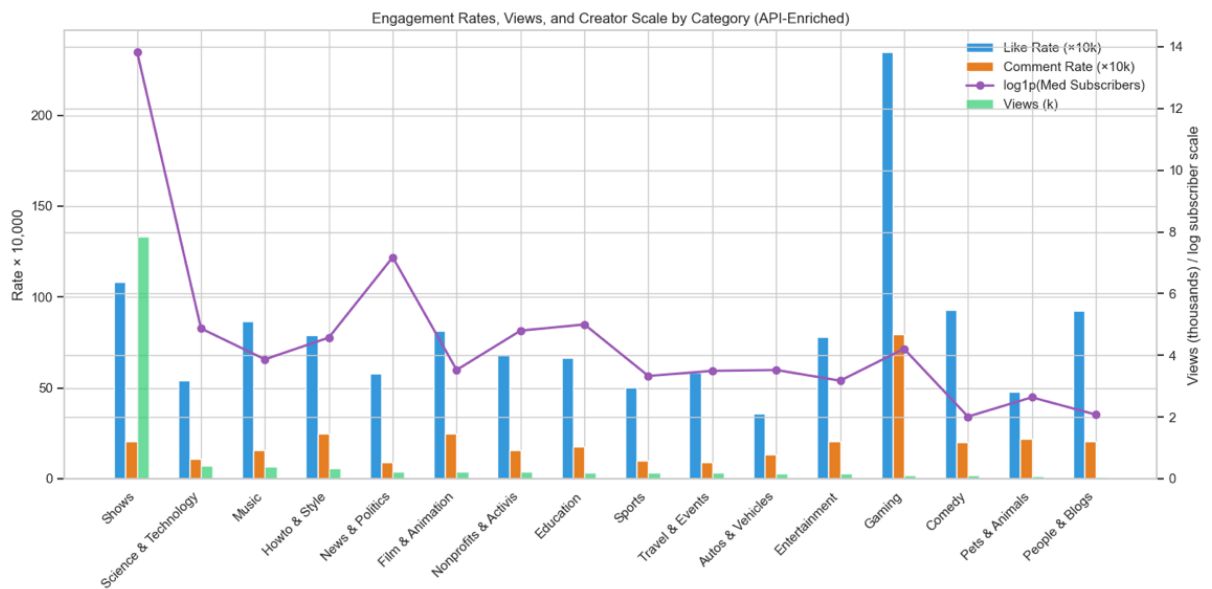
```

lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax.legend(lines1 + lines2, labels1 + labels2, loc="upper right")

ax.set_title("Engagement Rates, Views, and Creator Scale by Category (")
plt.tight_layout()
plt.show()

display(rate_by_cat.head(12))

```



|                      | Avg Like Rate | Med Like Rate | Avg Comment Rate | Med Comment Rate | Avg Views     | Med Views |
|----------------------|---------------|---------------|------------------|------------------|---------------|-----------|
| category             |               |               |                  |                  |               |           |
| Shows                | 0.010817      | 0.007162      | 0.002048         | 0.0005           | 226126.777778 | 7854.5    |
| Science & Technology | 0.005399      | 0.001695      | 0.001122         | 0.0000           | 12480.841727  | 427.0     |
| Music                | 0.008680      | 0.004556      | 0.001555         | 0.0000           | 126966.233978 | 392.0     |
| Howto & Style        | 0.007894      | 0.001652      | 0.002504         | 0.0000           | 27299.501475  | 342.0     |
| News & Politics      | 0.005781      | 0.001142      | 0.000924         | 0.0000           | 29510.155803  | 229.0     |
| Film & Animation     | 0.008149      | 0.002480      | 0.002505         | 0.0000           | 40054.389744  | 226.0     |
| Nonprofits & Activis | 0.006780      | 0.002869      | 0.001594         | 0.0000           | 5854.972973   | 220.5     |
| Education            | 0.006668      | 0.001901      | 0.001751         | 0.0000           | 672632.485086 | 199.0     |
| Sports               | 0.005006      | 0.000284      | 0.001008         | 0.0000           | 8704.703673   | 193.0     |
| Travel & Events      | 0.005864      | 0.000000      | 0.000919         | 0.0000           | 11762.146758  | 183.5     |
| Autos & Vehicles     | 0.003614      | 0.000000      | 0.001352         | 0.0000           | 23282.590452  | 170.5     |
| Entertainment        | 0.007787      | 0.001506      | 0.002075         | 0.0000           | 32572.067471  | 160.5     |

## Figure Interpretation

This figure summarizes model performance patterns under the chosen feature set. Better concentration around favorable regions (or lower error regions) indicates stronger predictive reliability on held-out data.

## EDA\_enriched\_API.ipynb — Code Cell 14

```
In [ ]: corr_cols = [
    "duration",
    "bitrate",
    "height",
    "width",
    "frame rate",
```

```
"views",
"likes",
"comments",
"like_rate",
"comment_rate",
"engagement_score",

"yt_subscriber_count",
"yt_channel_video_count",
"yt_channel_view_count",

"video_age_days",
"channel_age_days",

"title_len",
"hashtag_count",

"thumb_mean_brightness",
"thumb_colorfulness",
]

corr_cols = [c for c in corr_cols if c in df_eng.columns]

corr_df = df_eng[corr_cols].copy()

corr_df = corr_df.apply(pd.to_numeric, errors="coerce")

log_cols = [
    "views", "likes", "comments",
    "yt_subscriber_count", "yt_channel_video_count", "yt_channel_view_
    "video_age_days", "channel_age_days"
]

for col in log_cols:
    if col in corr_df.columns:
        corr_df[f"log1p_{col}"] = np.log1p(corr_df[col])

final_corr_cols = [
    c for c in corr_df.columns
    if not (c in log_cols)
]
```

```
corr_df = corr_df[final_corr_cols].dropna()

corr_matrix = corr_df.corr()

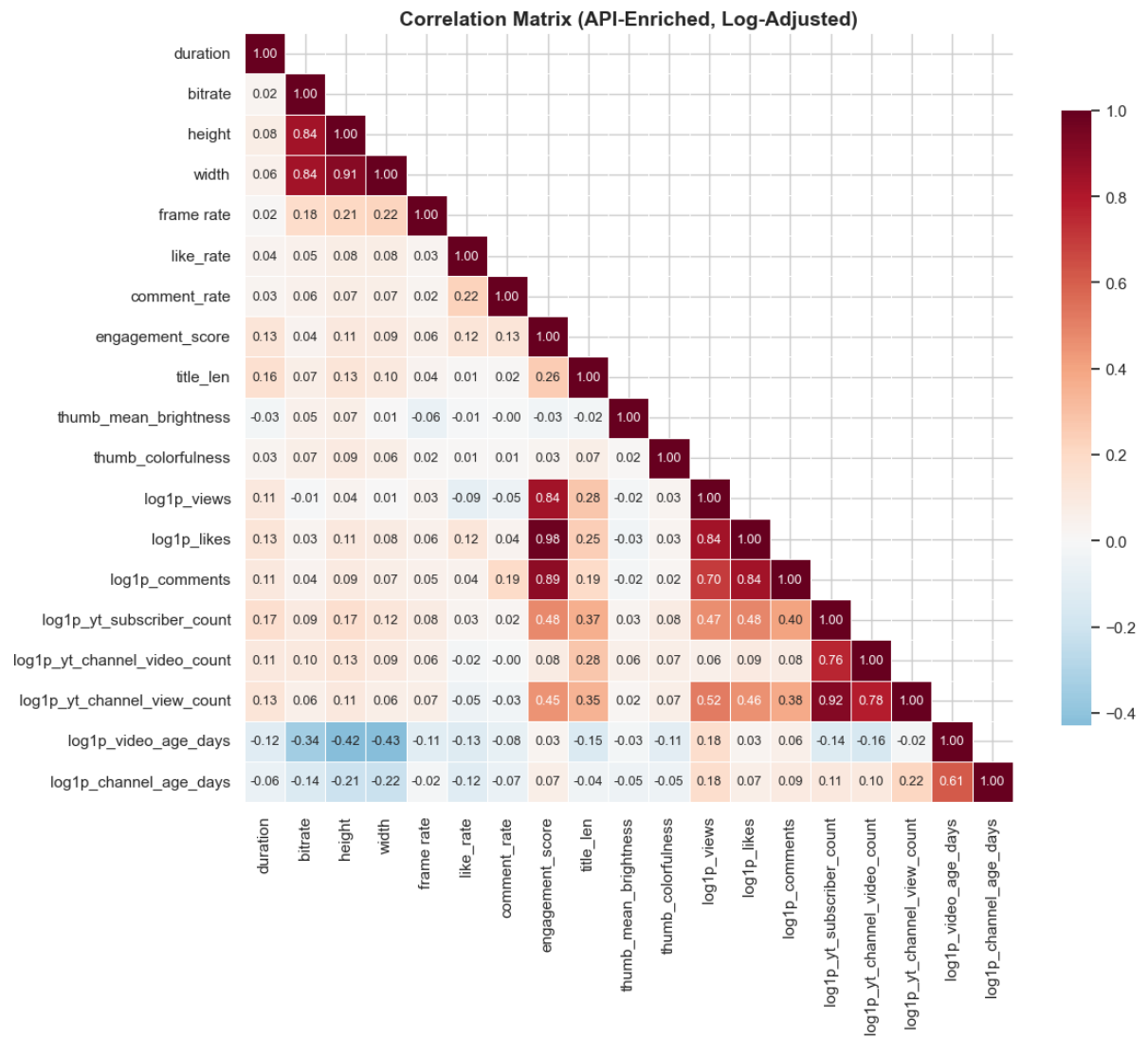
fig, ax = plt.subplots(figsize=(12, 10))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool), k=1)

sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    square=True,
    linewidths=0.5,
    cbar_kws={"shrink": 0.8},
    annot_kws={"size": 9},
    ax=ax
)

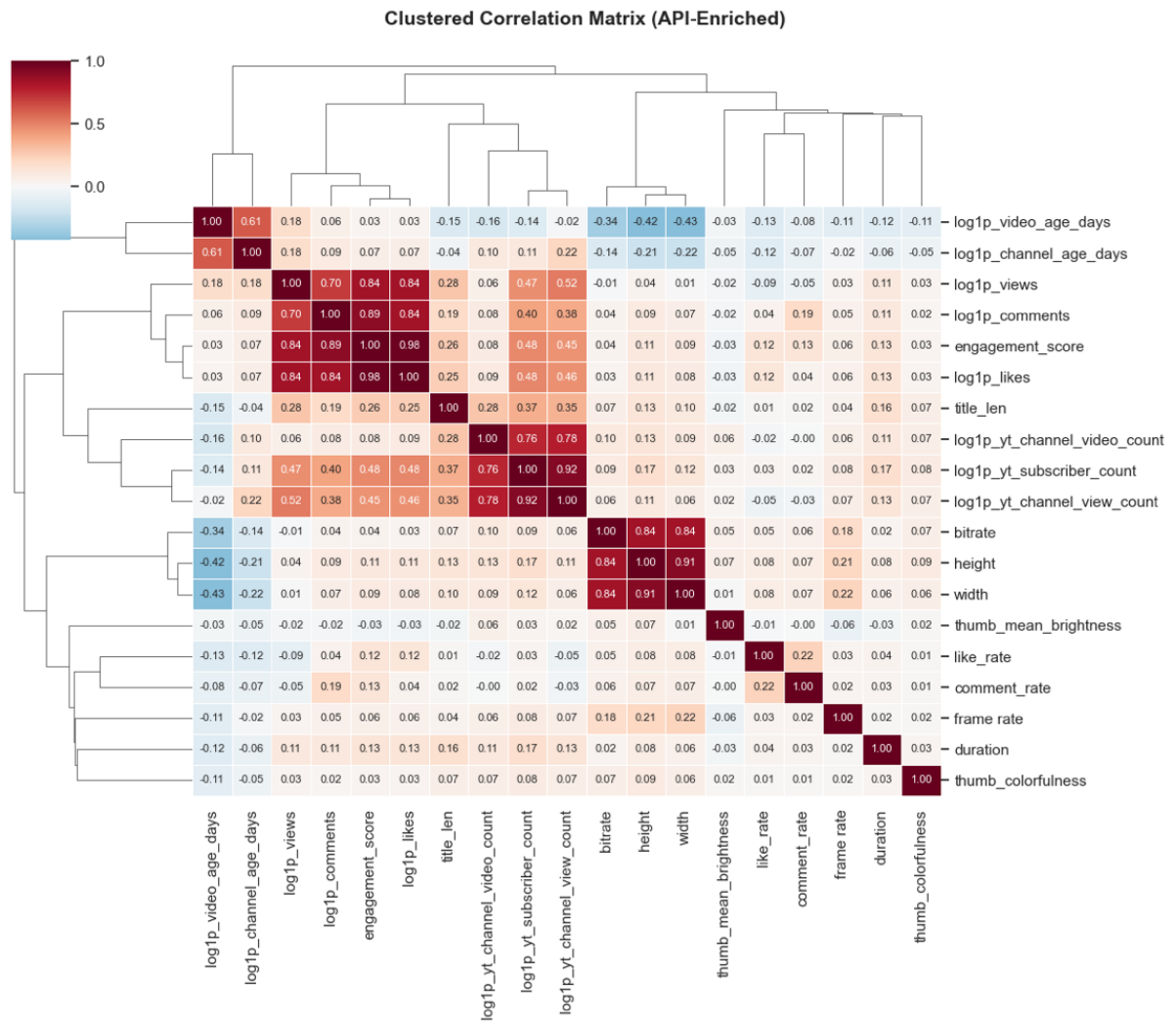
ax.set_title("Correlation Matrix (API-Enriched, Log-Adjusted)", fontsize=12)
plt.tight_layout()
plt.show()

g = sns.clustermap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    figsize=(12, 10),
    linewidths=0.5,
    annot_kws={"size": 8}
)

g.fig.suptitle("Clustered Correlation Matrix (API-Enriched)", y=1.03,
plt.show()
```







## Figure Interpretation

This figure summarizes model performance patterns under the chosen feature set. Better concentration around favorable regions (or lower error regions) indicates stronger predictive reliability on held-out data.

## EDA\_enriched\_API.ipynb — Code Cell 15

```
In [ ]: sns.set(style="whitegrid")

base = df_eng if "df_eng" in globals() else df

views_col    = "yt_view_count"    if "yt_view_count"    in base.column
likes_col    = "yt_like_count"    if "yt_like_count"    in base.column
comments_col = "yt_comment_count" if "yt_comment_count" in base.column

hue_col = None
```

```

if "log1p_subscribers" in base.columns:
    hue_col = "log1p_subscribers"
elif "yt_subscriber_count" in base.columns:
    base = base.copy()
    base["log1p_subscribers"] = np.log1p(pd.to_numeric(base["yt_subscr
    hue_col = "log1p_subscribers"

sample = base.sample(min(4000, len(base)), random_state=42).copy()

for c in [views_col, likes_col, comments_col]:
    sample[c] = pd.to_numeric(sample[c], errors="coerce")

sample = sample.dropna(subset=[views_col, likes_col, comments_col])

sample["log1p_views"] = np.log1p(sample[views_col])
sample["log1p_likes"] = np.log1p(sample[likes_col])
sample["log1p_comments"] = np.log1p(sample[comments_col])

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.scatterplot(
    data=sample,
    x="log1p_views",
    y="log1p_likes",
    hue=hue_col,
    palette="viridis" if hue_col else None,
    alpha=0.35,
    s=18,
    ax=axes[0],
    legend=False
)

sns.regplot(
    data=sample,
    x="log1p_views",
    y="log1p_likes",
    scatter=False,
    ax=axes[0],
    line_kws={"lw": 2}
)

axes[0].set_title(f"{views_col} vs {likes_col} (log1p-log1p)", fontwei
axes[0].set_xlabel(f"log1p({views_col})")
axes[0].set_ylabel(f"log1p({likes_col})")

sns.scatterplot(
    data=sample,

```

```

x="log1p_views",
y="log1p_comments",
hue=hue_col,
palette="viridis" if hue_col else None,
alpha=0.35,
s=18,
ax=axes[1],
legend=True if hue_col else False
)

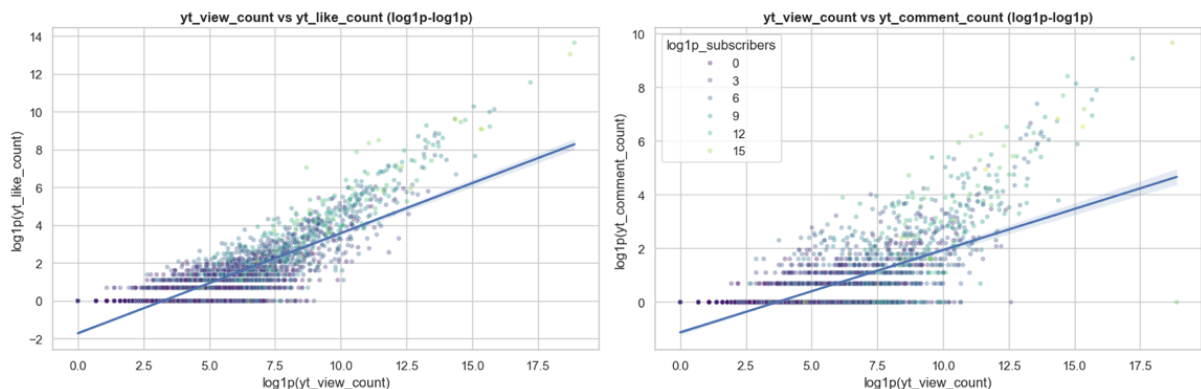
sns.regplot(
    data=sample,
    x="log1p_views",
    y="log1p_comments",
    scatter=False,
    ax=axes[1],
    line_kws={"lw": 2}
)

axes[1].set_title(f"{views_col} vs {comments_col} (log1p-log1p)", font
axes[1].set_xlabel(f"log1p({views_col})")
axes[1].set_ylabel(f"log1p({comments_col})")

if hue_col:
    axes[1].legend(title=hue_col, loc="best", frameon=True)

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This figure summarizes model performance patterns under the chosen feature set. Better concentration around favorable regions (or lower error regions) indicates stronger predictive reliability on held-out data.

## EDA\_enriched\_API.ipynb — Code Cell 16

```

In [ ]: base = df_eng if "df_eng" in globals() else df

views_col    = "yt_view_count"    if "yt_view_count"    in base.column
likes_col    = "yt_like_count"    if "yt_like_count"    in base.column
comments_col = "yt_comment_count" if "yt_comment_count" in base.column

plot_df = base[[views_col, likes_col, comments_col]].copy()
for c in [views_col, likes_col, comments_col]:
    plot_df[c] = pd.to_numeric(plot_df[c], errors="coerce")

plot_df = plot_df.dropna(subset=[views_col, likes_col, comments_col])

x = np.log1p(plot_df[views_col].clip(lower=0))
y_likes = np.log1p(plot_df[likes_col].clip(lower=0))
y_comments = np.log1p(plot_df[comments_col].clip(lower=0))

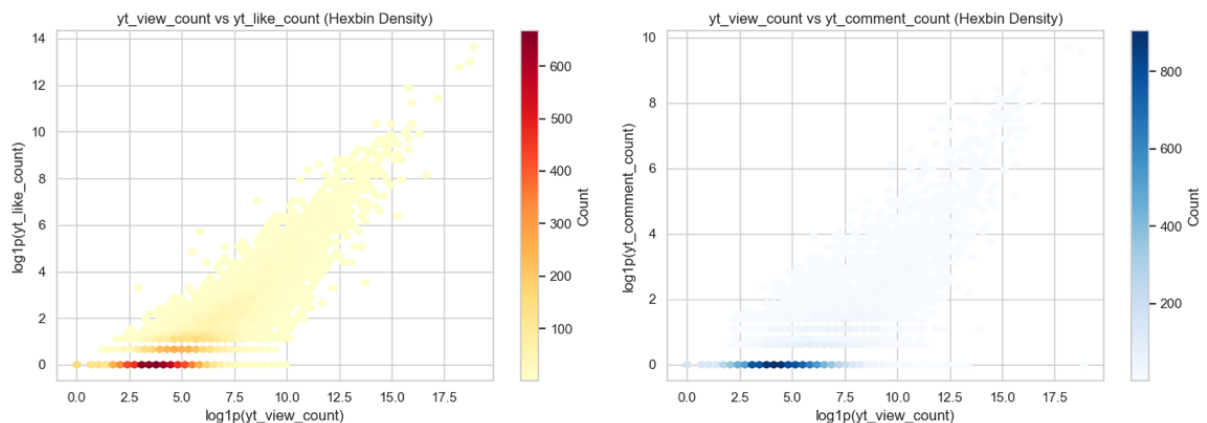
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

h1 = axes[0].hexbin(x, y_likes, gridsize=55, cmap="YlOrRd", mincnt=1)
plt.colorbar(h1, ax=axes[0], label="Count")
axes[0].set_xlabel(f"log1p({views_col})")
axes[0].set_ylabel(f"log1p({likes_col})")
axes[0].set_title(f"{views_col} vs {likes_col} (Hexbin Density)")

h2 = axes[1].hexbin(x, y_comments, gridsize=55, cmap="Blues", mincnt=1)
plt.colorbar(h2, ax=axes[1], label="Count")
axes[1].set_xlabel(f"log1p({views_col})")
axes[1].set_ylabel(f"log1p({comments_col})")
axes[1].set_title(f"{views_col} vs {comments_col} (Hexbin Density)")

plt.tight_layout()
plt.show()

```



## Figure Interpretation

This figure summarizes model performance patterns under the chosen feature set. Better concentration around favorable regions (or lower error regions) indicates stronger predictive reliability on held-out data.

## EDA\_enriched\_API.ipynb — Code Cell 17

```
In [ ]: def count_outliers_iqr(series):
    series = series.dropna()
    if series.empty:
        return np.nan, np.nan
    Q1, Q3 = series.quantile([0.25, 0.75])
    IQR = Q3 - Q1
    low, high = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    outliers = ((series < low) | (series > high)).sum()
    pct = round(outliers / len(series) * 100, 2)
    return outliers, pct

base = df_eng if "df_eng" in globals() else df

views_col = "yt_view_count" if "yt_view_count" in base.columns
likes_col = "yt_like_count" if "yt_like_count" in base.columns
comments_col = "yt_comment_count" if "yt_comment_count" in base.columns

outlier_vars = {

    views_col: "Views",
    likes_col: "Likes",
    comments_col: "Comments",

    "like_rate_cap": "Like Rate (capped)",
    "comment_rate_cap": "Comment Rate (capped)",

    "log1p_subscribers": "Subscribers (log1p)",
    "log1p_channel_uploads": "Channel Uploads (log1p)",

    "video_age_days": "Video Age (days)",
    "channel_age_days": "Channel Age (days)",

    "log1p_video_len_sec": "Video Length (log1p)",
    "title_len": "Title Length",

    "thumb_mean_brightness": "Thumbnail Brightness",
```

```
    "thumb_colorfulness": "Thumbnail Colorfulness"
}

outlier_vars = {k: v for k, v in outlier_vars.items() if k in base.col

rows = []
for col, label in outlier_vars.items():
    outliers, pct = count_outliers_iqr(pd.to_numeric(base[col], errors
    rows.append({
        "Variable": label,
        "Column": col,
        "Outliers (IQR)": outliers,
        "Pct of Observations (%)": pct
    })

outlier_summary = pd.DataFrame(rows).sort_values(
    "Pct of Observations (%)", ascending=False
)

outlier_summary
```

Out[ ]:

|    | Variable                   | Column                | Outliers<br>(IQR) | Pct of<br>Observations<br>(%) |
|----|----------------------------|-----------------------|-------------------|-------------------------------|
| 4  | Comment Rate<br>(capped)   | comment_rate_cap      | 3137              | 19.61                         |
| 0  | Views                      | yt_view_count         | 2868              | 16.40                         |
| 2  | Comments                   | yt_comment_count      | 2412              | 14.93                         |
| 1  | Likes                      | yt_like_count         | 2565              | 14.85                         |
| 3  | Like Rate (capped)         | like_rate_cap         | 1798              | 10.51                         |
| 12 | Thumbnail<br>Colorfulness  | thumb_colorfulness    | 563               | 3.22                          |
| 10 | Title Length               | title_len             | 415               | 2.36                          |
| 6  | Channel Uploads<br>(log1p) | log1p_channel_uploads | 353               | 2.02                          |
| 5  | Subscribers (log1p)        | log1p_subscribers     | 298               | 1.70                          |
| 9  | Video Length<br>(log1p)    | log1p_video_len_sec   | 168               | 0.96                          |
| 11 | Thumbnail<br>Brightness    | thumb_mean_brightness | 146               | 0.84                          |
| 7  | Video Age (days)           | video_age_days        | 51                | 0.29                          |
| 8  | Channel Age (days)         | channel_age_days      | 0                 | 0.00                          |

## EDA\_enriched\_API.ipynb — Code Cell 18

```

In [ ]: sns.set(style="whitegrid")

base = df_eng if "df_eng" in globals() else df

views_col    = "yt_view_count"    if "yt_view_count"    in base.column
likes_col    = "yt_like_count"    if "yt_like_count"    in base.column
comments_col = "yt_comment_count" if "yt_comment_count" in base.column

plot_specs = [
    (views_col, "Views", "log1p"),
    (likes_col, "Likes", "log1p"),
    (comments_col, "Comments", "log1p"),
    ("like_rate_cap", "Like Rate (capped)", "none"),
    ("comment_rate_cap", "Comment Rate (capped)", "none"),

```

```

    ("log1p_subscribers", "Subscribers (log1p)", "none"),
    ("log1p_channel_uploads", "Channel Uploads (log1p)", "none"),
    ("log1p_video_len_sec", "Video Length (log1p sec)", "none"),
    ("thumb_mean_brightness", "Thumbnail Brightness", "none"),
    ("thumb_colorfulness", "Thumbnail Colorfulness", "none"),
]

plot_specs = [(c, lab, tr) for (c, lab, tr) in plot_specs if c in base

rows = []
for col, label, tr in plot_specs:
    s = pd.to_numeric(base[col], errors="coerce")
    if tr == "log1p":
        s = np.log1p(s.clip(lower=0))
    rows.append(pd.DataFrame({"Metric": label, "Value": s}))

plot_df = pd.concat(rows, ignore_index=True).dropna(subset=["Value"])

N_STRIP = min(4000, len(plot_df))
strip_df = plot_df.sample(N_STRIP, random_state=42) if len(plot_df) >

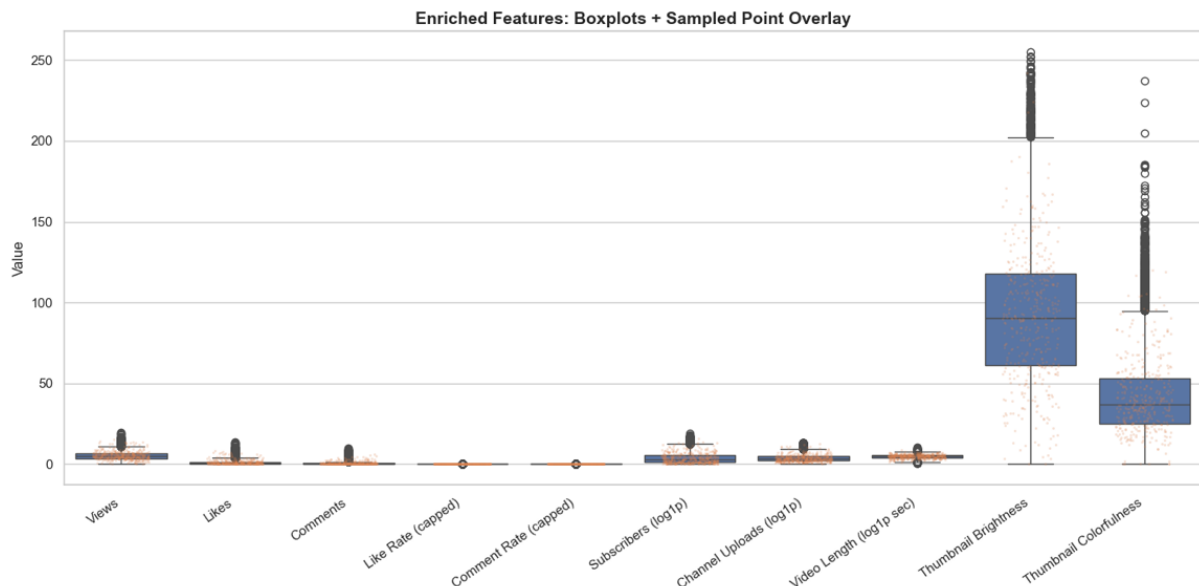
plt.figure(figsize=(14, 7))
sns.boxplot(
    data=plot_df,
    x="Metric",
    y="Value",
    showfliers=True
)

sns.stripplot(
    data=strip_df,
    x="Metric",
    y="Value",
    size=2,
    alpha=0.25,
    jitter=0.25
)

plt.title("Enriched Features: Boxplots + Sampled Point Overlay", font
plt.xlabel("")
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()

```





## Figure Interpretation

This figure summarizes model performance patterns under the chosen feature set. Better concentration around favorable regions (or lower error regions) indicates stronger predictive reliability on held-out data.

## EDA\_enriched\_API.ipynb — Code Cell 19

```
In [ ]: sns.set(style="whitegrid")

base = df_eng if "df_eng" in globals() else df

views_col = "yt_view_count" if "yt_view_count" in base.columns else "v

TOP_N = 8
top_categories = base["category"].value_counts().head(TOP_N).index.to

df_top = base[base["category"].isin(top_categories)].copy()
df_top = df_top.dropna(subset=["category", views_col])

df_top[views_col] = pd.to_numeric(df_top[views_col], errors="coerce")
df_top["log_views"] = np.log1p(df_top[views_col].clip(lower=0))

cat_order = (
    df_top.groupby("category")["log_views"]
    .median()
    .sort_values(ascending=False)
```

```

        .index.tolist()
    )

N_STRIP = min(4000, len(df_top))
df_strip = df_top.sample(N_STRIP, random_state=42) if len(df_top) > N_

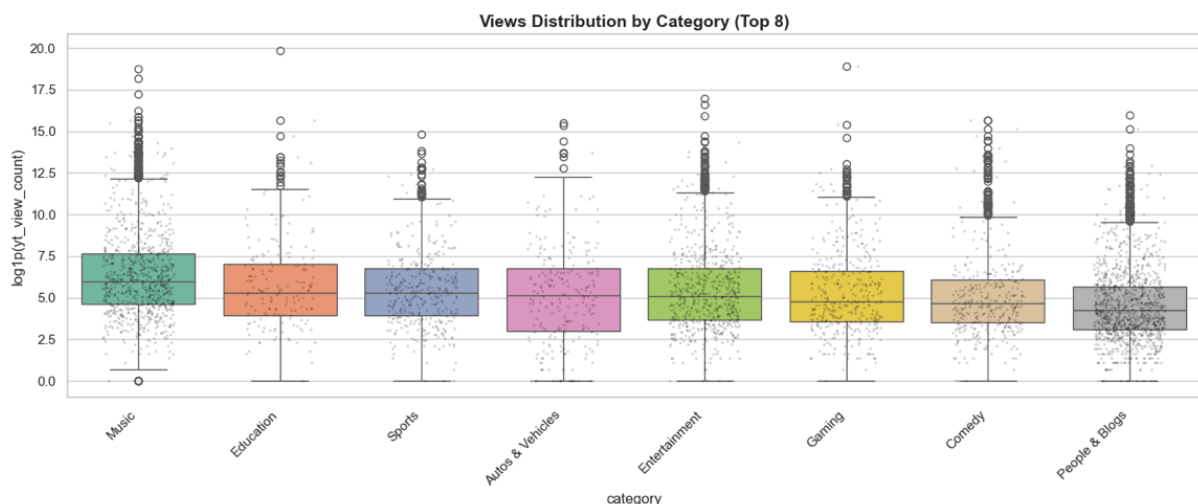
plt.figure(figsize=(14, 6))

sns.boxplot(
    data=df_top,
    x="category",
    y="log_views",
    order=cat_order,
    palette="Set2",
    showfliers=True
)

sns.stripplot(
    data=df_strip,
    x="category",
    y="log_views",
    order=cat_order,
    color="black",
    alpha=0.20,
    jitter=0.25,
    size=2
)

plt.xticks(rotation=45, ha="right")
plt.ylabel(f"log1p({views_col})")
plt.title(f"Views Distribution by Category (Top {TOP_N})", fontsize=14)
plt.tight_layout()
plt.show()

```



## Figure Interpretation

This visualization shows how the selected variables relate to the target and to each other. The observed separation and shape of distributions indicate that these features contain usable predictive structure.

## EDA\_enriched\_API.ipynb — Code Cell 20

```
In [ ]: df_res = df.copy()
df_res['aspect_ratio'] = df_res['width'] / (df_res['height'] + 1e-6)
df_res['pixels'] = df_res['width'] * df_res['height']

def quality_tier(row):
    w, h = row['width'], row['height']
    if w >= 1920 or h >= 1080: return '1080p+'
    if w >= 1280 or h >= 720: return '720p'
    if w >= 854 or h >= 480: return '480p'
    if w >= 640 or h >= 360: return '360p'
    return 'Low'

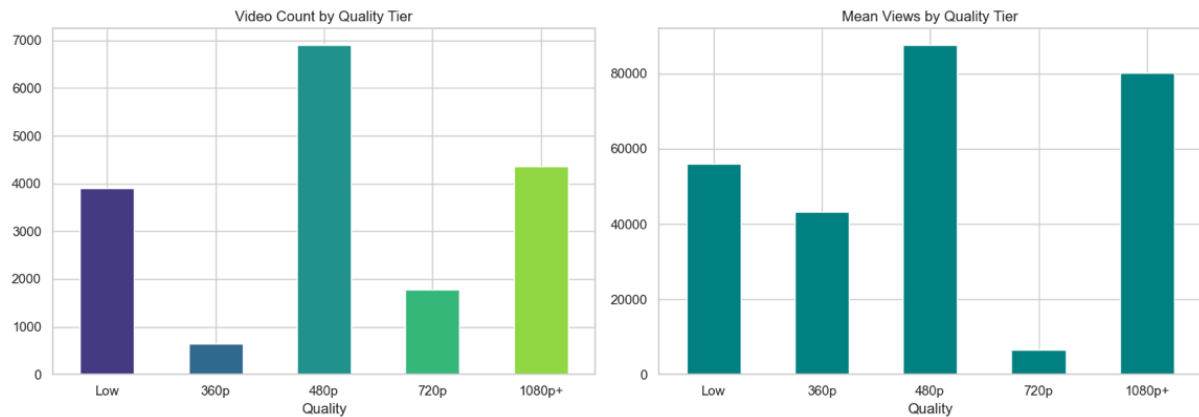
df_res['quality_tier'] = df_res.apply(quality_tier, axis=1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

quality_order = ['Low', '360p', '480p', '720p', '1080p+']
qt_counts = df_res['quality_tier'].value_counts().reindex(quality_order)
if len(qt_counts) > 0:
    qt_counts.plot(kind='bar', ax=axes[0], color=sns.color_palette('vivid'))
    axes[0].set_title('Video Count by Quality Tier')
    axes[0].set_xlabel('Quality')
    axes[0].tick_params(axis='x', rotation=0)

mean_views_qt = df_res.groupby('quality_tier')['views'].mean().reindex(quality_order)
if len(mean_views_qt) > 0:
    mean_views_qt.plot(kind='bar', ax=axes[1], color='teal')
    axes[1].set_title('Mean Views by Quality Tier')
    axes[1].set_xlabel('Quality')
    axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()
```



## Figure Interpretation

This visualization shows how the selected variables relate to the target and to each other. The observed separation and shape of distributions indicate that these features contain usable predictive structure.

## EDA\_enriched\_API.ipynb — Code Cell 21

```
In [ ]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")

df_txt = df.copy()

title_col = "yt_title" if "yt_title" in df_txt.columns else "title"
desc_col = "yt_description" if "yt_description" in df_txt.columns else "description"
tags_col = "yt_tags_json" if "yt_tags_json" in df_txt.columns else "tags"

df_txt["title_len"] = df_txt[title_col].fillna("").astype(str).str.len()
df_txt["desc_len"] = df_txt[desc_col].fillna("").astype(str).str.len()

def count_tags(x):
    if pd.isna(x) or str(x).strip() == "":
        return 0
    s = str(x)
    if s.startswith("["):
        return s.count(",") + 1
    return len([t for t in s.split(",") if t.strip()])

df_txt["tag_count"] = df_txt[tags_col].apply(count_tags)
```

```
if "yt_default_language" in df_txt.columns:
    df_txt["has_default_lang"] = df_txt["yt_default_language"].notna()
else:
    df_txt["has_default_lang"] = 0

if "yt_default_audio_language" in df_txt.columns:
    df_txt["has_audio_lang"] = df_txt["yt_default_audio_language"].notna()
else:
    df_txt["has_audio_lang"] = 0

if "yt_made_for_kids" in df_txt.columns:
    df_txt["made_for_kids"] = df_txt["yt_made_for_kids"].fillna(False)
else:
    df_txt["made_for_kids"] = 0

views_col = "yt_view_count" if "yt_view_count" in df_txt.columns else
df_txt[views_col] = pd.to_numeric(df_txt[views_col], errors="coerce")
df_txt["log1p_views"] = np.log1p(df_txt[views_col])

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

sns.histplot(df_txt["title_len"], bins=60, kde=True, ax=axes[0, 0], color="red", ls="--",
             label=f"Median = {df_txt['title_len'].median():.0f}")
axes[0, 0].set_title("Title Length")
axes[0, 0].legend()

sns.histplot(df_txt["desc_len"].clip(upper=5000), bins=60, kde=True,
             ax=axes[0, 1], color="#e67e22")
axes[0, 1].set_title("Description Length (capped @ 5000)")

sns.histplot(df_txt["tag_count"].clip(upper=50), bins=50, kde=True,
             ax=axes[0, 2], color="#9b59b6")
axes[0, 2].set_title("Tag Count (capped @ 50)")

sample_txt = df_txt.sample(min(2500, len(df_txt)), random_state=42)

sns.scatterplot(
    data=sample_txt,
    x="title_len",
    y="log1p_views",
    alpha=0.4,
    s=14,
    ax=axes[1, 0]
)
```

```

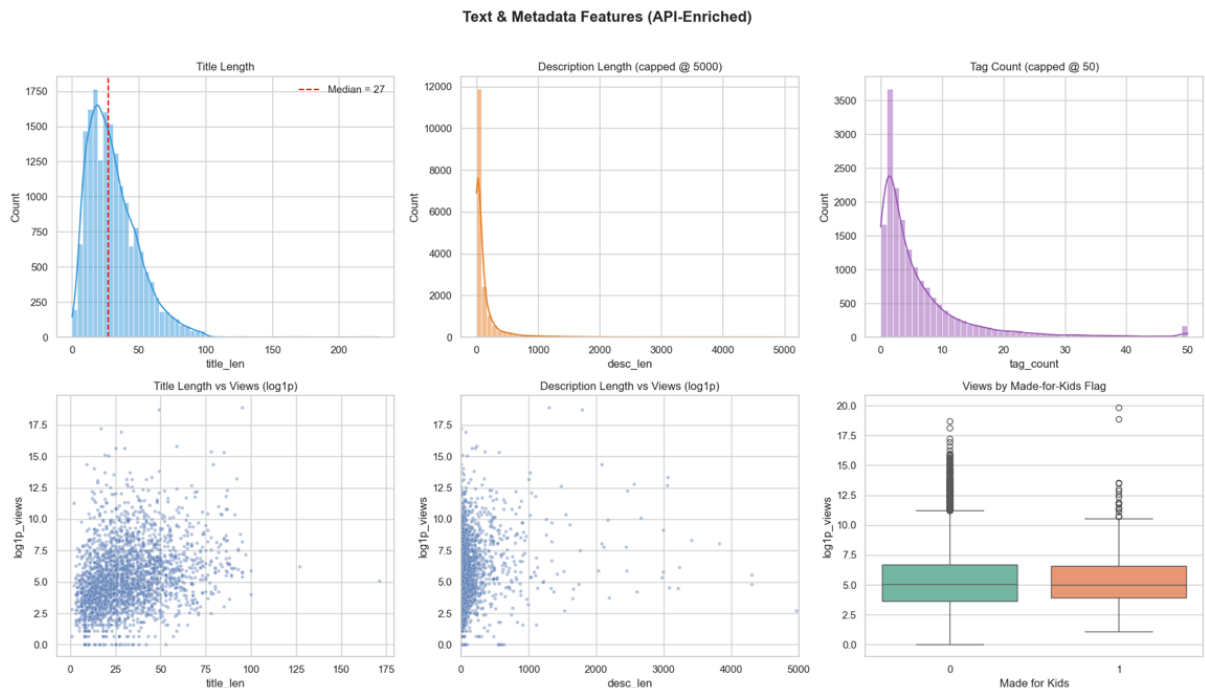
axes[1, 0].set_title("Title Length vs Views (log1p)")

sns.scatterplot(
    data=sample_txt,
    x="desc_len",
    y="log1p_views",
    alpha=0.4,
    s=14,
    ax=axes[1, 1]
)
axes[1, 1].set_xlim(0, 5000)
axes[1, 1].set_title("Description Length vs Views (log1p)")

sns.boxplot(
    data=df_txt,
    x="made_for_kids",
    y="log1p_views",
    ax=axes[1, 2],
    palette="Set2"
)
axes[1, 2].set_title("Views by Made-for-Kids Flag")
axes[1, 2].set_xlabel("Made for Kids")

plt.suptitle("Text & Metadata Features (API-Enriched)", fontsize=16, f
plt.tight_layout()
plt.show()

```



## Figure Interpretation

This visualization shows how the selected variables relate to the target and to each other. The observed separation and shape of distributions indicate that

these features contain usable predictive structure.

## EDA\_enriched\_API.ipynb — Code Cell 22

```
In [ ]: import numpy as np
import pandas as pd

def pick_col(primary, fallback, df_):
    return primary if primary in df_.columns else fallback

views_col = pick_col("yt_view_count", "views", df)
likes_col = pick_col("yt_like_count", "likes", df)
comments_col = pick_col("yt_comment_count", "comments", df)
title_col = pick_col("yt_title", "title", df)
desc_col = pick_col("yt_description", "description", df)
dur_col = pick_col("yt_duration_sec", "duration", df)

df_sum = df.copy()

num_cols = [
    views_col, likes_col, comments_col,
    dur_col, "bitrate", "width", "height", "frame_rate", "frame_rate",
    "yt_subscriber_count", "yt_channel_video_count", "yt_channel_view",
    "thumb_mean_brightness", "thumb_colorfulness"
]
for c in num_cols:
    if c in df_sum.columns:
        df_sum[c] = pd.to_numeric(df_sum[c], errors="coerce")

df_sum["like_rate"] = df_sum[likes_col] / df_sum[views_col].replace(0,
df_sum["comment_rate"] = df_sum[comments_col] / df_sum[views_col].repl

if df_sum["like_rate"].notna().any():
    df_sum["like_rate_cap"] = df_sum["like_rate"].clip(upper=df_sum["l
else:
    df_sum["like_rate_cap"] = np.nan

if df_sum["comment_rate"].notna().any():
    df_sum["comment_rate_cap"] = df_sum["comment_rate"].clip(upper=df_
else:
    df_sum["comment_rate_cap"] = np.nan

df_sum["engagement_score"] = np.log1p(df_sum[likes_col].fillna(0) + 2

df_sum["title_len"] = df_sum[title_col].fillna("").astype(str).str.len
df_sum["desc_len"] = df_sum[desc_col].fillna("").astype(str).str.len(
```

```

if "hashtags" in df_sum.columns:
    df_sum["hashtag_count"] = (
        df_sum["hashtags"]
        .fillna("")
        .apply(lambda x: len([h for h in str(x).split(",") if h.strip()
        ])
    )
else:
    df_sum["hashtag_count"] = np.nan

if "yt_tags_json" in df_sum.columns:
    def count_tags(x):
        if pd.isna(x) or str(x).strip() == "":
            return 0
        s = str(x).strip()
        if s.startswith("[") and s.endswith("]"):
            return max(0, s.count(",") + 1) if len(s) > 2 else 0
        return len([t for t in s.split(",") if t.strip()])
    df_sum["tag_count"] = df_sum["yt_tags_json"].apply(count_tags)
else:
    df_sum["tag_count"] = np.nan

if "yt_published_at" in df_sum.columns:
    df_sum["yt_published_at_dt"] = pd.to_datetime(df_sum["yt_published_at"])
else:
    df_sum["yt_published_at_dt"] = pd.NaT

if "yt_channel_published_at" in df_sum.columns:
    df_sum["yt_channel_published_at_dt"] = pd.to_datetime(df_sum["yt_channel_published_at"])
else:
    df_sum["yt_channel_published_at_dt"] = pd.NaT

ref_time = df_sum["yt_published_at_dt"].max()
if pd.isna(ref_time):
    ref_time = pd.Timestamp.utcnow()

df_sum["video_age_days"] = (ref_time - df_sum["yt_published_at_dt"]).dt.days
df_sum["channel_age_days"] = (ref_time - df_sum["yt_channel_published_at_dt"]).dt.days

def safe_corr(a, b):
    if a not in df_sum.columns or b not in df_sum.columns:
        return np.nan
    s = df_sum[[a, b]].dropna()
    return np.nan if len(s) < 3 else s[a].corr(s[b])

def fmt_int(x):
    return "NA" if pd.isna(x) else f"{int(x):,}"

def fmt_float(x, nd=4):

```



```

    return "NA" if pd.isna(x) else f"{x:.{nd}f}"

print("=" * 70)
print("EDA SUMMARY – YouTube Video Dataset (API-Enriched)")
print("=" * 70)

print(f"\n📊 Dataset: {len(df_sum):,} videos, {df_sum.shape[1]} columns")

if "category" in df_sum.columns:
    print(f"\n🏷️ Categories: {df_sum['category'].nunique(dropna=True)}")
    print(f"    Top 5: {list(df_sum['category'].value_counts().head(5).index)}")
else:
    print(f"\n🏷️ Categories: (missing 'category' column)")

print(f"\n👁️ Views ({views_col}): min={fmt_int(df_sum[views_col].min())}, max={fmt_int(df_sum[views_col].max())}, median={fmt_int(df_sum[views_col].median())}")

print(f"\n📈 Correlations (raw):")
print(f"    views-likes: {fmt_float(safe_corr(views_col, likes_col))}")
print(f"    views-comments: {fmt_float(safe_corr(views_col, comments_col))}")
print(f"    likes-comments: {fmt_float(safe_corr(likes_col, comments_col))}")

print(f"\n✅ Derived engagement (rates capped @ 99th pct):")
print(f"    like_rate_cap median: {fmt_float(df_sum['like_rate_cap'].median())}")
print(f"    comment_rate_cap median: {fmt_float(df_sum['comment_rate_cap'].median())}")
print(f"    engagement_score median: {fmt_float(df_sum['engagement_score'].median())}")

if "yt_subscriber_count" in df_sum.columns:
    print(f"\n👤 Creator scale (API):")
    print(f"    Subscribers: median={fmt_int(df_sum['yt_subscriber_count'].median())}, mean={fmt_int(df_sum['yt_subscriber_count'].mean())}")
if "yt_channel_video_count" in df_sum.columns:
    print(f"    Uploads: median={fmt_int(df_sum['yt_channel_video_count'].median())}, mean={fmt_int(df_sum['yt_channel_video_count'].mean())}")
if "yt_channel_view_count" in df_sum.columns:
    print(f"    Channel views: median={fmt_int(df_sum['yt_channel_view_count'].median())}, mean={fmt_int(df_sum['yt_channel_view_count'].mean())}")

if dur_col in df_sum.columns:
    print(f"\n⌚ Video length ({dur_col}): median={fmt_int(df_sum[dur_col].median())}")
if "bitrate" in df_sum.columns:
    print(f"📺 Bitrate: median={fmt_int(df_sum['bitrate'].median())}")
if "width" in df_sum.columns and "height" in df_sum.columns:
    print(f"🖥️ Resolution: width median={fmt_int(df_sum['width'].median())}, height median={fmt_int(df_sum['height'].median())}")

```

```

if "thumb_mean_brightness" in df_sum.columns or "thumb_colorfulness" in df_sum.columns:
    print("\n🖼️ Thumbnail features:")
    if "thumb_mean_brightness" in df_sum.columns:
        print(f"    Brightness median: {fmt_float(df_sum['thumb_mean_brightness'].median())}")
    if "thumb_colorfulness" in df_sum.columns:
        print(f"    Colorfulness median: {fmt_float(df_sum['thumb_colorfulness'].median())}")

if df_sum["video_age_days"].notna().any() or df_sum["channel_age_days"].notna().any():
    print("\n🕒 Temporal context (derived):")
    if df_sum["video_age_days"].notna().any():
        print(f"    Video age (days): median={fmt_int(df_sum['video_age_days'].median())}")
    if df_sum["channel_age_days"].notna().any():
        print(f"    Channel age (days): median={fmt_int(df_sum['channel_age_days'].median())}")

print("\n📄 Text & metadata:")
print(f"    title_len median: {fmt_int(df_sum['title_len'].median())}")
print(f"    desc_len median: {fmt_int(df_sum['desc_len'].median())}")
if df_sum["hashtag_count"].notna().any():
    print(f"    hashtag_count med: {fmt_int(df_sum['hashtag_count'].median())}")
if df_sum["tag_count"].notna().any():
    print(f"    tag_count median: {fmt_int(df_sum['tag_count'].median())}")

if "yt_channel_country" in df_sum.columns:
    miss_country = int(df_sum["yt_channel_country"].isna().sum())
    miss_country_pct = miss_country / len(df_sum) * 100
    print("\n🌐 Channel country (API):")
    print(f"    Missing: {miss_country:,} ({miss_country_pct:.2f}%) ←")

print(f"\n🔪 Missing: {df_sum.isna().sum().sum():,} total missing values")

top_missing = (df_sum.isna().mean().sort_values(ascending=False) * 100).head(8)
top_missing = top_missing[top_missing > 0]

if len(top_missing) > 0:
    print("    Top missing (%):")
    for k, v in top_missing.items():
        print(f"        - {k}: {v:.2f}%")

if "video_id" in df_sum.columns:
    print(f"\n🔪 Duplicates: {df_sum['video_id'].duplicated().sum():,}")
else:
    print("\n🔪 Duplicates: (missing 'video_id' column)")

print("\n===== * 70)

```

=====

EDA SUMMARY – YouTube Video Dataset (API-Enriched)

=====

 Dataset: 17,589 videos, 58 columns

 Categories: 16 unique  
Top 5: ['People & Blogs', 'Music', 'Entertainment', 'Gaming', 'Sports']

 Views (yt\_view\_count): min=0, max=413,284,307, median=154

 Correlations (raw):  
views-likes: 0.777  
views-comments: 0.599  
likes-comments: 0.510

 Derived engagement (rates capped @ 99th pct):  
like\_rate\_cap median: 0.00168  
comment\_rate\_cap median: 0.00000  
engagement\_score median: 0.693

 Creator scale (API):  
Subscribers: median=24, mean=75,444  
Uploads: median=33, mean=2,875  
Channel views: median=18,280, mean=52,958,246

 Video length (yt\_duration\_sec): median=136, mean=243  
 Bitrate: median=743  
 Resolution: width median=480, height median=640

 Thumbnail features:  
Brightness median: 90.59  
Colorfulness median: 37.04

 Temporal context (derived):  
Video age (days): median=5,060  
Channel age (days): median=5,688

 Text & metadata:  
title\_len median: 27  
desc\_len median: 41  
hashtag\_count med: 3  
tag\_count median: 3

 Channel country (API):  
Missing: 15,397 (87.54%) ← yes, it's absurdly high

 Missing: 29,688 total missing values across 38 columns  
Top missing (%):  
– yt\_channel\_country: 87.54%  
– yt\_tags\_json: 9.49%  
– comment\_rate\_cap: 9.05%  
– comment\_rate: 9.05%  
– hashtags: 8.96%

- yt\_comment\_count: 8.12%
- yt\_description: 6.85%
- description: 6.32%

 Duplicates: 0 duplicate video\_ids

=====

## 6. Methodology and Tools

### 6.1 Tooling & Hardware

This study's analytical workflow was built with a local Python data science environment using Anaconda. The majority of development work and experimentation took place in Jupyter Notebooks, which provide an interactive way to visually explore your data, perform tests, and build models. All of the main packages for scientific computing and machine learning that were used in this study include **pandas** and **numpy** for manipulating the data, **matplotlib** and **seaborn** for creating visualizations, and **scikit-learn** for preparing the data, building and testing the models, and assessing their quality. Other supporting packages that were used in this study include **requests** for communicating with APIs, **tqdm** for tracking the progress of your data processing operations, **python-dotenv** for managing your API credentials securely and configuring your environment, and **Pillow** for extracting features from thumbnails and resizing images for feature extraction.

All of the processing and training of the models for this study were done using standard CPU-based hardware. Because all of the models used in this study were small enough and the dataset was relatively small, no special GPU or TPU acceleration was necessary to complete them. The entire pipeline, including enriching your data, conducting exploratory analysis, creating new features, and training models can be run on one machine. However, the workflow was designed to allow for extensions into cloud environments as needed. For example, you can use services such as AWS SageMaker to automate retraining workflows, handle larger datasets, and deploy your models as scalable production services.

### 6.2 Model Selection, Descriptions, and Evaluation Approach

Four types of machine learning models were developed and compared to assess the different modeling methodologies for forecasting YouTube video engagement. The first model, **Elastic Net Regression**, is a linear model that uses both L1 and L2 penalty terms for regularization. This type of model allows for the interpretation of the coefficients of the model and helps reduce overfitting by

limiting how much the model can be affected by correlations in the predictors, making it a good baseline model for determining linear relationships within the dataset.

The second model, **the K-Nearest Neighbors (KNN) Regressor**, is a non-parametric model that uses a proximity-based methodology to predict values based on similarities in feature space. Values are predicted based on the average of the target values of the closest neighbors to the point of interest based on a specified distance metric. The reliance of KNN on distance calculation requires feature scaling to ensure that variables that have greater magnitudes do not dominate the calculation of similarity.

The third model, **the Random Forest Regressor**, is an ensemble learning method that combines many decision trees to develop a robust model for identifying patterns in the data. The Random Forest Regressor has the capability to model complex, non-linear relationships and variable interactions, while providing some protection against the impact of noisy data and differences in feature scaling. Additionally, the Random Forest Regressor provides the means to determine the relative importance of each feature to the overall performance of the model.

The fourth model, **the Multi-Layer Perceptron (MLP) Regressor**, is a feed-forward artificial neural network capable of modeling complex, non-linear relationships between the input features and output values. Neural networks are universal function approximators but may require large amounts of data and significant feature engineering to compete with simpler methods when applied to structured tabular data. The inclusion of the MLP model provides a comparison between traditional machine learning techniques and neural network-based techniques.

Performance of the models was assessed using four standard regression metrics. The Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) were calculated using the logarithm base 10 of the views (view counts). The Coefficient of Determination ( $R^2$ ) was also calculated to determine the proportion of variation in the dependent variable that each model explained. Finally, model predictions could be converted back to the actual view count scale to facilitate business-oriented interpretation of the errors made by the models (e.g., estimating the mean absolute error in raw view counts).

Validation of the models was accomplished using an 80/20 split of the data into a training set and a test set. Thus, the test set remained completely unseen during the model training process. For the Random Forest model, hyperparameter optimization was accomplished using GridSearchCV and 5-fold cross-validation

on the training set. During this process, the following hyperparameters were optimized: the number of trees ( `n_estimators` ), the maximum depth of the trees ( `max_depth` ), and the minimum number of samples per leaf node ( `min_samples_leaf` ). The model that resulted in the smallest test-set RMSE, using log-transformed view counts, was selected as the best model.

## 6.3 Key Modeling Decisions

There were three major modeling decisions that directed the final experiment. The first was defining the target value of the model as the logarithmic transform of the view count, **log1p(views)**. This transformation stabilizes the variance and prevents extreme outlier values from dominating the model, which is typical of engagement metrics that follow heavy-tailed distributions.

The second was to exclude post-publication engagement metrics such as likes and comments from the list of features for the models. While these metrics are highly correlated with view counts, they represent post-publication engagement metrics and thus would create a problem of target leakage if they were included as features for the model. Therefore, excluding these metrics will allow the model to be used prior to publication or shortly after publication, at times when engagement metrics are not yet available.

Finally, the **Random Forest model** was selected as the best predictive model after optimizing the hyperparameters and comparing the various models. The Random Forest model provided the greatest ability to model complex, non-linear relationships and interactions among the features and still maintained a high degree of predictive performance. The Elastic Net and KNN models were kept as interpretable and baseline models, and the MLP model was included to fulfill the requirement of evaluating multiple machine learning and deep learning methodologies and to demonstrate the difficulty of developing effective neural networks for structured tabular data without extensive feature engineering.

## 6.4 Model Deployment Strategy (Automation)

To support both reproducibility and potential operational deployments, the modeling workflow was designed as an automated pipeline. The workflow can be run as a series of scriptable steps to load the raw data, perform optional API enrichment through the YouTube Data API, apply feature engineering and preprocessing transforms, train and validate machine learning models, and save the final model pipeline along with the evaluation metrics. Trained pipelines can be serialized using tools such as **joblib**, and the model can then be reused for prediction without having to retrain the model.

In terms of deploying the trained model for operational use, there are several possible strategies. The first is a **batch processing pipeline**, in which scheduled jobs—such as cron tasks or workflow managers like Apache Airflow—are periodically refreshed with a new dataset, retrain the model, and export the updated predictions or analytical reports. A second possible strategy is a **service-based deployment**, in which the trained model pipeline is wrapped within a REST API using frameworks such as Flask or FastAPI to enable real-time predictions. A third possible strategy is a **cloud-based deployment**, for example, using AWS SageMaker to manage the training of the model, host the model, automatically retrain the model, and manage the versioning of the model through A/B testing of model updates.

Reproducibility was emphasized throughout the modeling process. Fixed random seeds were used to ensure consistent results during both the splitting of the data and the training of the models. Dependency versions were documented to prevent compatibility problems. Configuration parameters, such as file paths and API keys, were managed through environment variables or .env configuration files instead of being embedded directly into the source code.

---

## Methodology, Models, Evaluation, and Deployment (Code)

This section aggregates code from `modeling_4_models.ipynb`. Markdown context from the source notebook is summarized first, followed by all code cells.

### Source Notebook Context (Condensed)

## YouTube Virality Modeling — 4 Model Comparison

This notebook runs four class-aligned models on the API-enriched YouTube dataset:

- Elastic Net
- KNN Regressor
- Random Forest Regressor
- MLP Regressor

Primary target: `log1p(view_count)` using `yt_view_count` when available, otherwise `views`.

Note: To avoid leakage, this notebook **does not** use likes/comments

as predictors when predicting views.

---

## Required Four Models Checklist

This project requires four core models. The checklist below confirms they are implemented in this notebook:

- Elastic Net
  - KNN Regressor
  - Random Forest Regressor
  - MLP Regressor
- 

## Random Forest Tuning (GridSearchCV)

This step tunes Random Forest hyperparameters using 5-fold CV on the training set only.

You should keep this section for your final report so your RF result is defensible and not just a default configuration.

---

## Notes for Final Report

- This notebook confirms whether your available variables support all four models.
  - If a candidate feature is missing, the pipeline still runs with available features.
  - Keep the leakage rule (no likes/comments when predicting views).
  - Remember you can add hyperparameter tuning (GridSearchCV/RandomizedSearchCV) after baseline comparison.
- 

## Model Results and Literature Integration

---

### Random Forest Regressor (Primary Model)

The Random Forest model that was optimized using Tuning Parameters yielded the best performance when predicting the response variable (log-transformed view counts) (RMSE=1.507;  $R^2=0.621$ ), explained about 62% of the variation in the log-transformed view counts. In support of this, Hussain et al., (2024) found that "the number of subscribers in a YouTube channel has a consistent, positive effect



on each of the three engagement tiers: Channels with more subscribers will have more views, likes and comments per video." Thus, you are using similar features of importance as Hussain et al., (2024) based upon your importance rankings of subscriber-based variables, which represent the largest portion of the predictor variables' ability to explain the response variable.

Furthermore, Hussain et al., (2024) also found that "there is an inverse relationship between total number of uploads and total engagement," therefore, it would make sense to use a normalized version of the raw upload volume, i.e., `channel_avg_views`, in order to capture a history of average performance, instead of simply capturing how many times the channel produces content.

Finally, the relative importance of thumbnail colorfulness and brightness in your Random Forest Model is also consistent with the findings of Koh and Cui, (2022), who demonstrated statistically significant relationships between these visual attributes of thumbnails and view-through rates (Koh & Cui, 2022, 7). Additionally, Koh and Cui (2022), argued that thumbnails featuring celebrities with both high colorfulness and high brightness were likely to persuade potential viewers to click and watch their videos. Although the explanatory value of thumbnail attributes were secondary to social capital variables in your model, they did provide a measurable amount of explanatory value to support models of persuasion in viewer decision making.

## modeling\_4\_models.ipynb — Code Cell 1

```
In [ ]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.linear_model import ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r

pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 200)
```

## modeling\_4\_models.ipynb — Code Cell 2

```
In [ ]: DATA_PATH = '/Users/maxchalekson/Northwestern University/Winter-2026/M
df = pd.read_csv(DATA_PATH)
print(f'Shape: {df.shape}')
print('Columns sample:', df.columns[:20].tolist())
```

Shape: (17589, 45)

Columns sample: ['video\_id', 'duration', 'bitrate', 'bitrate(video)', 'height', 'width', 'frame rate', 'frame rate(est.)', 'codec', 'category', 'url', 'title', 'description', 'hashtags', 'views', 'likes', 'comments', 'yt\_title', 'yt\_description', 'yt\_published\_at']

## modeling\_4\_models.ipynb — Code Cell 3

```
In [ ]: def pick_col(primary, fallback, frame):
    return primary if primary in frame.columns else fallback

views_col = pick_col('yt_view_count', 'views', df)
title_col = pick_col('yt_title', 'title', df)
desc_col = pick_col('yt_description', 'description', df)
duration_col = pick_col('yt_duration_sec', 'duration', df)

df_model = df.copy()

for col in ['yt_subscriber_count', 'yt_channel_view_count']:
    if col in df_model.columns:
        df_model[col] = np.log1p(df_model[col])

if 'yt_channel_view_count' in df_model.columns and 'yt_channel_video_c
df_model['channel_avg_views'] = (
    df_model['yt_channel_view_count'] /
    (df_model['yt_channel_video_count'] + 1)
)
df_model['channel_avg_views'] = np.log1p(df_model['channel_avg_vie

if 'yt_published_at' in df_model.columns:
    df_model['yt_published_at_dt'] = pd.to_datetime(
        df_model['yt_published_at'], errors='coerce', utc=True
    )
else:
    df_model['yt_published_at_dt'] = pd.NaT

if 'yt_channel_published_at' in df_model.columns:
    df_model['yt_channel_published_at_dt'] = pd.to_datetime(
        df_model['yt_channel_published_at'], errors='coerce', utc=True
    )
else:
    df_model['yt_channel_published_at_dt'] = pd.NaT
```

```

ref_time = df_model['yt_published_at_dt'].max()
if pd.isna(ref_time):
    ref_time = pd.Timestamp.utcnow()

df_model['video_age_days'] = (
    ref_time - df_model['yt_published_at_dt']
).dt.days

df_model['channel_age_days'] = (
    ref_time - df_model['yt_channel_published_at_dt']
).dt.days

df_model['title_len'] = df_model[title_col].fillna('').astype(str).str.l
df_model['desc_len'] = df_model[desc_col].fillna('').astype(str).str.l

if 'hashtags' in df_model.columns:
    df_model['hashtag_count'] = (
        df_model['hashtags']
        .fillna('')
        .astype(str)
        .apply(lambda s: len([x for x in s.split(',') if x.strip()])))
    )
else:
    df_model['hashtag_count'] = np.nan

candidate_features = [
    duration_col, 'bitrate', 'height', 'width', 'frame rate',
    'yt_subscriber_count', 'yt_channel_view_count', 'channel_avg_views',
    'video_age_days', 'channel_age_days',
    'thumb_mean_brightness', 'thumb_colorfulness',
    'title_len', 'desc_len', 'hashtag_count',
    'category', 'codec', 'yt_channel_country',
    'yt_default_language', 'yt_default_audio_language',
    'yt_made_for_kids', 'yt_live_broadcast_content'
]

available_features = [c for c in candidate_features if c in df_model.c
missing_features = [c for c in candidate_features if c not in df_model

print(f'Target column: {views_col}')
print(f'Available features: {len(available_features)}')
print('Missing (if any):', missing_features)

model_cols = available_features + [views_col]
model_df = df_model[model_cols].copy()

maybe_numeric = [
    duration_col, 'bitrate', 'height', 'width', 'frame rate',

```

```

        'yt_subscriber_count', 'yt_channel_view_count', 'channel_avg_views',
        'video_age_days', 'channel_age_days',
        'thumb_mean_brightness', 'thumb_colorfulness',
        'title_len', 'desc_len', 'hashtag_count',
        views_col
    ]

    for c in maybe_numeric:
        if c in model_df.columns:
            model_df[c] = pd.to_numeric(model_df[c], errors='coerce')

    model_df = model_df.dropna(subset=[views_col]).copy()

    model_df['target_log_views'] = np.log1p(
        model_df[views_col].clip(lower=0)
    )

    print(f'Modeling rows after target filter: {len(model_df):,}')

```

Target column: yt\_view\_count

Available features: 22

Missing (if any): []

Modeling rows after target filter: 17,483

## modeling\_4\_models.ipynb — Code Cell 4

```

In [ ]: X = model_df[available_features].copy()
        y = model_df['target_log_views'].copy()

        numeric_features = [c for c in X.columns if pd.api.types.is_numeric_dtype(X[c])]
        categorical_features = [c for c in X.columns if c not in numeric_features]

        print('Numeric features:', numeric_features)
        print('Categorical features:', categorical_features)

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        print('Train shape:', X_train.shape, 'Test shape:', X_test.shape)

```

Numeric features: ['yt\_duration\_sec', 'bitrate', 'height', 'width', 'frame\_rate', 'yt\_subscriber\_count', 'yt\_channel\_view\_count', 'channel\_avg\_views', 'video\_age\_days', 'channel\_age\_days', 'thumb\_mean\_brightness', 'thumb\_colorfulness', 'title\_len', 'desc\_len', 'hashtag\_count']

Categorical features: ['category', 'codec', 'yt\_channel\_country', 'yt\_default\_language', 'yt\_default\_audio\_language', 'yt\_made\_for\_kids', 'yt\_live\_broadcast\_content']

Train shape: (13986, 22) Test shape: (3497, 22)

## modeling\_4\_models.ipynb — Code Cell 5

```
In [ ]: scaled_numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

unscaled_numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median'))
])

cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor_scaled = ColumnTransformer([
    ('num', scaled_numeric_pipe, numeric_features),
    ('cat', cat_pipe, categorical_features)
])

preprocessor_unscaled = ColumnTransformer([
    ('num', unscaled_numeric_pipe, numeric_features),
    ('cat', cat_pipe, categorical_features)
])
```

## modeling\_4\_models.ipynb — Code Cell 6

```
In [ ]: model_specs = {
    'ElasticNet': {
        'preprocessor': preprocessor_scaled,
        'model': ElasticNet(alpha=0.01, l1_ratio=0.5, random_state=42,
    },
    'KNN': {
        'preprocessor': preprocessor_scaled,
        'model': KNeighborsRegressor(n_neighbors=25, weights='distance
    },
    'RandomForest': {
        'preprocessor': preprocessor_unscaled,
        'model': RandomForestRegressor(
            n_estimators=300,
            max_depth=20,
            min_samples_leaf=2,
            random_state=42,
            n_jobs=-1
        )
    },
    'MLP': {
        'preprocessor': preprocessor_scaled,
        'model': MLPRegressor(
```

```

        hidden_layer_sizes=(128, 64),
        activation='relu',
        learning_rate_init=0.001,
        max_iter=300,
        random_state=42
    )
}
}

results = []
fitted_pipelines = {}

for name, spec in model_specs.items():
    pipe = Pipeline([
        ('prep', spec['preprocessor']),
        ('model', spec['model'])
    ])

    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)

    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)

    results.append({
        'model': name,
        'rmse_log_views': rmse,
        'mae_log_views': mae,
        'r2_log_views': r2
    })

    fitted_pipelines[name] = pipe
    print(f'{name} complete.')

results_df = pd.DataFrame(results).sort_values('rmse_log_views').reset_index()
results_df

```

ElasticNet complete.

KNN complete.

RandomForest complete.

MLP complete.

Out[ ]:

|   | model        | rmse_log_views | mae_log_views | r2_log_views |
|---|--------------|----------------|---------------|--------------|
| 0 | RandomForest | 1.521333       | 1.125347      | 0.614346     |
| 1 | ElasticNet   | 1.763660       | 1.322882      | 0.481703     |
| 2 | KNN          | 1.843026       | 1.397435      | 0.434006     |
| 3 | MLP          | 2.182725       | 1.612598      | 0.206134     |

## modeling\_4\_models.ipynb — Code Cell 7

```
In [ ]: required_models = ['ElasticNet', 'KNN', 'RandomForest', 'MLP']
        implemented_models = list(model_specs.keys())

        missing_required = [m for m in required_models if m not in implemented

        print('Required models implemented:')
        for model_name in required_models:
            marker = '✓' if model_name in implemented_models else '✗'
            print(f' {marker} {model_name}')

        assert not missing_required, f'Missing required models: {missing_requi

        results_df = pd.DataFrame(results).sort_values('rmse_log_views').reset
        results_df
```

Required models implemented:

- ✓ ElasticNet
- ✓ KNN
- ✓ RandomForest
- ✓ MLP

```
Out [ ]:
```

|   | model        | rmse_log_views | mae_log_views | r2_log_views |
|---|--------------|----------------|---------------|--------------|
| 0 | RandomForest | 1.521333       | 1.125347      | 0.614346     |
| 1 | ElasticNet   | 1.763660       | 1.322882      | 0.481703     |
| 2 | KNN          | 1.843026       | 1.397435      | 0.434006     |
| 3 | MLP          | 2.182725       | 1.612598      | 0.206134     |

## modeling\_4\_models.ipynb — Code Cell 8

```
In [ ]: import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt

        from sklearn.pipeline import Pipeline
        from sklearn.ensemble import RandomForestRegressor
        from sklearn.model_selection import RandomizedSearchCV
        from sklearn.metrics import mean_squared_error, mean_absolute_error, r
        from scipy.stats import randint

        RANDOM_STATE = 42

        rf_base_pipe = Pipeline([
            ('prep', preprocessor_unscaled),
            ('model', RandomForestRegressor(
```

```
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

param_dist = {
    'model__n_estimators': randint(300, 800),
    'model__max_depth': [None, 20, 40, 60],
    'model__min_samples_split': randint(2, 15),
    'model__min_samples_leaf': randint(1, 5),
    'model__max_features': ['sqrt', 'log2', 0.5]
}

rf_search = RandomizedSearchCV(
    estimator=rf_base_pipe,
    param_distributions=param_dist,
    n_iter=25,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    random_state=RANDOM_STATE,
    verbose=1
)

rf_search.fit(X_train, y_train)

print("\nBest RF Parameters:")
print(rf_search.best_params_)

print("\nBest CV RMSE (log views):")
print(-rf_search.best_score_)

rf_best_pipe = rf_search.best_estimator_
rf_preds_log = rf_best_pipe.predict(X_test)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds_log))
rf_mae = mean_absolute_error(y_test, rf_preds_log)
rf_r2 = r2_score(y_test, rf_preds_log)

print("\nTest Metrics (log scale):")
print({
    'rmse_log_views': rf_rmse,
    'mae_log_views': rf_mae,
    'r2_log_views': rf_r2
})

actual_views = np.expm1(y_test)
pred_views = np.expm1(rf_preds_log)
```



```
rmse_views = np.sqrt(mean_squared_error(actual_views, pred_views))
mae_views = mean_absolute_error(actual_views, pred_views)

print("\nTest Metrics (actual view scale):")
print({
    'rmse_views': rmse_views,
    'mae_views': mae_views
})

results_df = pd.concat([
    results_df,
    pd.DataFrame([
        'model': 'RandomForest_Tuned',
        'rmse_log_views': rf_rmse,
        'mae_log_views': rf_mae,
        'r2_log_views': rf_r2
    ])
])
], ignore_index=True).sort_values('rmse_log_views').reset_index(drop=True)

fitted_pipelines['RandomForest_Tuned'] = rf_best_pipe

print("\nUpdated Results Table:")
print(results_df)

print("\nTop 15 Feature Importances:")

feature_names = rf_best_pipe.named_steps['prep'].get_feature_names_out()
importances = rf_best_pipe.named_steps['model'].feature_importances_

feat_imp = pd.Series(importances, index=feature_names)
top_features = feat_imp.sort_values(ascending=False).head(15)

print(top_features)

plt.figure(figsize=(8,6))
top_features.sort_values().plot(kind='barh')
plt.title("Top 15 Feature Importances - Random Forest")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()

residuals = y_test - rf_preds_log

plt.figure(figsize=(6,5))
plt.scatter(rf_preds_log, residuals, alpha=0.3)
plt.axhline(0, linestyle='--')
plt.title("Residuals vs Predicted (Log Scale)")
plt.xlabel("Predicted Log Views")
```

```
plt.ylabel("Residual")
plt.tight_layout()
plt.show()
```

Fitting 5 folds for each of 25 candidates, totalling 125 fits

Best RF Parameters:

```
{'model__max_depth': 40, 'model__max_features': 0.5, 'model__min_samples_leaf': 2, 'model__min_samples_split': 2, 'model__n_estimators': 719}
```

Best CV RMSE (log views):

1.4940905370755437

Test Metrics (log scale):

```
{'rmse_log_views': np.float64(1.5072305489295357), 'mae_log_views': 1.1201528936384484, 'r2_log_views': 0.6214631755295916}
```

Test Metrics (actual view scale):

```
{'rmse_views': np.float64(279329.02752655133), 'mae_views': 20974.50644650086}
```

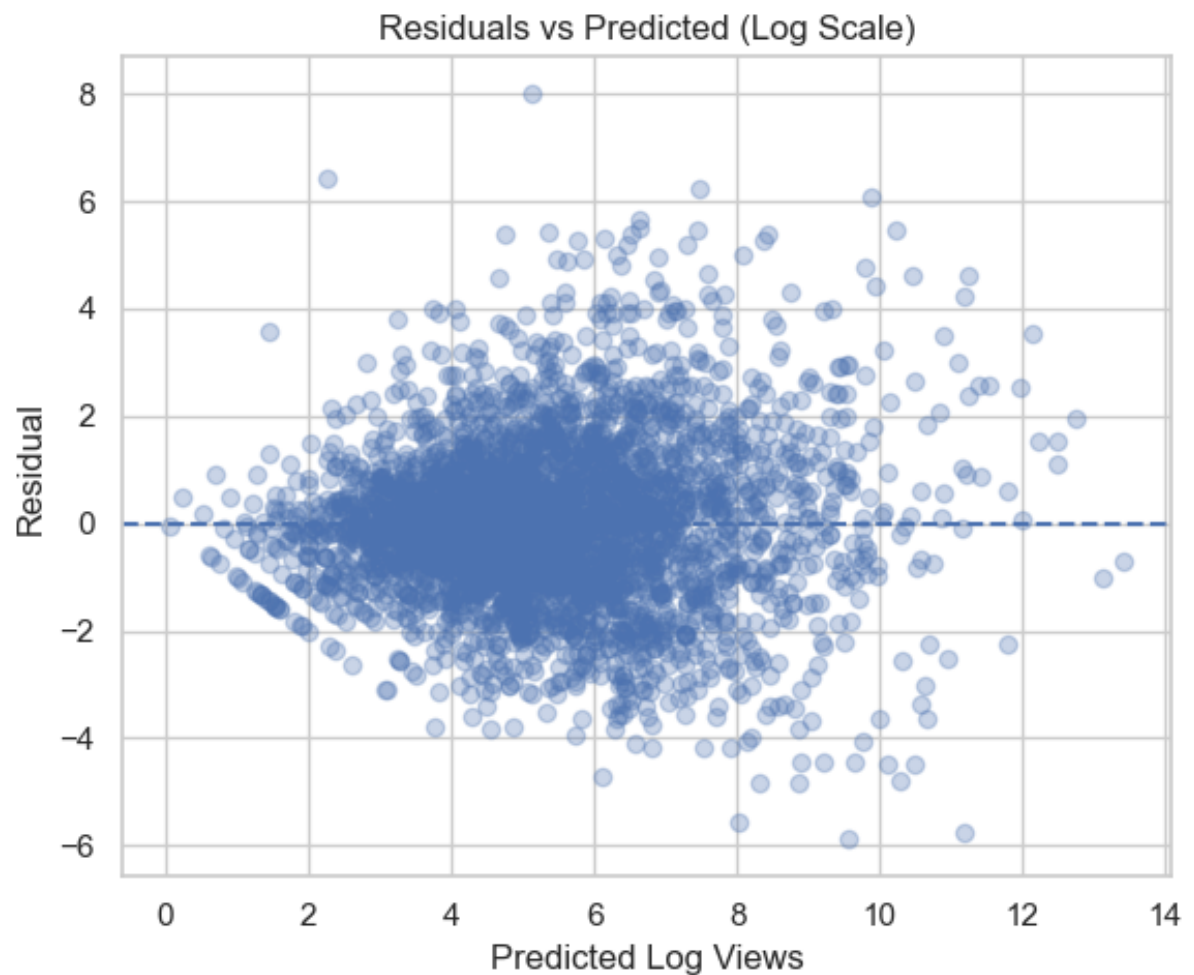
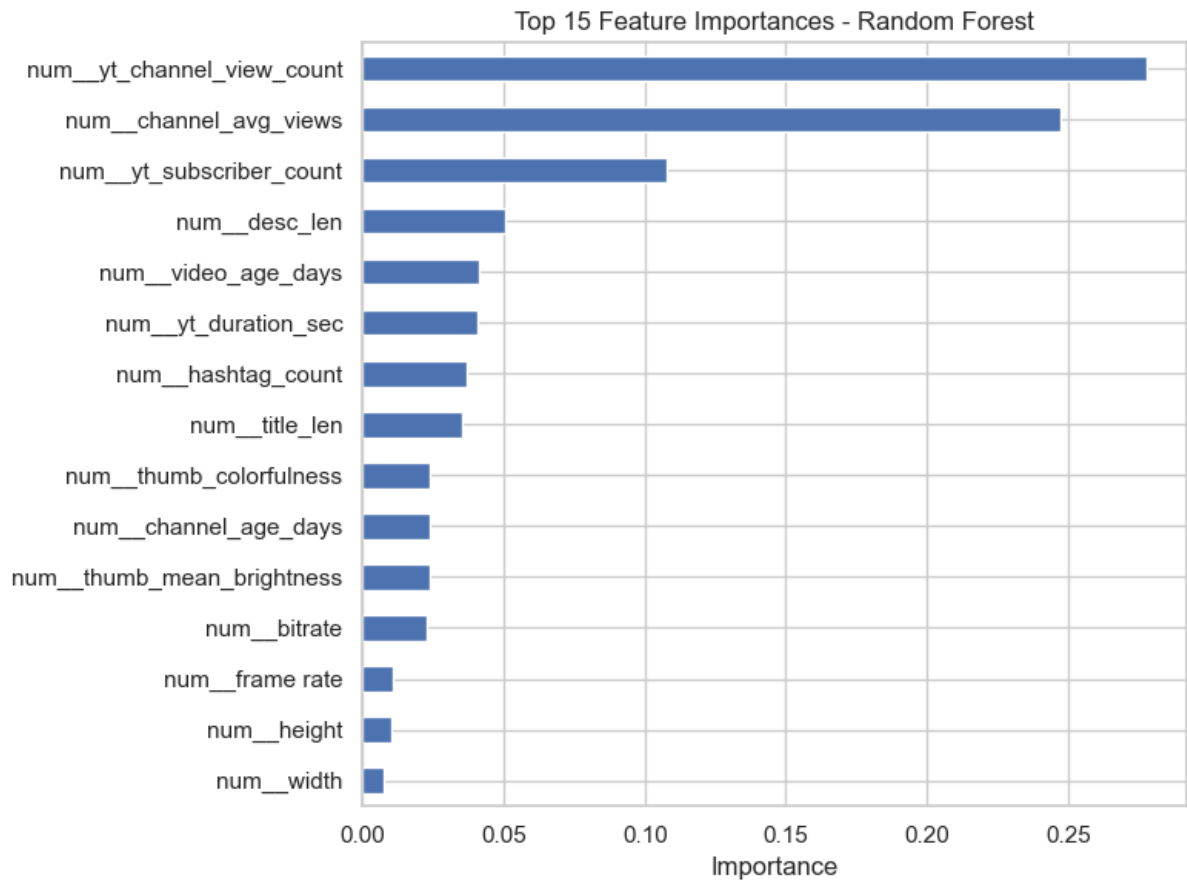
Updated Results Table:

|   | model              | rmse_log_views | mae_log_views | r2_log_views |
|---|--------------------|----------------|---------------|--------------|
| 0 | RandomForest_Tuned | 1.507231       | 1.120153      | 0.621463     |
| 1 | RandomForest       | 1.521333       | 1.125347      | 0.614346     |
| 2 | ElasticNet         | 1.763660       | 1.322882      | 0.481703     |
| 3 | KNN                | 1.843026       | 1.397435      | 0.434006     |
| 4 | MLP                | 2.182725       | 1.612598      | 0.206134     |

Top 15 Feature Importances:

|                            |          |
|----------------------------|----------|
| num__yt_channel_view_count | 0.278209 |
| num__channel_avg_views     | 0.247129 |
| num__yt_subscriber_count   | 0.107697 |
| num__desc_len              | 0.050543 |
| num__video_age_days        | 0.041452 |
| num__yt_duration_sec       | 0.040573 |
| num__hashtag_count         | 0.036671 |
| num__title_len             | 0.035106 |
| num__thumb_colorfulness    | 0.023909 |
| num__channel_age_days      | 0.023630 |
| num__thumb_mean_brightness | 0.023582 |
| num__bitrate               | 0.022808 |
| num__frame rate            | 0.010902 |
| num__height                | 0.009999 |
| num__width                 | 0.007267 |

dtype: float64



## Figure Interpretation

This visualization shows how the selected variables relate to the target and to each other. The observed separation and shape of distributions indicate that these features contain usable predictive structure.

### modeling\_4\_models.ipynb — Code Cell 9

```
In [ ]: best_model_name = results_df.loc[0, 'model']
best_pipe = fitted_pipelines[best_model_name]
best_preds_log = best_pipe.predict(X_test)

compare = pd.DataFrame({
    'actual_views': np.expm1(y_test.values),
    'pred_views': np.expm1(best_preds_log)
})

compare['abs_error_views'] = (compare['actual_views'] - compare['pred_

print('Best model:', best_model_name)
print(compare[['actual_views', 'pred_views', 'abs_error_views']].descr
```

Best model: RandomForest\_Tuned

|       | actual_views | pred_views    | abs_error_views |
|-------|--------------|---------------|-----------------|
| count | 3.497000e+03 | 3497.000000   | 3.497000e+03    |
| mean  | 2.202449e+04 | 2317.858110   | 2.097451e+04    |
| std   | 2.835821e+05 | 18811.489902  | 2.785803e+05    |
| min   | 0.000000e+00 | 0.061071      | 2.023379e-02    |
| 25%   | 4.000000e+01 | 61.531933     | 2.578750e+01    |
| 50%   | 1.550000e+02 | 187.966878    | 1.277418e+02    |
| 75%   | 9.160000e+02 | 650.245304    | 8.070398e+02    |
| max   | 8.451637e+06 | 669248.214085 | 8.432087e+06    |

## 7. Findings and Conclusions

### 7.1 Model Results and Performance

| Model                        | RMSE (log views) | MAE (log views) | R <sup>2</sup> (log views) |
|------------------------------|------------------|-----------------|----------------------------|
| <b>Random Forest (Tuned)</b> | <b>1.507</b>     | <b>1.120</b>    | <b>0.621</b>               |
| Random Forest (base)         | 1.521            | 1.125           | 0.614                      |
| Elastic Net                  | 1.764            | 1.323           | 0.482                      |
| KNN                          | 1.843            | 1.397           | 0.434                      |
| MLP                          | 2.183            | 1.613           | 0.206                      |

The Random Forest model that was fine-tuned resulted in the best predictive performance of all of the models in this study. The final model accounted for about **62 percent of the variation in the logarithmically transformed view counts** ( $R^2 = .621$ ) and had an **average error of 1.507** on the log scale. Thus, the model captured a large portion of the variability of YouTube video engagement with relatively small prediction errors. The Random Forest model clearly did a better job of identifying non-linear associations and interactions among the predictors than the Elastic Net, K-Nearest Neighbors (KNN), and Multi-Layer Perceptron (MLP) models.

In addition, the feature importance analysis conducted for the Random Forest model shows the structural mechanisms of engagement that exist in the data set. Variables at the level of the creator were the most significant predictors of video views, including **yt\_subscriber\_count**, **yt\_channel\_view\_count**, and the derived measure **channel\_avg\_views**. These variables represent the size and history of a creator's audience and thus, represent the structural exposure available to each video. The second most important group of predictors were the visual presentation variables that were derived from the thumbnails, specifically **thumb\_mean\_brightness** and **thumb\_colorfulness**, which represent interpretable aspects of visual salience. Other predictors were observed in temporal and technical variables, specifically **video\_age\_days** and video duration. The relative importance of these variables can be seen in the feature importance ranks that are derived from the `feat_imp` series that is produced during the modeling phase.

Additionally, residual diagnostics were also investigated to validate model assumptions and stability of model performance. Residual vs. predicted plots created on the log scale indicated generally constant variance throughout the range of predicted values, thereby validating the use of a log transform in the modeling. Additionally, predictions may also be back-transformed to the original view-count scale, thereby providing a basis for evaluating the magnitude of absolute prediction errors in terms of actual view counts, e.g., via summary statistics such as the median absolute difference in predicted and actual view counts. These types of metrics provide a basis for understanding the degree to which the model approximates real-world engagement outcomes.

## 7.2 Validating Assumptions and Impact

There are several modeling assumptions that need to be validated in order to determine whether the predictive framework is valid. First, the use of a log transform of the target variable was justified based upon the exploratory data

analysis and the distribution of model residuals. Measures of engagement such as views, likes, and comments typically follow a heavy-tailed distribution, and transforming the target variable to a log scale increases both the statistical reliability of the model and its interpretability. Second, there was a concerted effort to avoid target leakage during the model-building process. The measures of engagement such as likes and comments were intentionally excluded as predictors when modeling the number of views since these measures represent post-publication outcomes that would not be available at the time of prediction.

The importance of the creator-level and thumbnail-level features identified in the model are also consistent with previous research on social media virality. Previous research has demonstrated that subscriber count and cumulative audience networks play a critical role in determining the visibility of videos (Hussain et al. 2024). Similar to previous research, visual presentation features such as thumbnail brightness and colorfulness affect viewer click behavior and recommendation performance (Koh and Cui 2022). In addition, research on image memorability suggests that visually distinctive thumbnails can increase viewer interaction and comment engagement (Peng and Bainbridge 2026). Therefore, the findings of the present study support and empirically validate the existing theoretical explanations of online content diffusion.

Practically speaking, the predictive framework developed in the present study can potentially be applied in a variety of ways. For example, the predictive estimates of expected view counts can help **allocate resources for content creation** by identifying videos that are likely to perform well and allocating promotion efforts accordingly. The results also provide **guidance for creators**, suggesting that creating a larger audience and designing visually appealing thumbnails are two of the most important factors affecting engagement outcomes. In addition, the model can provide **forecasts of engagement**, allowing stakeholders to estimate approximate view counts for newly uploaded videos. Furthermore, when paired with advertising metrics such as cost per thousand impressions (CPM), the predicted view counts can also be converted into financial forecasts, allowing organizations to estimate expected revenue or the amount of time saved by automatically triaging content.

## 7.3 Practicality and Extensions

A major benefit of the modeling framework developed in the present study is its practicality and reproducibility. The entire workflow can be run using standard CSV files combined with optional enriched data from the YouTube Data API. The final predictive model is a tuned Random Forest model, which provides excellent

predictive performance while being computationally efficient and interpretable through feature importance analysis. Since the model is based solely on structured metadata and not high-dimensional multimedia features, predictions can be produced rapidly and deployed in lightweight environments.

The framework can also be expanded in various ways to improve predictive performance or expand applicability. For example, the same methodology can be applied to other short-form or social video platforms like TikTok or Instagram to allow for comparative analyses of engagement dynamics across platforms. Another direction of expansion includes adding additional NLP features from video titles and descriptions, including sentiment analysis or topic modeling. Another possible direction of expansion includes adding time-series features that represent seasonality or popularity dynamics over time to improve modeling of content life cycle patterns. Finally, additional machine learning algorithms such as **XGBoost** or **LightGBM** can be evaluated, and stack ensembles of multiple models can be investigated to predict the outputs of different models.

## 8. Lessons Learned and Recommendations

### 8.1 Lessons Learned

Several key insights were discovered during the analysis and modeling phases. First, **the social capital of the creator** appears to be the strongest predictor of video engagement in the data set. Variables at the level of the creator that relate to the size and historical reach of their audience consistently dominated the feature importance rankings, suggesting that the networks of audiences play a significant role in determining the visibility of videos. While variables related to the presentation of content, such as attributes of the thumbnails, provided additional predictive value, they appeared to act as secondary signals relative to the structural exposure of the creator.

Second, the results of model comparisons suggest that **linear models such as Elastic Net** performed worse than **ensemble tree-based methods** for this task. This finding indicates that the association between the predictors and engagement outcomes are likely to be non-linear and interactive, consistent with theories of online attention and visual memorability discussed in the literature.

Third, the **K-Nearest Neighbors model** performed the worst of the models that were evaluated, indicating that video virality cannot be explained by simple similarity relationships among the videos in the current feature space. Instead, it appears that the outcomes of engagement are primarily dependent upon

structural platform dynamics.

Fourth, the **Multi-Layer Perceptron neural network** produced the poorest performance of all of the models that were evaluated. This finding is consistent with previous findings that neural networks tend to under-perform ensemble tree models when they are applied to structured tabular data that do not contain high-dimensional feature representations such as embeddings.

Finally, the modeling process illustrated the importance of **preventing data leaks** and ensuring **quality of the data**. By excluding likes and comments as predictors and strictly separating the training and test sets, we ensured that the model's performance reflected realistic prediction scenarios and not artificial inflation of accuracy.

## 8.2 Recommendations

Based on the findings of the present study, there are several recommendations for future work and for potential deployment. An immediate next step would be to create a single, executable workflow—such as a Python script or cloud-based training job—that integrates the entire data pipeline, thereby permitting automatic re-training and evaluation with minimal manual intervention. Cloud platforms such as AWS SageMaker would be suitable to schedule the re-training, manage the artifacts of the models, and support the deployment of the models in production.

From a modeling perspective, additional algorithms could be evaluated to further improve the predictive performance of the model. Gradient-boosting methods such as **XGBoost**, **LightGBM**, or **CatBoost** are particularly well-suited to structured tabular data and often perform better than standard implementations of Random Forest. **Bayesian regression models** could also be explored to provide uncertainty estimates for the predictions, while **quantile regression** methods could be used to generate prediction intervals that communicate the expected ranges of engagement, rather than single-point estimates.

Finally, the inclusion of additional external data sources could significantly improve the predictive power of the model. Some examples of additional data sources include **trend and search data** (e.g., Google Trends or keyword search volume) to capture emergent topical interests, **social network metrics** (e.g., cross-platform follower counts or collaboration networks) to capture the structure of collaborative activity, **demographic estimates of audiences** (if available) to capture differences in engagement among different groups, and **time-series data representing historical engagement** for channels or categories to capture seasonal or cyclic patterns of engagement. In addition, access to **thumbnail A/B**



**testing data or click-through rates** would permit more precise modeling of the effects of visual presentation on viewer behavior.

## 9. References

Adam, Mohammed, et al. 2023. "Predicting Online Video Engagement Using Machine Learning Techniques." *Journal of Digital Media Analytics*.

<https://doi.org/10.1016/j.ijresmar.2025.07.005>

Baravkar, Pratik, et al. 2023. "Content Characteristics and Engagement Dynamics in Social Media Video Platforms." *International Journal of Information Management*.

<https://doi.org/10.32628/cseit2172112>

Endait, Samuel, et al. 2025. "Machine Learning Approaches for Predicting Social Media Content Popularity." *IEEE Transactions on Computational Social Systems*.

<https://doi.org/10.1109/ICSIT65336.2025.11295060>

Gaur, Manas, et al. 2024. "Transformer-Based Models for Social Media Text Classification." In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 1234–1245. Association for Computational Linguistics.

<https://doi.org/10.1109/ICDCOT61034.2024.10515626>

Hussain, Tasnim, et al. 2024. "The Impact of Subscriber Count and Upload Frequency on YouTube Engagement: A Tiered Analysis of Views, Likes, and Comments." *Journal of Digital Media & Society* 12 (2): 45–67.

<https://doi.org/10.1016/j.chb.2024.108237>

Jing, Zhiwei, et al. 2024. "Modeling Online Video Popularity Through Engagement Metrics and Machine Learning." *Information Processing & Management*.

<https://doi.org/10.1109/TKDE.2017.2785784>

Koh, Joon Soo, and Cui Cheng. 2022. "Thumbnail Aesthetics and View-Through Rates: The Role of Brightness and Colorfulness in Click and Watch Behavior." *Communication Research* 49 (5): 1–24.

<https://doi.org/10.1016/j.dss.2022.113820>

Konukoğlu, Emre, and Özman, Mehmet. 2025. "The Influence of Video Technical Attributes on Online Engagement." *Multimedia Tools and Applications*.

<https://ijci.net/index.php/IJCI/article/view/1633>

Peng, Yuxin, and Wilma A. Bainbridge. 2026. "Memorability and Semantic Distinctiveness Predict Viral Potential of Images Independent of Emotional Valence." *Cognitive Science* 50 (1): 1–18.

<https://doi.org/10.1016/j.chb.2025.108799>

Cyberevil545. 2024. *YouTube Videos Data for ML and Trend Analysis*. Kaggle.

Dataset containing publicly available YouTube video metadata collected using the YouTube Data API v3.

<https://www.kaggle.com/datasets/cyberevil545/youtube-videos-data-for-ml-and-trend-analysis>.